

AUTOMATED GOALKEEPER PROJECT V.3

By: Rahiel Rodriguez, Independent Studies

Idaho State University

College of Technology

Robotics and Communication Systems Engineering Technology

Spring Semester, 2026

Prepared for: Shane Slack

Table of Contents

Abstract.....	6
Background.....	8
Proposed Problem.....	12
Environmental & Ethical Considerations.....	14
Block Diagrams.....	16
System Overview.....	17
Section 6.1: Web Camera.....	17
Section 6.2: PC Processing Stage.....	17
Section 6.3: Main Board and Motor Stage.....	18
Section 6.4: Goal Sensors.....	18
Section 6.5: Mechanical System.....	19
Main Board Schematics and PCB Layout.....	20
Section 7.1: Power Supply Section.....	20
Section 7.2: Serial Communication Section.....	23
Section 7.3: Microcontroller Section.....	24
Section 7.4: Home Proximity Sensors Section.....	27
Section 7.5: Flex and IR Motion Sensors Section.....	29
Section 7.6: Debug Section.....	33
Calibration Procedure.....	35
Troubleshooting Procedure.....	36
Section 9.1: Camera Troubleshooting.....	36
Section 9.2: Python & Assembly Troubleshooting.....	36
Section 9.3: Motor Operation Troubleshooting.....	36
Section 9.4: Proximity Sensors Troubleshooting.....	37
Section 9.5: Flex and IR Motion Sensors Troubleshooting.....	37
Software Description.....	39
Section 10.1: Python Code Description.....	39
Section 10.1.1: Ports and Camera Description.....	39
Section 10.1.2: Position Comparison & Transmission Description.....	40

Section 10.1.3: Camera Data Processing Routine Description.....	42
Section 10.1.4: Kalman Filter Description.....	43
Section 10.1.5: Kick Detection and Zone Prediction Description.....	44
Section 10.1.6: Manual Control Routine Description.....	46
Section 10.1.7: Ready and Homing Sequence Routine Description.....	46
Section 10.1.8: Shot Result Routine Description.....	48
Section 10.1.9: Calibration and Debug Console Description.....	49
Section 10.2: Assembly Code Description.....	49
Section 10.2.1: Data Received Routine Description.....	50
Section 10.2.2: Main Routine Description.....	51
Section 10.2.3: Pulse Routine and Speed Profile Description.....	52
Section 10.2.4: Sensor Result Routine Description.....	55
Section 10.2.5: PIC Homing Sequence Routine Description.....	55
Mechanical Drawings.....	58
Engineering Log.....	60
Cost Analysis.....	61
Conclusion/Recommendations.....	63
Section 14.1: Conclusions.....	63
Section 14.2: Status of the Objectives.....	63
Section 14.3: Known Issues and Recommended Solutions.....	64
Section 14.4: Future Work.....	64
Appendix A: Schematics.....	65
Appendix B: PCB Design.....	72
Appendix C: Python Code.....	73
Appendix D: Assembly Code.....	89
References.....	103

Table of Figures

Figure 2.1 Robokeeper Picture.....	8
Figure 2.2 Robokeeper V.1 Picture.....	9
Figure 2.3 Robokeeper V.2 Picture.....	10
Figure 2.4 Robokeeper V.3 Picture.....	11
Figure 5.1 Automated Goalkeeper V.3 System Block Diagram.....	16
Figure 7.1 Automated Goalkeeper V.3 Project Power Supply Schematic Section.....	21
Figure 7.2 Automated Goalkeeper V.3 Project +5 V Power Supply Section on PCB.....	21
Figure 7.3 Automated Goalkeeper V.3 Project +24 V Regulator Schematic Section.....	22
Figure 7.4 Automated Goalkeeper V.3 Project Fan Outputs Schematic Section.....	23
Figure 7.5 Automated Goalkeeper V.3 Project +48 V and +24 V Sections on PCB.....	23
Figure 7.6 Automated Goalkeeper V.3 Project Communication Schematic Section.....	24
Figure 7.7 Automated Goalkeeper V.3 Project Communication Section on PCB.....	24
Figure 7.8 Automated Goalkeeper V.3 Project Microcontroller Schematic Section.....	25
Figure 7.9 Automated Goalkeeper V.3 Project Microcontroller Section on PCB.....	26
Figure 7.10 Automated Goalkeeper V.3 Project Motor Driver Connector (J5) on PCB.....	26
Figure 7.11 Automated Goalkeeper V.3 Project Homing Proximity Sensors Schematic Section.....	28
Figure 7.12 Automated Goalkeeper V.3 Project Homing Proximity Sensors on PCB.....	29
Figure 7.13 Automated Goalkeeper V.3 Project Flex Sensor Amplifier Stage Schematic Section... 31	
Figure 7.14 Automated Goalkeeper V.3 Project Flex Sensor One-Shots Schematic Section.....	32
Figure 7.15 Automated Goalkeeper V.3 Project IR Motion Sensor Headers Schematic Section.....	32
Figure 7.16 Automated Goalkeeper V.3 Project Goal Sensors Circuits on PCB.....	33
Figure 7.17 Automated Goalkeeper V.3 Project Debug Schematic Section.....	34
Figure 7.18 Automated Goalkeeper V.3 Project Debug Section on PCB.....	34
Figure 10.1 Automated Goalkeeper V.3 Project Python Interface (layout drawing).....	39
Figure 10.1.1 Automated Goalkeeper V.3 Project Python Position Comparison Flowchart.....	41
Figure 10.1.2 Automated Goalkeeper V.3 Project Python Positions and Direction Byte Bitmap.....	42
Figure 10.1.3 Automated Goalkeeper V.3 Project Python Camera Data Processing Routine.....	43
Figure 10.1.4 Automated Goalkeeper V.3 Project Python Kick Detection State Machine.....	45
Figure 10.1.5 Automated Goalkeeper V.3 Project Python Goal Zone Map.....	46

Figure 10.1.6 Automated Goalkeeper V.3 Project Python Homing Sequence Routine Flowchart...	48
Figure 10.1.7 Automated Goalkeeper V.3 Project Python Shot Result Routine.....	49
Figure 10.2.1 Automated Goalkeeper V.3 Project Assembly Received Data Routine Flowchart.	51
Figure 10.2.2 Automated Goalkeeper V.3 Project Assembly Main Routine and Direction Change Flowchart.....	52
Figure 10.2.3 Automated Goalkeeper V.3 Project Assembly Pulse Routine Flowchart.....	53
Figure 10.2.4 Automated Goalkeeper V.3 Project Assembly Speed Profile (from the firmware values).....	54
Figure 10.2.5 Automated Goalkeeper V.3 Project Assembly Sensor Result Routine Flowchart..	55
Figure 10.2.6 Automated Goalkeeper V.3 Project Assembly Homing Sequence Flowchart.....	56
Figure 10.2.7 Automated Goalkeeper V.3 Project PORTA Bitmap.....	56
Figure 10.2.8 Automated Goalkeeper V.3 Project PORTB Bitmap.....	57
Figure 10.2.9 Automated Goalkeeper V.3 Project PORTC Bitmap.....	57
Figure 11.1 Automated Goalkeeper V.3 Project Main PCB Mechanical Drawings (KiCad layout with dimensions).....	58
Figure 11.2 Automated Goalkeeper V.3 Project Complete Structure.....	59
Figure 15.1 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 1/7: Top level (rotated 90 degrees).....	65
Figure 15.2 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 2/7: Power (rotated 90 degrees).....	66
Figure 15.3 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 3/7: Communications (rotated 90 degrees).....	67
Figure 15.4 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 4/7: PIC16F1788 (rotated 90 degrees).....	68
Figure 15.5 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 5/7: Debug (rotated 90 degrees).....	69
Figure 15.6 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 6/7: 24V Section (rotated 90 degrees).....	70
Figure 15.7 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 7/7: Goal Sensors (rotated 90 degrees).....	71
Figure 16.1 Main Board PCB Layout (KiCad).....	72

Abstract

The Automated Goalkeeper Project V.3 develops and enhances the prototype inspired by the RoboKeeper system and the improved Automated Goalkeeper Project V.2. The system, developed as an interactive robot goalkeeper, is centered on a 1080p web camera, a PIC16F1788 microcontroller, and a custom Python interface that tracks a moving ball, predicts the zone of the goal that the ball will reach, and moves a stepper-motor-driven goalkeeper along a lead screw to block it.

This iteration was directed towards optimizing software efficiency, improving real-time object tracking, adding a trajectory prediction algorithm (Kalman filter), building a more robust mechanical design and complete goal structure, and making the system more portable for public demonstrations. The key upgrades are: the replacement of the Pixy V2.1 camera by a web camera processed with OpenCV; the change from Visual Basic to Python; a Kalman filter that detects the kick and predicts the impact zone; a new custom PCB (Rev E.2) with a +24 V section, flex sensor conditioning, IR motion sensor headers and fan outputs; goal-line sensors that report whether the ball was blocked or scored; a new lead screw and 3D-printed carbon-fiber mechanism; and a simplified one-byte serial protocol with a trapezoidal speed profile generated by the microcontroller.

The system is functional: the goalkeeper reaches all nine positions and the structure withstood the ball being kicked with full force. However, the lower positions (1, 2, 8 and 9) still present problems holding the goalkeeper, and the block-rate tests are pending. Further development is needed to complete the mechanical integration, finish the integration of the goal sensors with the software and firmware, and validate the predictive tracking. The project is ultimately a practical educational platform for robotics and control systems rather than a professional or commercial sport solution.

Background

The Automated Goalkeeper V.3 Project is a continuation of the Automated Goalkeeper Project V.1, developed during the Fall 2024 semester, and the Automated Goalkeeper Project V.2, developed during the Spring 2025 semester, both made by Rahiel Rodriguez. This project takes inspiration from a pre-existing product known as the RoboKeeper, a soccer robot goalkeeper introduced in 2007 and being used today across the globe as a soccer phenomenon. The machines find their presence in a number of countries like Spain, England, Portugal, Argentina, and the United States. RoboKeepers are also commonly found at international events and at permanent establishments. They come in different sizes--from full to mini soccer varieties--and have been employed for other sporting activities as well, including hockey and handball.

Figure 2.1 Robokeeper Picture

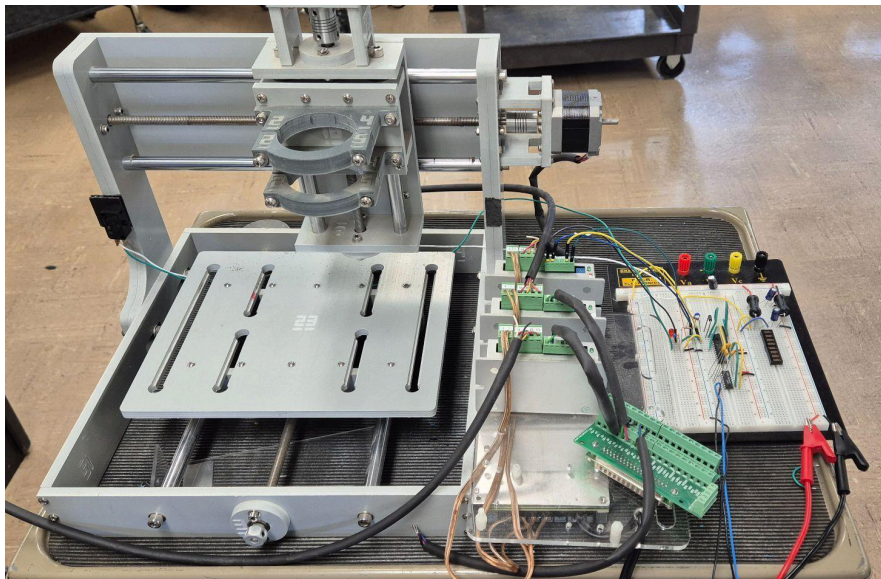


(4attention GmbH & Co. KG, n.d.)

The primary objective of the Automated Goalkeeper V.1 project, developed during the Fall 2024 semester, was to establish the bases for future development stages. This included the

integration of an Assembly program and a Visual Basic application to control a CNC 3D printer stepper motor via a PIC16F1788 microcontroller. A Pixy V2.1 camera was employed to detect the position of a colored object--in this case, a blue ball--by tracking its movement based on color recognition. The camera transmitted the positional data to a computer, where a Visual Basic program processed the pixel coordinates and converted them into corresponding step values. These values were then communicated to the PIC microcontroller, which generated the necessary pulse signals for the motor driver to execute the required motion.

Figure 2.2 Robokeeper V.1 Picture



(December 10, 2024)

The Automated Goalkeeper V.2 was developed during the Spring semester of 2025. The system used a Pixy V2.1 camera for real-time color-based ball tracking, a PIC16F1788 microcontroller for motor control, and a custom Visual Basic interface running on a PC to translate camera pixel coordinates into stepper motor commands that moved a goalkeeper figure along a goal line. Key upgrades in that version included a newly designed custom PCB, improved serial communication between the PC and microcontroller, a more powerful stepper

motor with a dedicated driver circuit, optical fiber proximity sensors for a homing sequence, and three operational modes (manual, automatic, and semi-automatic). The V.2 report identified the pending work: more efficient code, a longer lead screw, a redesigned goalkeeper mechanism, vector-based trajectory prediction and additional peripherals.

Figure 2.3 Robokeeper V.2 Picture



Finally, the Automated Goalkeeper V.3 was developed during the Spring semester of 2026 as an Independent Studies project. It addresses the pending work of V.2: the Pixy V2.1 camera was replaced by a 1080p web camera processed with OpenCV; the Visual Basic interface was rewritten in Python and now includes a Kalman filter that predicts the zone where the ball will reach the goal; the main board was redesigned (Rev E.2) to add a +24 V section, flex sensor conditioning, IR motion sensor headers and fan outputs; a new lead screw, a 3D-printed carbon-fiber mechanism and a complete goal structure were built; and the goalkeeper now receives feedback from the goal (blocked, goal or timeout) after every shot.

Figure 2.4 Robokeeper V.3 Picture



Proposed Problem

The objective of this phase of the Robotic Goalkeeper project is to enhance the second version of the project made in 2025 by implementing a series of hardware and software improvements to provide better performance, robustness, and mobility. The significant upgrades are: replacing the Pixy V2.1 camera with a 1080p web camera to improve resolution and correct tracking and data processing; changing the programming language from Visual Basic to Python; changing the screw rod attached to the stepper motor; developing a new motion mechanism in order to move the goalkeeper correctly; drawing and cutting the goalkeeper; developing an improved custom PCB; adding different sensors on the goal and goalkeeper; and designing a box that can protect the different parts of the project and make them portable. All components must be strategically positioned to minimize the risk of damage while maintaining portability for ease of transport and quick assembly at various event locations.

The major areas for enhancement are discussed below:

- **Software Optimization:** Designing a fresh software solution capable of processing the web camera data at a faster and more efficient speed to enable the goalkeeper to respond with top speed. It includes adjusting and optimizing the original Assembly code; changing from Visual Basic to Python to implement OpenCV in order to use the improved camera resolution and speed and create a more responsive and personalized color detection; and adding a Kalman filter to predict the direction the ball is going towards and send the corresponding position to the main board.
- **Mechanical Design:** Select appropriate materials and design a lightweight and impact-resilient goalkeeper figure to block soccer ball impacts; change to a shorter screw rod with more spacing in order to move the goalkeeper further with fewer rotations; and design and 3D print a functional mechanism that helps to lean the goalkeeper from side to side to block the ball, making it fast but robust so it does not fall.
- **Custom PCB development:** Develop an improved custom PCB that uses USB communication with a PIC16F1788, has a 24 V section for sensors and fans to cool the system, and has connections for all sensors.
- **Component Layout:** Locate electronic and mechanical components where they will not be vulnerable to potential ball impact yet remain readily accessible and stable during operation.

- **Portability:** Design the frame to be easy to dismantle and put together, with the system readily transportable to and installable at various events.
- **Sensor additions:** Add optical fiber sensors for the homing sequence; add a flex sensor at the bottom of the goalkeeper to detect impacts on the goalkeeper; and add a motion sensor at the top of the goal to detect movement from the ball if it passed the goalkeeper.

Environmental & Ethical Considerations

The advancement of robotics and artificial intelligence (AI) has sparked ongoing discussions about automation's impact on various industries, including sports. While some robotic technologies aim to replace human athletes, the Robotic Goalkeeper Project serves a different purpose. Instead of competing with professional goalkeepers, it is designed as an interactive tool for recruitment events and training exercises.

Some argue that robots may eventually replace human goalkeepers, referencing projects like Artemis, a humanoid soccer-playing robot developed by UCLA. According to TechWire Asia "Experts tout her abilities as surpassing those of Lionel Messi. Artemis, developed by researchers from UCLA, is among only three robots worldwide capable of running. She can also walk and jump, and her name -- an anagram for 'A Robot That Exceeds Messi In Soccer' -- hints at her extraordinary skills." The Robotic Goalkeeper differs significantly from projects like Artemis; it does not learn or adapt (its trajectory prediction is a fixed mathematical model), it cannot move beyond its fixed position, and it is limited to blocking penalties and free kicks. Given these constraints, it is unlikely to replace professional goalkeepers.

Another consideration is the economic factor. While some may view robotic goalkeepers as a cost-effective alternative to professional players, the long-term costs of maintenance and large-scale production could outweigh any savings. An article made by Sara Brown for the MIT Sloan School of Management website mentions that Daron Acemoglu, a Massachusetts Institute of Technology (MIT) professor, said: "Our evidence shows that robots increase productivity. They are very important for continued growth and for firms, but at the same time they destroy jobs and they reduce labor demand. Those effects of robots also need to be taken into account." The main purpose of robots is to increase productivity and make more money; however, the Robotic Goalkeeper is not designed for industrial efficiency but rather as an interactive tool for

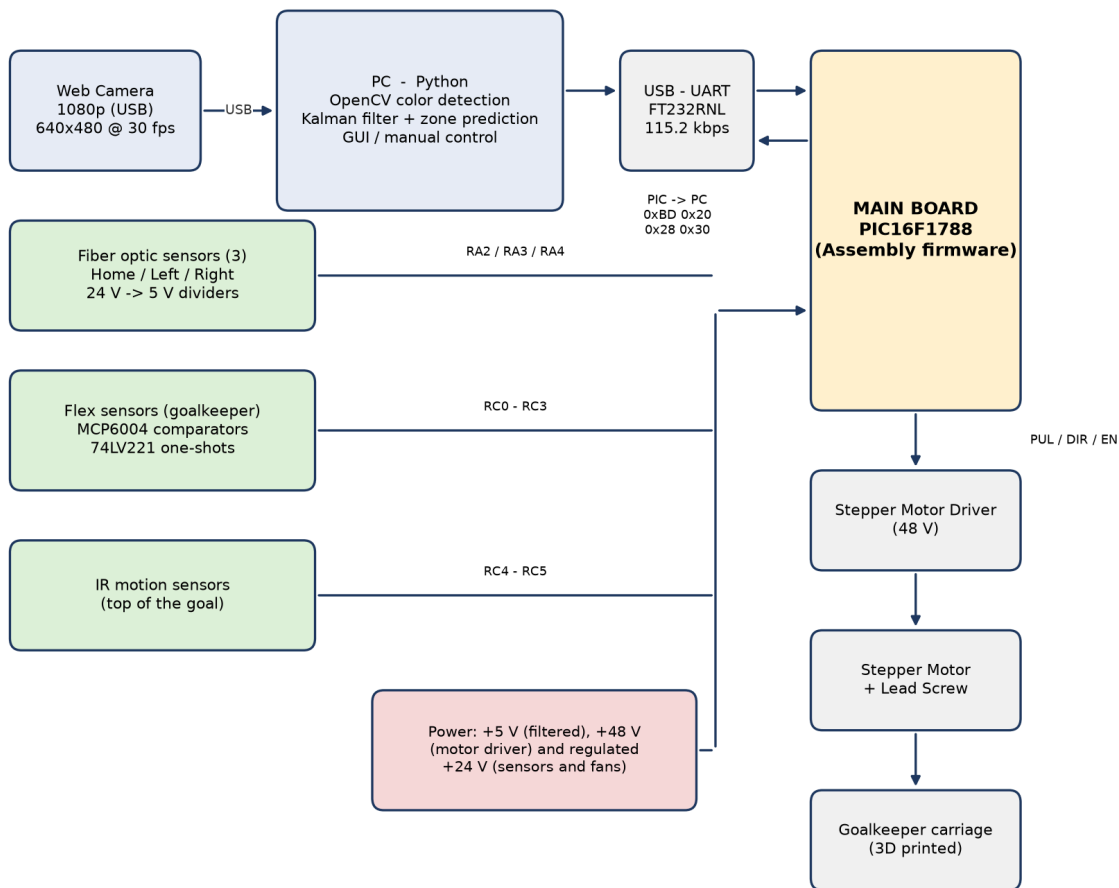
recruitment and training. At Idaho State University, it serves as an educational and training aid rather than a viable replacement for professional players.

One of the benefits that the Robotic Goalkeeper has is the fact that it can help to stimulate in a different way all the abilities and capabilities of the people. According to an article made by Eric Hamilton for the University of Florida News webpage, a study made by Amanda Studnicki, a veteran of varsity tennis in college, and Daniel Ferris, her advisor, showed that "When playing against another human, players' neurons worked in unison, like they were all speaking the same language. In contrast, when players faced a ball-serving machine, the neurons in their brains were not aligned with one another." Therefore, playing against a robot can stimulate senses, reactions and thoughts in order to get ready to play against it, creating a new challenge for the user. "What's clear is that our brains process these two experiences very differently, which suggests that training with a machine might not offer the same experience as playing against a real opponent." (Hamilton). In conclusion, the Robotic Goalkeeper Project raises important discussions about the role of automation in sports. Unlike AI-driven robots such as Artemis, it lacks mobility and learning capabilities, making it unsuitable for replacing professional goalkeepers. While cost considerations limit its practicality in professional soccer, it remains a valuable tool for recruitment and training, offering players a unique and interactive development experience.

Block Diagrams

Figure 5.1 shows the block diagram of the Automated Goalkeeper V.3. The web camera sends the video to the PC through USB. The Python program detects the ball, predicts the zone where it will reach the goal and sends one byte to the main board through the USB-UART converter. The PIC16F1788 generates the pulses and the direction signal for the stepper motor driver, which moves the goalkeeper along the lead screw. The sensors on the goal and on the goalkeeper are read by the PIC, which reports the result of each shot back to the PC.

Figure 5.1 Automated Goalkeeper V.3 System Block Diagram



System Overview

Section 6.1: Web Camera

The circuit starts with a 1080p USB web camera that replaces the Pixy V2.1 camera used in V.1 and V.2. The Pixy V2.1 did not provide enough resolution and made the correct tracking of the ball complicated, so the camera was changed to a web camera whose image is processed on the PC with OpenCV. The camera is placed approximately 10 cm from the floor and points at 90 degrees (completely frontal) towards the goal, so the image corresponds to the plane of the goal.

The Python program opens the camera through DirectShow and configures it to a resolution of 640 x 480 pixels at 30 frames per second, which is the frame used for the zone calculations. The ball is detected by color: the frame is converted to the HSV color space and a range of hue, saturation and value is applied to obtain a mask of the ball. The default range (hue 0 to 35, saturation 110 to 255, value 58 to 255) corresponds to an orange ball and can be recalibrated from the program (see Section 8).

Section 6.2: PC Processing Stage

The PC runs a Python program (customtkinter interface, OpenCV, cvzone, NumPy and pySerial) that processes the camera image every 16 milliseconds. For every frame the program finds the position of the ball, updates a Kalman filter that estimates its position, velocity and acceleration, and runs a state machine that waits for a kick. When a kick is detected, the trajectory is extrapolated until it reaches the border of the image and the goal zone (1 to 9) is calculated and transmitted to the microcontroller as a single byte.

The program has two ways of operation: a manual mode, where the position of the goalkeeper is selected by the user with the nine position buttons of the interface; and the

prediction mode (the evolution of the V.2 semi-automatic mode), where the computer keeps track of the ball until it starts moving, predicts where it will arrive and moves the goalkeeper to the right position. The automatic follow mode and the single sequence mode of V.2 were removed. The PC also handles the homing sequence through the Ready button and shows the result of every shot reported by the main board.

Section 6.3: Main Board and Motor Stage

The PC communicates with the main circuit board using USB communication that is converted to TTL serial communication by an FT232RNL. The microcontroller uses the byte received from the computer to know the direction it must go and the number of positions it must move. Using multipliers, it calculates how many pulses have to be sent to the stepper motor driver and generates them with a trapezoidal speed profile (acceleration, cruise and deceleration) to avoid losing steps. Finally, the stepper driver, which is powered with 48 V, converts those pulses to the signals used to energize each phase of the stepper motor, which turns the lead screw and moves the goalkeeper. When the movement ends, the microcontroller waits for the sensors of the goal and reports the result to the PC.

Section 6.4: Goal Sensors

Three types of sensors are connected to the main board. Three optical fiber proximity sensors (home, left and right) are used for the homing sequence, as in V.2. A flex sensor at the bottom of the goalkeeper detects the impact of the ball on the goalkeeper (the ball was blocked). An IR motion sensor at the top of the goal detects the movement of the ball if it passed the goalkeeper (goal). The signal of the flex sensor is small, so it is conditioned by a MCP6004 (which brings it to a voltage that can be detected reliably) and by a 74LV221 one-shot, which keeps the signal active for a longer time so the microcontroller does not miss it. The integration

of the flex and IR motion sensors with the software and the hardware is still in progress, since the complete tests have not been done. Section 7 describes the circuits.

Section 6.5: Mechanical System

The horizontal movement of the goalkeeper is produced by a lead screw driven by the stepper motor. A new screw rod, shorter and with more spacing (lead), was selected so the goalkeeper travels further with fewer rotations. The goalkeeper is mounted on a 3D-printed carriage that moves along the screw, and the mechanism that holds and leans the goalkeeper was designed and printed with carbon-fiber-filled filament. The whole assembly is mounted on a base at the bottom of a complete goal structure (see the picture of the project in Chapter 2). The mechanical parts are listed in Chapter 11.

Main Board Schematics and PCB Layout

The main board of the Automated Goalkeeper V.3 (Rev E.2) was designed in KiCad 9.0.1 and is organized in six schematic sheets: Power, Communications, PIC16F1788, Debug, 24V Section and Goal Sensors (the complete sheets are in Appendix A). The board measures 7.45 x 5.3 inches (see Chapter 11) and has four mounting holes. The following sections describe every section of the board and show its schematic and its location on the PCB.

Section 7.1: Power Supply Section.

Figure 7.1 shows the power supply section of the main board. On the top there is a +5 V power supply filter that uses two 470 uF capacitors (C1, C2) and two 10 uH inductors (L1, L2) to filter the noise coming from the power supply into the circuit. The +5 V rail has more filtering due to its direct connection to the microcontroller: if the PIC16F1788 gets noise from the power supply it will affect its operation and can cause problems. On the bottom, the +48 V input (J2) has a 100 uF capacitor (C3) and a 0.1 uF capacitor (C4) to filter out the noise. Both rails have an indicator LED (D1 for +5 V and D2 for +48 V) and a test point. Figure 7.2 shows the location of the +5 V section on the PCB.

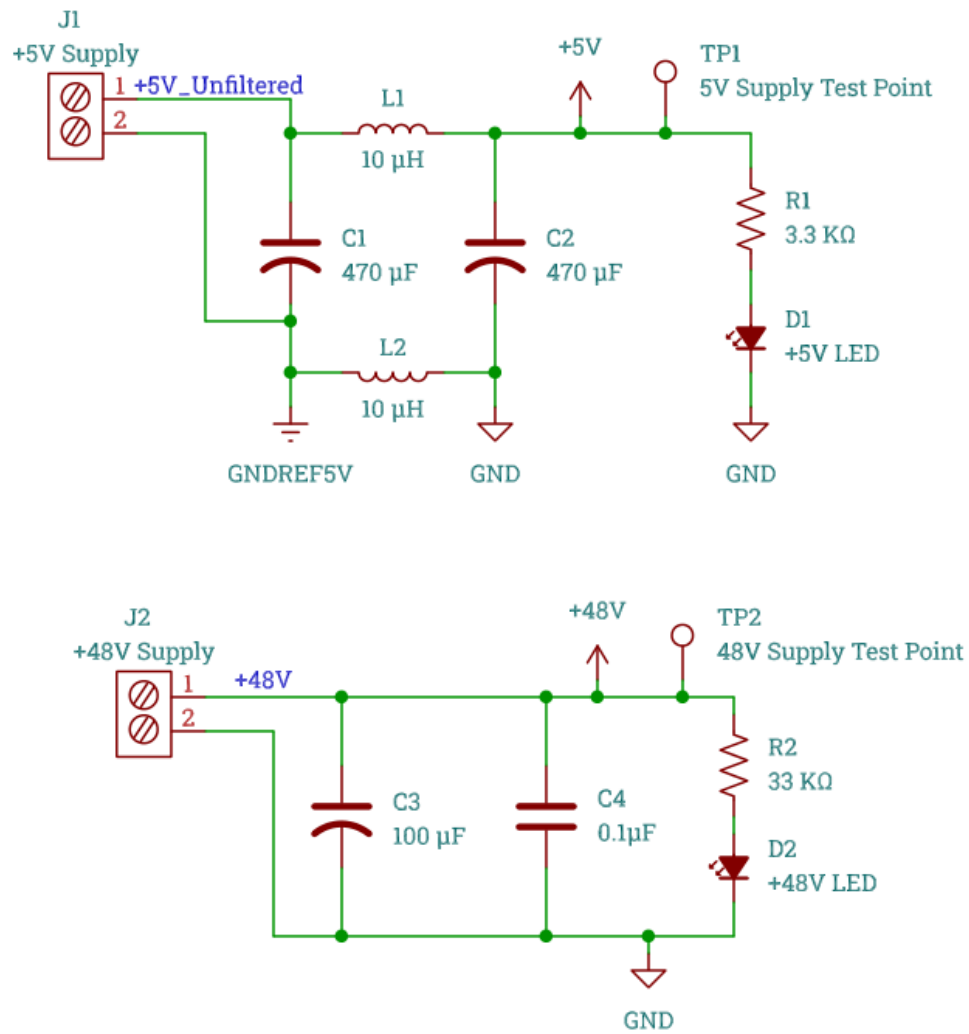
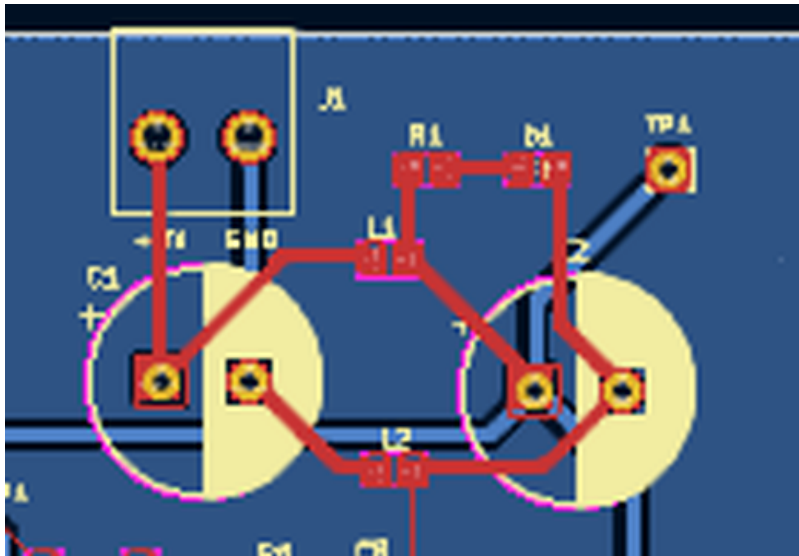
Figure 7.1 Automated Goalkeeper V.3 Project Power Supply Schematic Section

Figure 7.2 Automated Goalkeeper V.3 Project +5 V Power Supply Section on PCB



The +24 V section (sheet 24V Section) is new in V.3 and supplies the sensors and the cooling fans. Figure 7.3 shows that the +48 V input is reduced with a linear regulator built with an NJD35N04 power transistor (Q1) used as a pass element: a 23 V zener diode (D15) with a 2.2 kohm resistor (R8) sets the base voltage, and a 25 ohm, 50 W resistor (R9) takes part of the power from the +48 V input. The output is filtered by two 10 uF capacitors (C20, C21) and is available as the "24V Output" net with a test point (TP10). This rail feeds the optical fiber sensors, the IR motion sensors and the two fan connectors (J12 and J13), shown in Figure 7.4. The +48 V rail and this regulator are located at the top right of the board, as shown in Figure 7.5. The stepper motor itself is powered with 48 V: the 48 V supply goes to the stepper motor driver, which is external to the board, and from the driver to the motor; the board only sends the pulse, direction and enable signals to the driver through J5.

Figure 7.3 Automated Goalkeeper V.3 Project +24 V Regulator Schematic Section

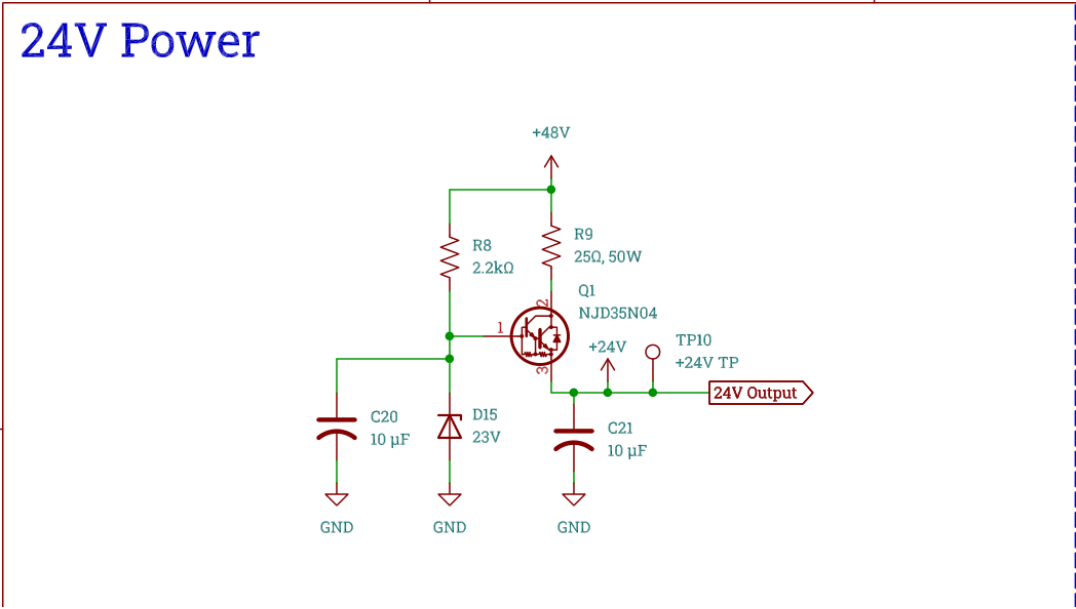


Figure 7.4 Automated Goalkeeper V.3 Project Fan Outputs Schematic Section

Fans

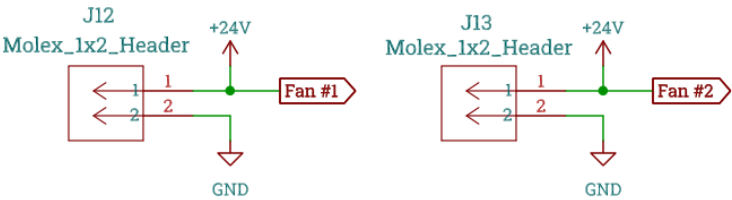
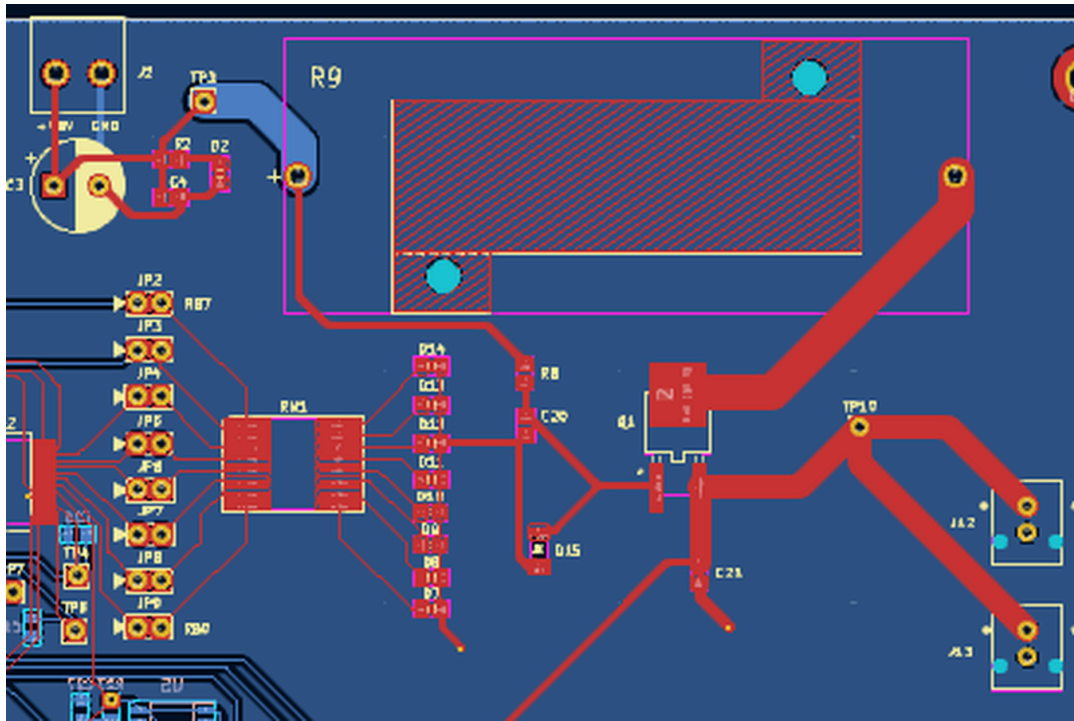


Figure 7.5 Automated Goalkeeper V.3 Project +48 V and +24 V Sections on PCB



Section 7.2: Serial Communication Section.

Figure 7.6 shows the PC-Microcontroller communication stage of the main board. A micro-USB connector (J3) goes into a FT232RNL USB to TTL converter (U1). The purpose is to use USB communication instead of RS-232 to transmit and receive information between the PIC16F1788 and the dedicated computer used for this project. The converter also has two LEDs (D3 and D4) with 270 ohm resistors to indicate when transmission or reception of information is occurring, a jumper (JP1) to select the power source, and test points for the TX and RX lines. The TXD line of the FT232RNL is connected to RC7 (the receiver pin of the PIC) and the RXD line to RC6 (the transmitter pin of the PIC). Figure 7.7 shows this section on the PCB.

Figure 7.6 Automated Goalkeeper V.3 Project Communication Schematic Section

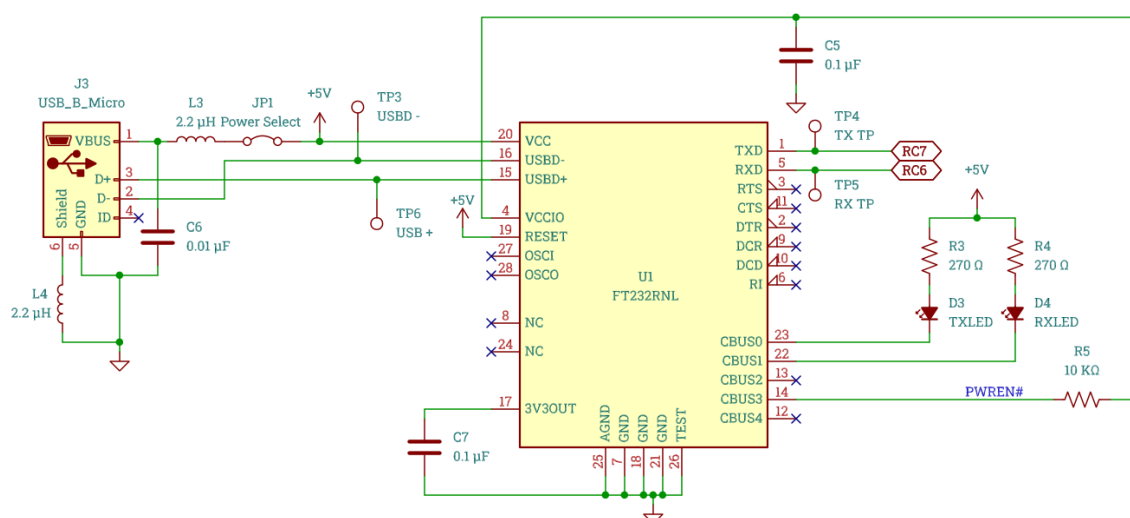
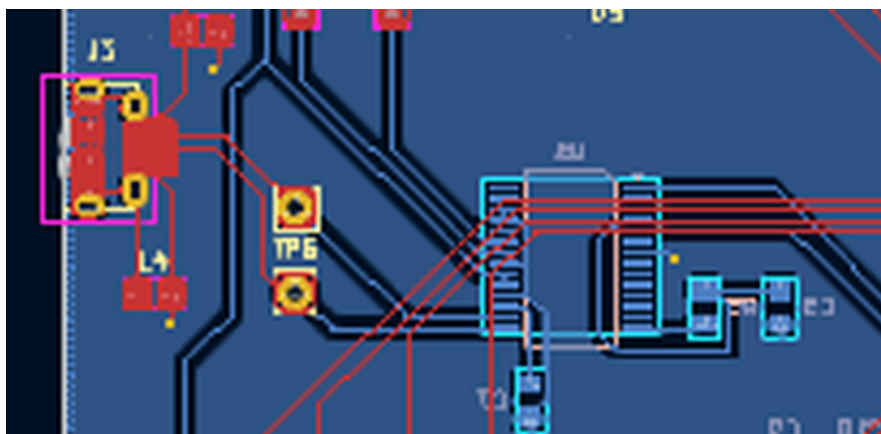


Figure 7.7 Automated Goalkeeper V.3 Project Communication Section on PCB



Section 7.3: Microcontroller Section.

The PIC16F1788 microcontroller is connected to the +5 V filtered power supply (Figure 7.8). The MCLR pin is connected to a 100 kohm resistor (R6) in series with a 22 uF capacitor (C8) and it is connected to the pin through a 1N4001 diode (D5). This side of the circuit gives a soft start to the microcontroller when power just gets applied. A 6-pin header (J4) is connected to the microcontroller in order to connect the programmer (PICKit) to the board. Each pin of PORTC has a 1 uF capacitor to ground (C9 to C16) to filter noise.

The output PUL+ pin (PORTA, bit 0) of the microcontroller sends a square wave signal to the pulse input of the stepper motor driver to make the motor move, and the DIR+ pin (PORTA, bit 1) determines the direction of the motor by being high or low. These signals, together with the enable signal (generated with a diode, a 100 kohm resistor and a 22 uF capacitor, R7, D6 and C19) and their filter capacitors (C17 and C18), are connected to a 6-pin screw terminal (J5) that goes to the motor driver. The information received from the computer and sent to it is handled at PORTC, where RC6 is used for transmitting and RC7 for receiving. Figure 7.9 shows this section on the PCB, and Table 7.1 summarizes the use of the pins.

Figure 7.8 Automated Goalkeeper V.3 Project Microcontroller Schematic Section

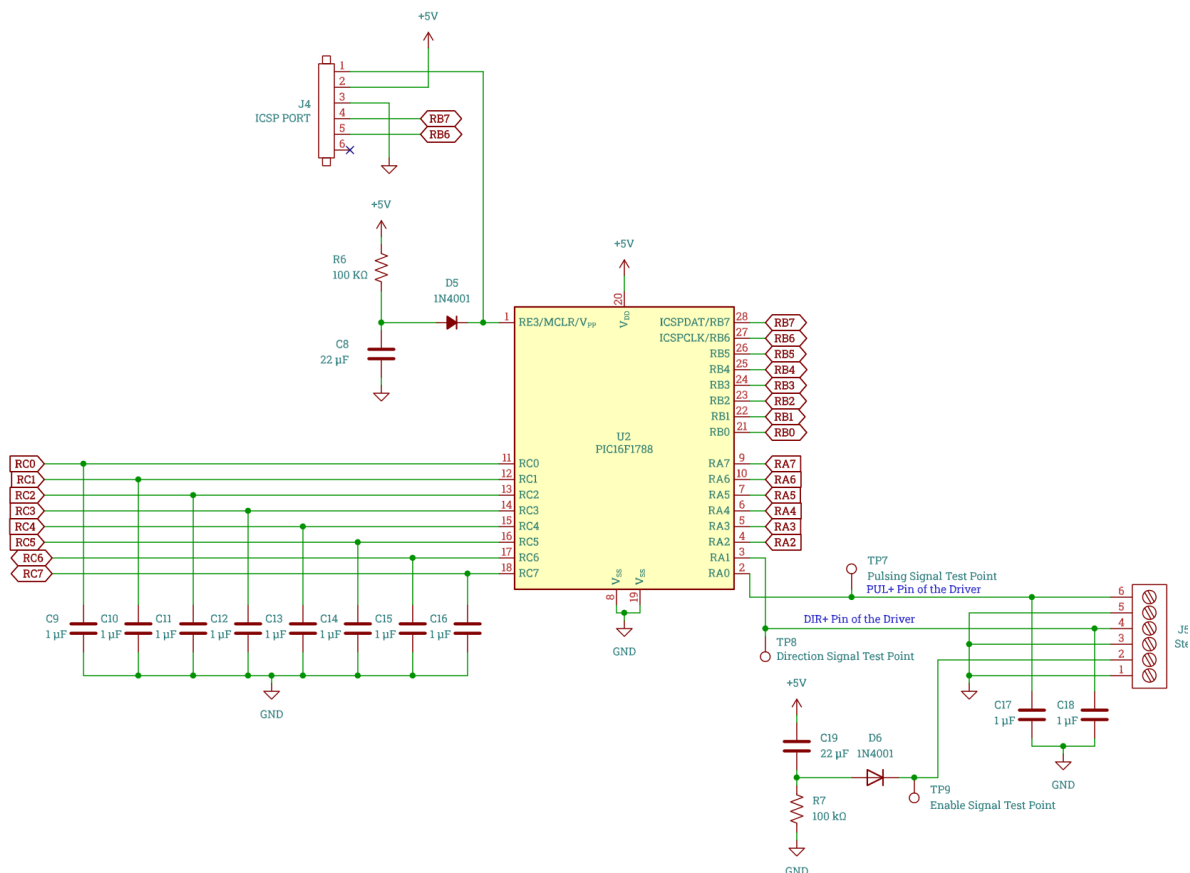


Figure 7.9 Automated Goalkeeper V.3 Project Microcontroller Section on PCB

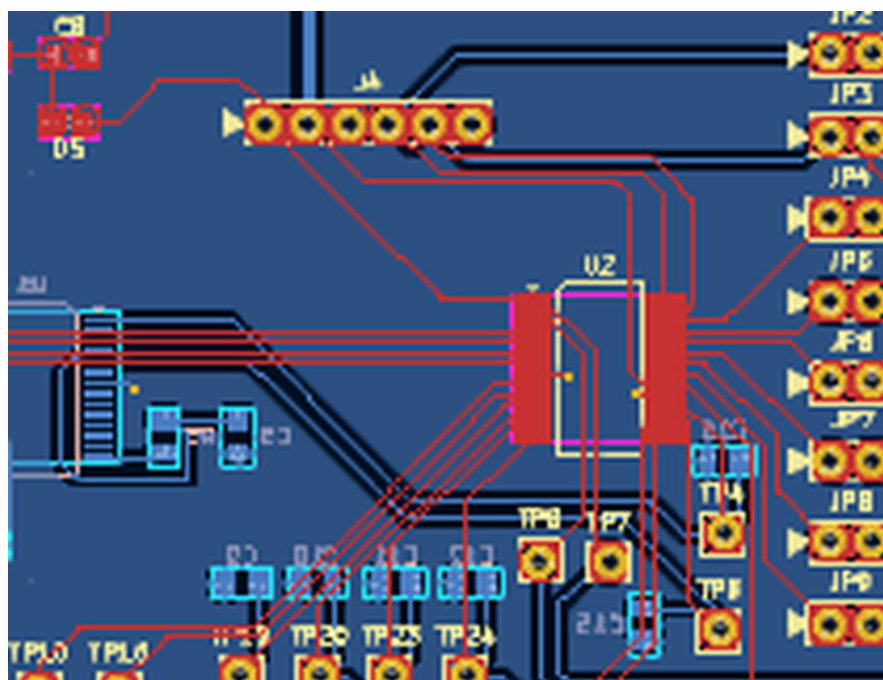


Figure 7.10 Automated Goalkeeper V.3 Project Motor Driver Connector (J5) on PCB



Table 7.1 PIC16F1788 pins used by the firmware

Pin	Direction	Function in the firmware
RA0	Output	PUL+ : step pulses to the motor driver
RA1	Output	DIR+ : direction (0 / 1) to the motor driver
RA2	Input	Home optical fiber sensor
RA3	Input	Left optical fiber sensor (change direction)
RA4	Input	Right optical fiber sensor (change direction)
RB0	Output	Debug LED: flex sensor detected
RB2	Output	Debug LED: waiting for a sensor (blinks)
RB4	Output	Debug LED: motion sensor detected
RC0	Input	Flex sensor (goalkeeper) - reported as BLOCKED
RC1	Input	Motion sensor - reported as GOAL
RC6	Output	UART TX to the FT232RNL
RC7	Input	UART RX from the FT232RNL

Section 7.4: Home Proximity Sensors Section.

For the homing sequence of the motor -centering the goalkeeper- 3 optical fiber proximity sensors are used (Figure 7.11). These sensors use 2 optical fiber cables, one that sends light out, and the other that receives that light back when it is reflected. When the light is received back, the sensor sends out a 24 V high or low signal. Because the PIC16F1788 uses TTL logic, a voltage divider (20 kohm and 5.1 kohm) is used to send only about 5 V to the microcontroller. The Left (J7, RA3) and Right (J8, RA4) sensors change the direction in which the motor is moving in order to move the goalkeeper to the center of the goal, and the Home sensor (J6, RA2) detects when the goalkeeper is in the middle. Three additional sensor modules (J9, J10 and J11, connected to RA5, RA7 and RA6) were left available on the board, but the current firmware does not use them. Figure 7.12 shows these connectors on the PCB.

Figure 7.11 Automated Goalkeeper V.3 Project Homing Proximity Sensors Schematic Section

Fiber Optics Sensors

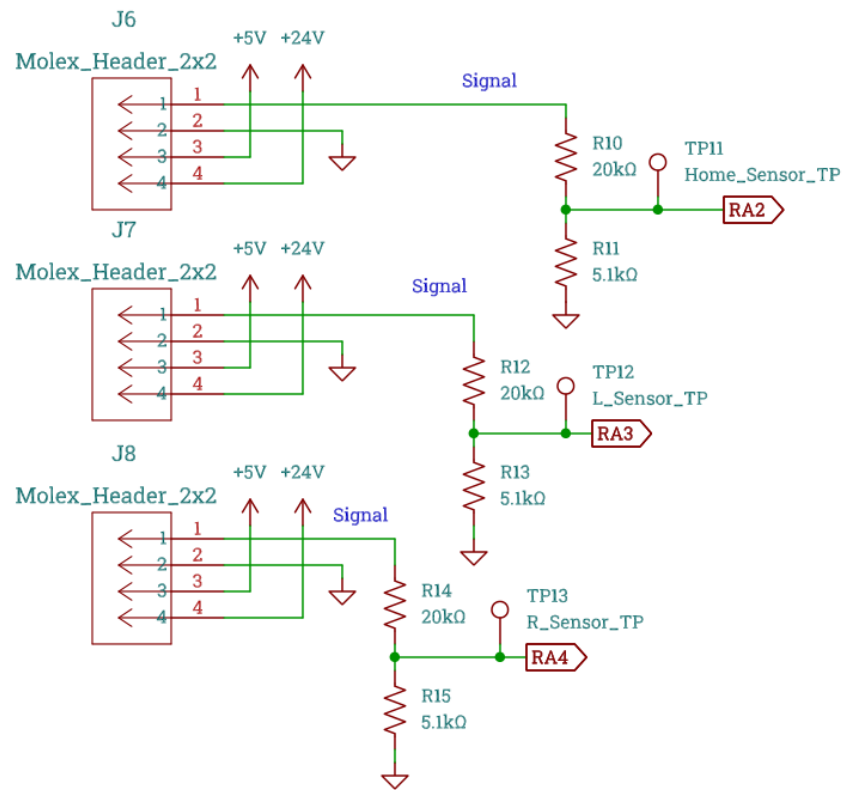
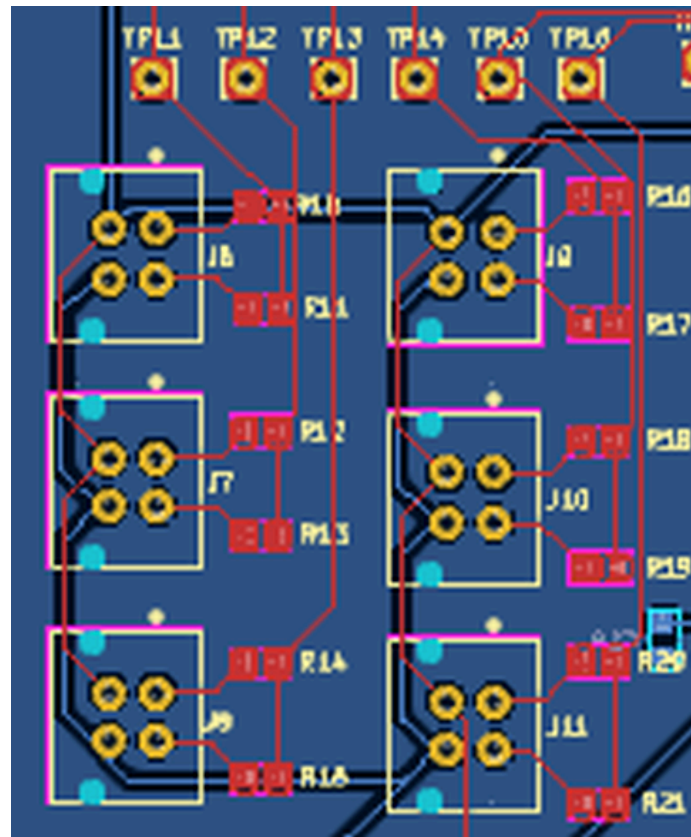


Figure 7.12 Automated Goalkeeper V.3 Project Homing Proximity Sensors on PCB



Section 7.5: Flex and IR Motion Sensors Section.

This section is new in V.3 and detects the result of a shot. **Flex sensor amplifier stage:** each of the four flex sensor connectors (J14 to J17) forms a voltage divider with a 24 kohm resistor and a 0.1 uF capacitor. The voltage is compared by a channel of the MCP6004 quad operational amplifier (U4A to U4D) with a reference set by a 10 kohm trimmer (RV1 to RV4), so the small change of the flex sensor when it bends is converted into a clean logic level and the sensitivity can be adjusted (Figure 7.13). **One-shots:** the output of every comparator goes to a 74LV221 monostable multivibrator (U3 and U5). Each one-shot uses a 46 kohm resistor and a 22 uF capacitor, which keeps the signal high for a time in the order of a second (approximately $0.7 \times R \times C = 0.7$ s). This keeps the impact signal long enough to be read and prevents the microcontroller from missing it (Figure 7.14). The outputs of the four one-shots are connected to

RC0, RC1, RC2 and RC3. **IR motion sensors:** two 4-pin headers (J18 and J19) supply +5 V and +24 V to the IR motion sensors and bring their signal to RC4 and RC5 (Figure 7.15). Figure 7.16 shows all of these circuits on the PCB.

Note: the firmware version 3 configures RC2 to RC5 as outputs and only checks RC0 (as the flex sensor) and RC1 (as the motion sensor). The flex and IR motion sensors are not completely integrated with the software and the hardware yet, because the complete tests have not been carried out; the current firmware works with the inputs RC0 and RC1. Using the four flex channels and the two IR headers as routed on Rev E.2 requires a small update of the firmware (see Section 14.3).

Figure 7.13 Automated Goalkeeper V.3 Project Flex Sensor Amplifier Stage Schematic Section

Flex Sensor Amp Stage

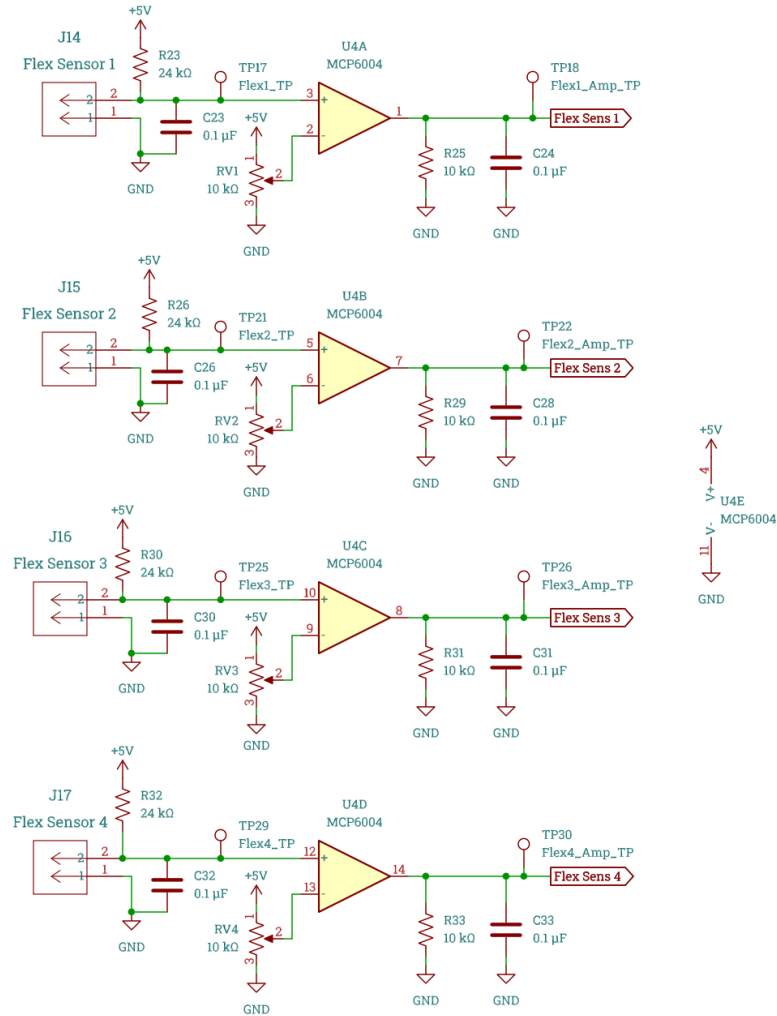


Figure 7.14 Automated Goalkeeper V.3 Project Flex Sensor One-Shots Schematic Section

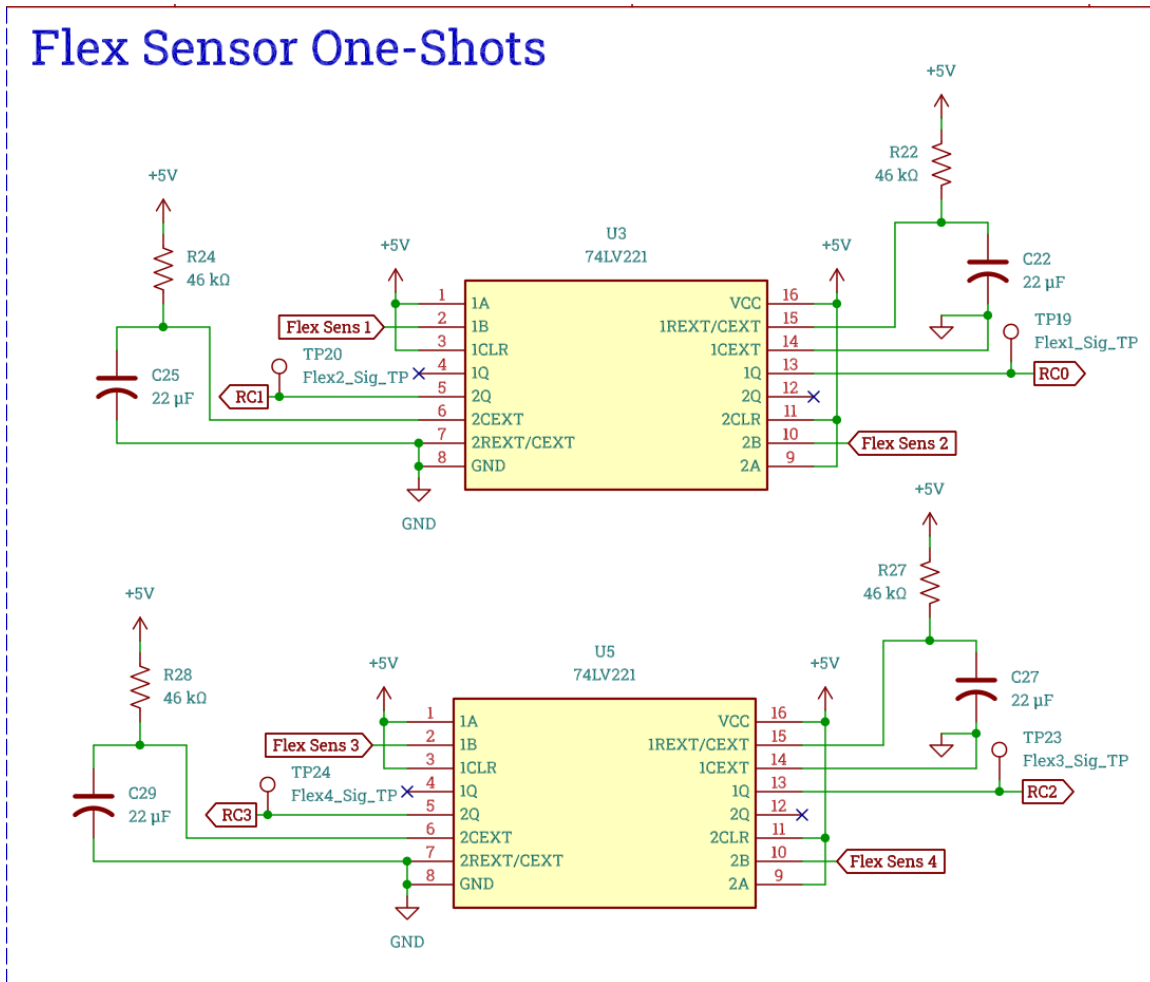


Figure 7.15 Automated Goalkeeper V.3 Project IR Motion Sensor Headers Schematic Section

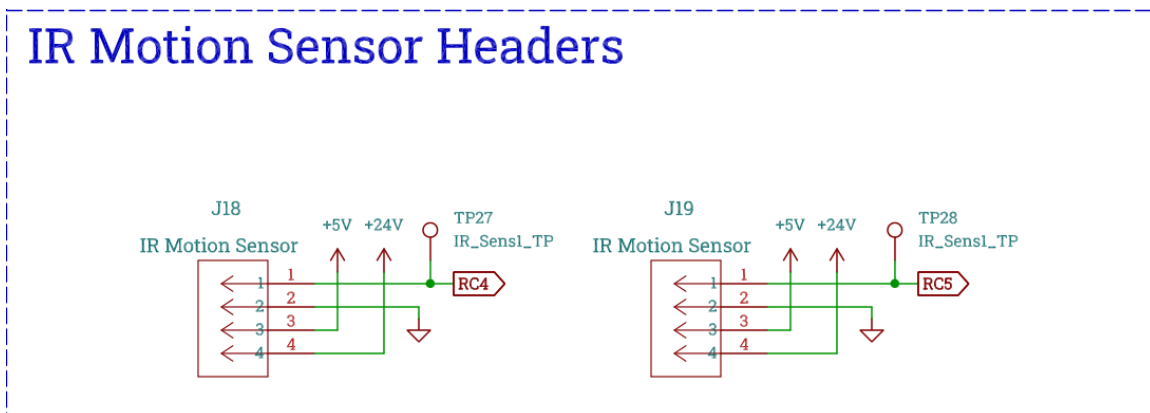
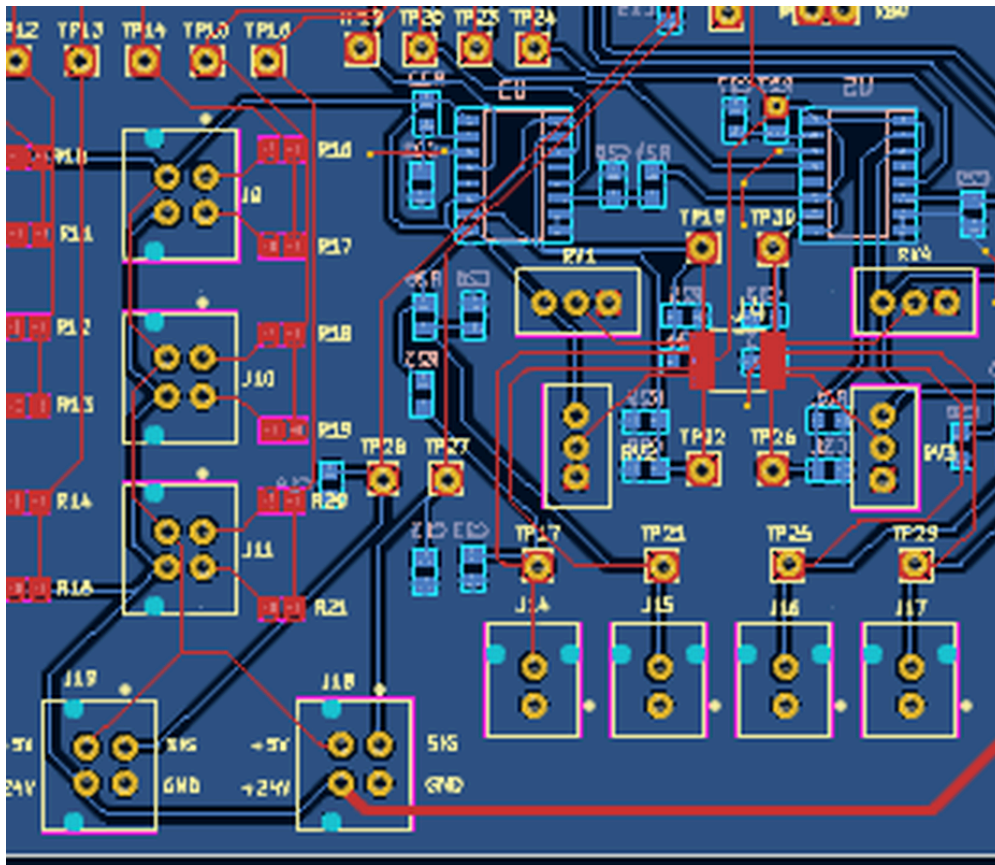


Figure 7.16 Automated Goalkeeper V.3 Project Goal Sensors Circuits on PCB



Section 7.6: Debug Section

PORTB of the PIC16F1788 is used for debugging operation. It is normally connected through the jumpers JP2 to JP9 to a 330 ohm resistor network (RN1) and eight LEDs (D7 to D14, "Debug 0" to "Debug 7"), as shown in Figure 7.17. If it is needed to test some function or solve some problem of the system, the PIC can be reprogrammed to use PORTB for tests using LEDs as indicators for the needed operation. The jumpers must be removed when PORTB is used for alternate functions. The current firmware uses RB0 (flex sensor detected), RB2 (blinks while waiting for a sensor) and RB4 (motion sensor detected). Figure 7.18 shows the debug section on the PCB.

Figure 7.17 Automated Goalkeeper V.3 Project Debug Schematic Section

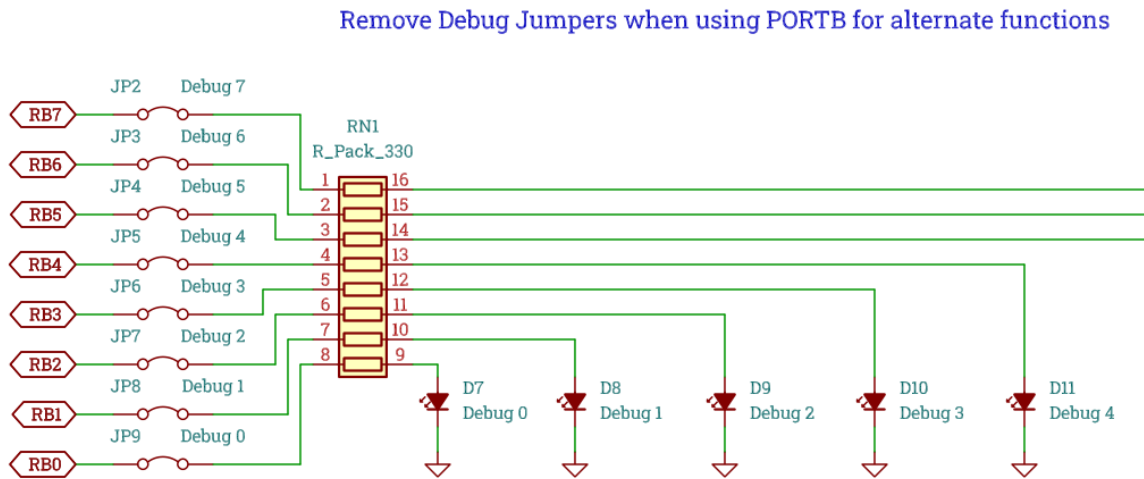
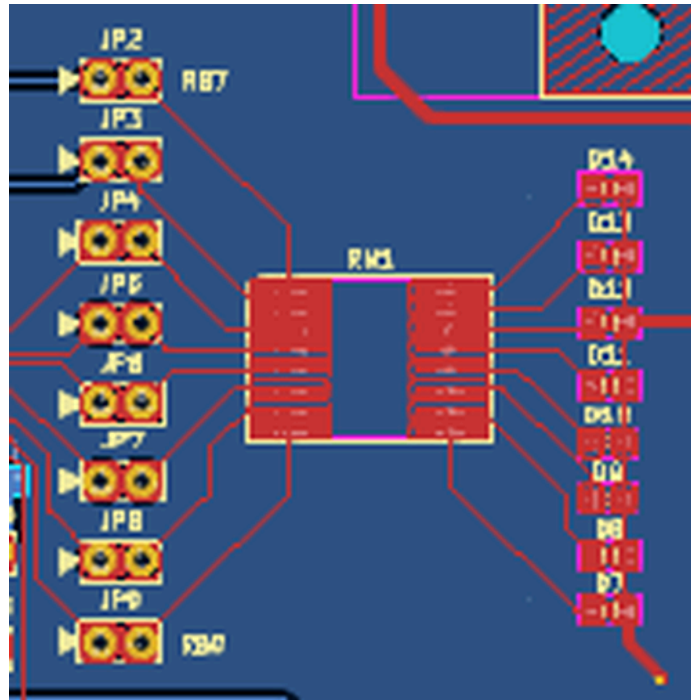


Figure 7.18 Automated Goalkeeper V.3 Project Debug Section on PCB



Calibration Procedure

In order to calibrate the Automated Goalkeeper V.3 for correct operation it is needed to:

1. Turn on the power supplies (+5 V and +48 V) and verify that the +5 V and +48 V LEDs of the board are on. Verify that the +24 V test point (TP10) has voltage.
2. Connect the web camera, place it approximately 10 cm above the floor pointing frontally to the goal, and connect the USB cable of the main board. Open the Python program.
3. Select the serial port of the main board in the "PIC Port" drop-down list (press "Update Ports" if it does not appear) and the camera in the "X Camera Port" list. The live image with the zone grid must appear on the screen.
4. Calibrate the color detection: press "Camera Calibration" (or Ctrl+Y). Adjust the six trackbars (Hue, Saturation and Value, minimum and maximum) until only the ball is white in the mask. Press Q in the OpenCV window to save the values. The interface is blocked until Q is pressed.
5. Use the manual control buttons (Position 1 to 9) to verify the correct behavior of the motor and the goalkeeper in every position. This must be done before pressing Ready, because the manual buttons are disabled while the system is homing.
6. Press the Ready button to start the homing sequence. The button shows "Homing..." and turns green "Ready" when the main board confirms that the goalkeeper is in the center (byte 0xBD).
7. Adjust the trimmers RV1 to RV4 of the board until the flex sensor is triggered by an impact on the goalkeeper (check the test points of the flex amplifier and one-shot outputs), and verify that the IR motion sensor detects the ball when it enters the goal.
8. Kick the ball and verify that the state changes from "WAITING FOR KICK" to "BALL TRACKING", that the goalkeeper moves to the predicted zone and that the result (BLOCKED, GOAL or NO SENSOR) is shown on the button. Press Ready to arm the system again.
9. It is ready for operation. Use Ctrl+R (Reset Kalman) if the tracking gets confused, and Ctrl+D to open the debug console.

Troubleshooting Procedure

Section 9.1: Camera Troubleshooting.

- If the program shows "No Cameras" or the image is black, disconnect and connect back the USB cable, or change it for a new one. Verify that no other program is using the camera and select another index in the "X Camera Port" list. If nothing works, replace the camera.
- If the ball is not detected or the detection is unstable, repeat the color calibration (Section 8). Add extra light in order to get rid of shadows, and remove objects with a color similar to the ball from the image. Only objects with an area larger than 1000 pixels are considered.
- If the system detects false kicks while the ball is still, verify that the ball is not moving in the image and that nothing with the color of the ball crosses the frame. The kick threshold (KICK_VELOCITY_THRESHOLD) and the minimum movement (MIN_MOVEMENT) can be increased in the code.
- If the goalkeeper moves to a wrong position, verify that the camera has not moved: the zone grid assumes that the image is 640 x 480 pixels and that the camera is frontal to the goal.

Section 9.2: Python & Assembly Troubleshooting.

- If any routine, such as the homing sequence or manual movement, does nothing, check that the correct port is selected in the "PIC Port" list and that the message "Serial port not open" is not shown in the console. Check the RX and TX LEDs of the board. If they do not blink, check the USB cable and the FT232RNL drivers. If the LEDs blink but nothing happens, check the power of the PIC. If power is on and going into the PIC, check if the code got corrupted and needs to be programmed again. If these solutions do not work, replace the part.
- If the message "Zone X predicted but system not armed" appears, press the Ready button and wait for the homing to finish. The system only transmits predictions after the main board confirms the home position.
- If the Ready button stays in "Homing...", check the home sensor (Section 9.4) and the power of the +24 V section. The console shows the bytes received from the PIC.
- If the goalkeeper stops in a position that is different from the one shown by the program (for example after being pushed by a ball), press Ready to home the goalkeeper again. The program assumes that the goalkeeper is at the last transmitted position.

Section 9.3: Motor Operation Troubleshooting.

- If the motor does not move, check that it has power and the stepper driver alarm is not on. If power is being applied and current is not too high, then do all troubleshooting processes in order to check if any part of the communication is failing. If all communication is working properly, verify that the square wave signal goes out of the PIC16F1788 (PUL+ test point

TP7). If the signal is running properly, check if the motor is making a high pitch sound when it tries to move; this means that the frequency is too high and needs to be slowed down. If the frequency is fine for the motor and it is still not moving, verify that all its cables are doing connection correctly. If connections are correct and it is still not moving, replace the motor.

- If the motor moves in the opposite direction to the desired one, check that the direction bit (most significant bit) of the byte transmitted from the computer is being sent correctly. If it is correct, check that the direction output (RA1, test point TP8) toggles. If the assembly code operates correctly, swap the order in which the cables are connected into the driver to check if phases are connected backwards.
- If the motor does not change direction, check software operation as indicated previously, and if nothing works, replace the stepper driver because it indicates that it is failing.
- If the motor loses steps or stalls at high speed, reduce the speed values of the ramp in the assembly code (Section 10.2.3) or check the current setting of the driver.
- If the lower positions (1, 2, 8 and 9) cannot hold the goalkeeper, see the known issues of Section 14.3.

Section 9.4: Proximity Sensors Troubleshooting.

- If the proximity sensors do not send out the 24 V signal when the light is reflected back into the fiber optic cable, first check if power is going correctly to it (+24 V test point TP10). If power is on and getting into the sensors, change the aluminum tape or reflecting object for a wider one to ensure light reflection. If nothing of this works, replace the part.
- If the proximity sensor signal going into the PIC16F1788 is not 5 V, check the voltage of the signal coming out of the sensor. The sensors are capable of working with a voltage going from 12 V up to 24 V, and if the voltage used is lower, the voltage going out of the voltage dividers will be lower. If the voltage that is going out of the sensors is correct, check the resistor values of the voltage dividers (20 kohm and 5.1 kohm) to know if they are correct or need to be replaced.

Section 9.5: Flex and IR Motion Sensors Troubleshooting.

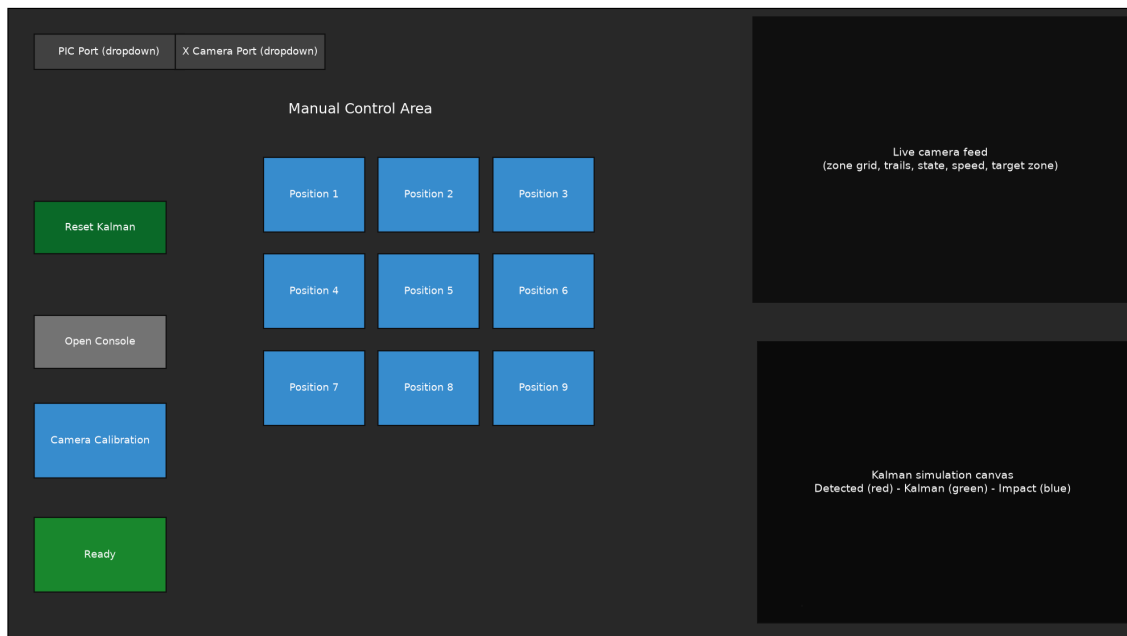
- If the program always answers "NO SENSOR" after about 5 seconds, no sensor was detected. Check the test points of the flex amplifier and of the one-shot outputs while flexing the sensor. Turn the trimmers RV1 to RV4 to increase the sensitivity and verify the connections of the sensor.
- If the program reports "BLOCKED" or "GOAL" without a real event, reduce the sensitivity of the trimmers, and check that the flex sensor is not bent by its own cable or by the vibration of the goalkeeper.
- If the IR motion sensor is not detected, verify its +5 V and +24 V supply and its connection. Verify that the firmware reads the input where the sensor is connected (see Section 7.5).

Software Description

Section 10.1: Python Code Description

The Visual Basic program of V.2 was replaced by a Python program (Main Code, listed in Appendix C). The program uses customtkinter for the interface, OpenCV and cvzone for the image processing, NumPy for the Kalman filter and pySerial for the communication with the main board. It is organized in the following blocks: application setup, Kalman filter and state machine, zone functions, simulation canvas, camera functions, calibration, main camera loop, serial/motor functions, debug console and graphical interface. The layout of the interface is shown in the figure below.

Figure 10.1 Automated Goalkeeper V.3 Project Python Interface (layout drawing)



Section 10.1.1: Ports and Camera Description

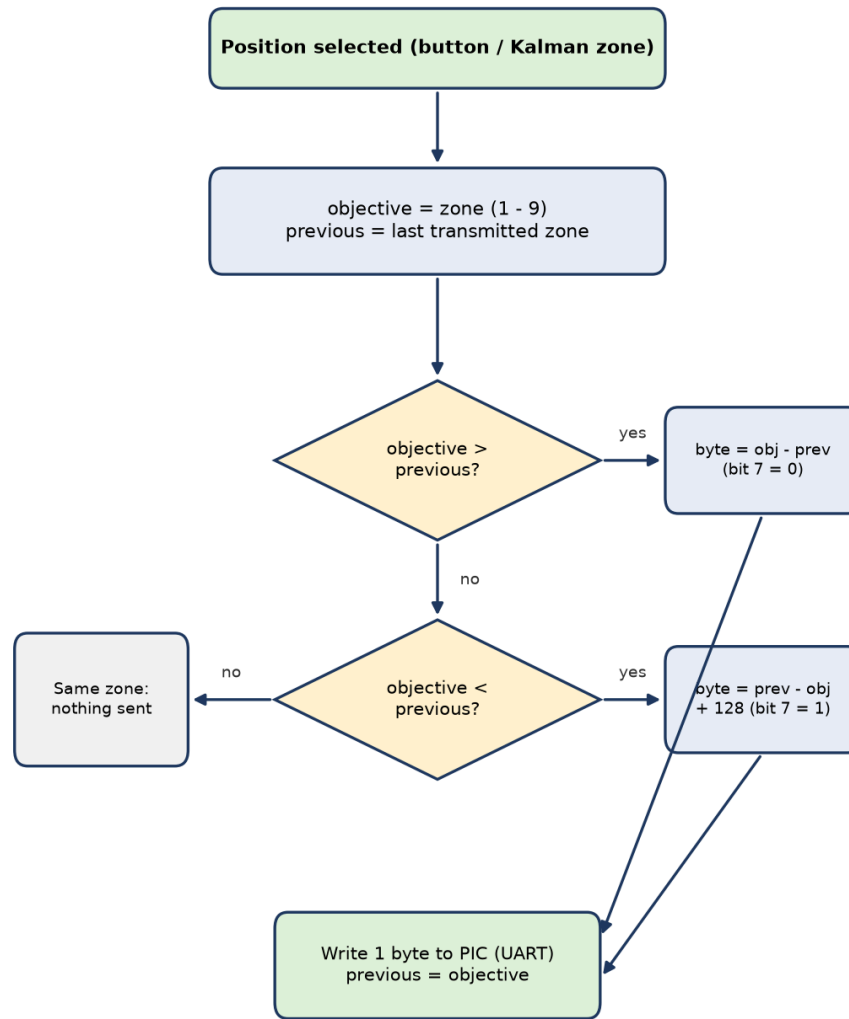
When the program starts, it scans the serial ports of the computer, loads them in the "PIC Port" drop-down list and opens the first port at 115.2 kbps. If the "Update Ports" button is pressed, the list is updated, and when a port is selected in the list the connection is closed and

opened again with the new port. The program also scans the camera indexes 0 to 9 (DirectShow), shows the working ones in the "X Camera Port" list and opens the first one at 640 x 480 pixels and 30 frames per second. The camera can be changed at any time from the list.

Section 10.1.2: Position Comparison & Transmission Description

When a position (zone from 1 to 9) is selected for transmission, either by a manual button or by the prediction routine, the program compares the objective position with the previous transmitted position (the goalkeeper starts at position 5, the center). The handshake byte (0x24) used in V.2 was eliminated: only the positioning data is sent now, which solved the transmission issue found previously. Figure 10.1.1 shows the flowchart of the routine.

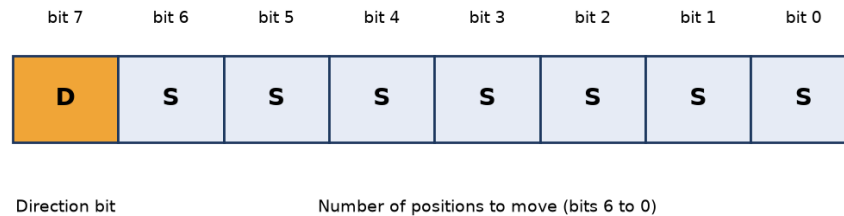
Figure 10.1.1 Automated Goalkeeper V.3 Project Python Position Comparison Flowchart



In case the objective position is greater than the previous position, the difference is calculated and the direction bit -the MSB of the data byte- is cleared, this being done by just transmitting the number of positions needed to reach the objective. On the other hand, if the objective position is smaller than the previous position, the difference is calculated and the direction bit is set by adding a decimal 128 to this number. The byte is transmitted through the serial port and the objective becomes the previous position. If both positions are equal, nothing is sent. The multiplication of the difference by the number of pulses per position is done by the

microcontroller (Section 10.2), so the byte only contains the number of positions (V.2 multiplied the position by a factor in Visual Basic). Figure 10.1.2 shows the format of the byte.

Figure 10.1.2 Automated Goalkeeper V.3 Project Python Positions and Direction Byte Bitmap



D: direction bit (0/1). S: number of positions to move, assigned in bits 6 to 0.

Table 10.1 Messages exchanged between the PC and the main board (V.3 protocol)

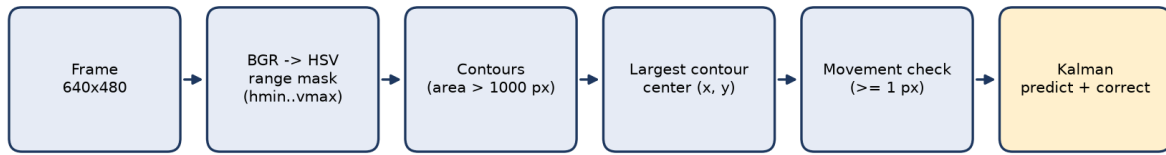
Byte	Direction	Meaning
PC -> PIC 0x01 - 0x08	MSB = 0	Move 1 to 8 positions with RA1 = 0
PC -> PIC 0x81 - 0x88	MSB = 1	Move 1 to 8 positions with RA1 = 1
PC -> PIC 0xBD	-	Start the homing sequence
PIC -> PC 0xBD	-	Homing finished (goalkeeper at home)
PIC -> PC 0x20	-	BLOCKED: flex sensor detected the ball
PIC -> PC 0x28	-	GOAL: motion sensor detected the ball
PIC -> PC 0x30	-	NO SENSOR: timeout without detection

Section 10.1.3: Camera Data Processing Routine Description

The routine is executed every 16 ms. It reads a frame from the camera and applies the color detection of cvzone (ColorFinder): the frame is converted to HSV and the six limits (hue, saturation and value, minimum and maximum) generate a mask. The contours of the mask with an area larger than 1000 pixels are found and the center (x, y) of the first one (the largest) is used as the position of the ball. If there is no contour, the ball is considered not detected in that frame.

Figure 10.1.3 shows the processing sequence.

Figure 10.1.3 Automated Goalkeeper V.3 Project Python Camera Data Processing Routine



Executed every 16 ms (app.after). The same loop updates the GUI and the simulation canvas.

The image shown on the screen includes the contours, the zone grid, the trails of the detected (green) and Kalman (blue) positions, the velocity arrow, the predicted impact point with its zone highlighted, and a text with the state of the state machine, the speed of the ball in pixels per frame and the target zone.

Section 10.1.4: Kalman Filter Description

A six-state Kalman filter (OpenCV) is used to estimate the position, velocity and acceleration of the ball. The state vector is $[x, y, vx, vy, ax, ay]$ and the model is a constant acceleration model where one frame is the unit of time: $x = x + vx + 0.5 ax$, $vx = vx + ax$ and $ax = ax$ (the same for the y axis). The camera only measures the position (x, y) , so the measurement matrix only takes the first two states. The prediction is executed in every frame, while the correction is executed only when the ball is detected and it moved at least 1 pixel since the last detection; this prevents that the noise of a still ball generates a fake velocity.

Table 10.2 Kalman filter and kick detection parameters

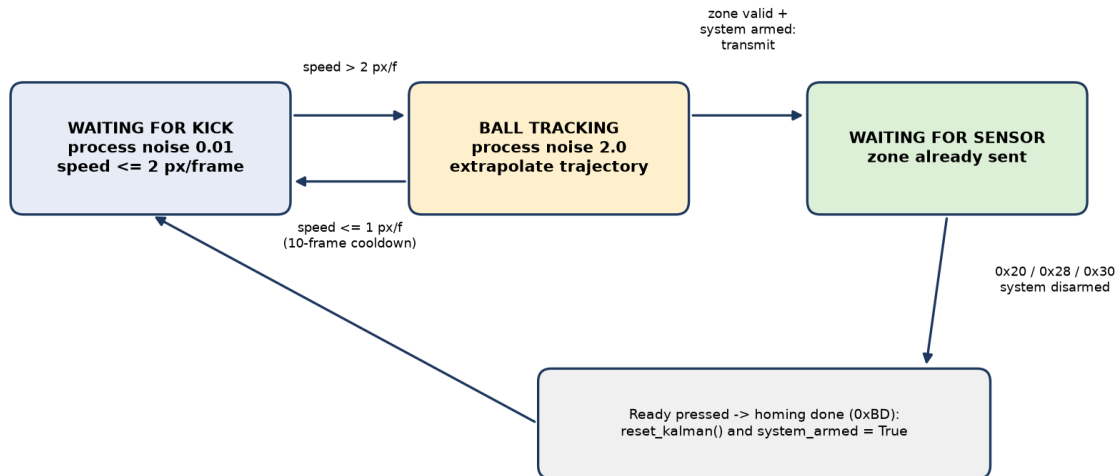
Parameter	Value	Purpose
Measurement noise	0.5	Reasonable trust in the camera detection
Process noise, waiting	0.01	Filter stays calm while the ball is still
Process noise, tracking	2.0	Filter reacts fast to the kick
Initial error covariance	100 (0.1 after a reset)	Fast convergence after the kick / calm start
MIN_MOVEMENT	1 pixel	Minimum movement to correct the filter
KICK_VELOCITY_THRESHOLD	2 px/frame	Speed that indicates a kick
KICK_COOLDOWN_FRAMES	10 frames	Time without kick detection after a reset

The function `reset_kalman()` (Ctrl+R, the "Reset Kalman" button, or automatically after homing) zeros the state and covariance matrices, restores the calm process noise, clears the trails and the simulation dots, and blocks the kick detection for 10 frames so residual velocity from the last shot does not generate a false kick.

Section 10.1.5: Kick Detection and Zone Prediction Description

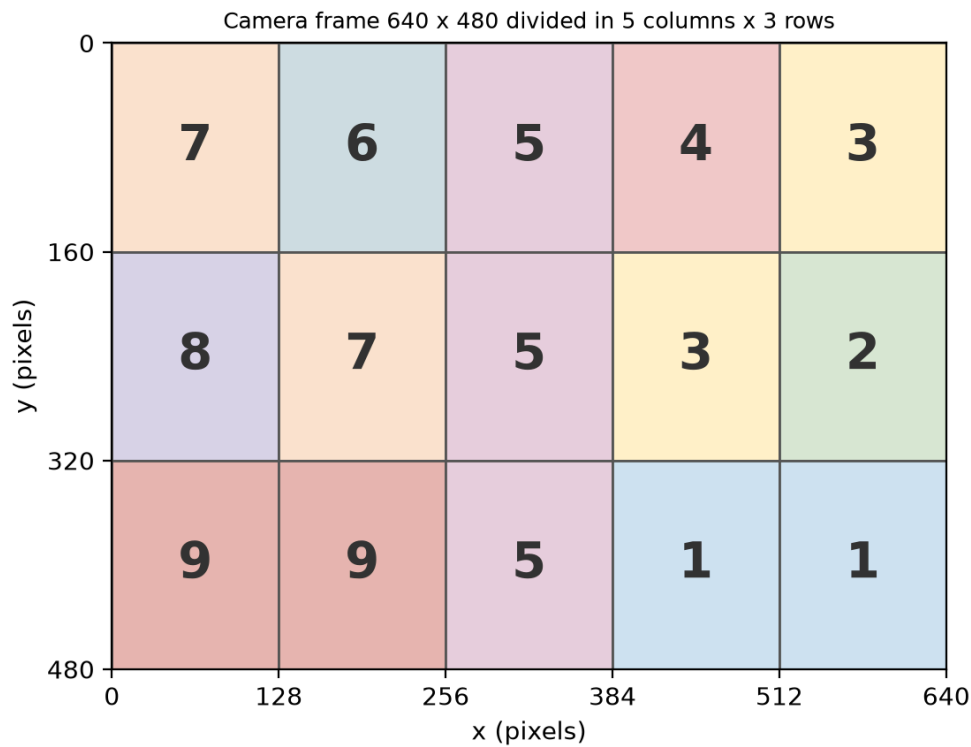
The prediction routine is the evolution of the semi-automatic mode of V.2, which was incomplete. It is a state machine (Figure 10.1.4) with three states. In **WAITING FOR KICK** the filter is kept calm. When the estimated speed goes above 2 pixels per frame and the ball is detected, the kick is confirmed and the state changes to **BALL TRACKING**. In this state the position, velocity and acceleration estimated by the filter are used to simulate the trajectory frame by frame (up to 200 frames) until the ball reaches the border of the image. The impact point (kept 1 pixel inside of the frame) is converted to a goal zone with the zone map of Figure 10.1.5. If the zone is valid, it was not transmitted before and the system is armed (Ready pressed and homing finished), the zone is transmitted once to the main board and the state changes to **WAITING FOR SENSOR**. If the speed falls under 1 pixel per frame before the transmission, the program returns to **WAITING FOR KICK** without transmitting.

Figure 10.1.4 Automated Goalkeeper V.3 Project Python Kick Detection State Machine



The 640 x 480 image is divided in 5 columns of 128 pixels and 3 rows of 160 pixels and every cell is assigned to one of the nine goal positions (Figure 10.1.5). The map is mirrored so the zone numbers match the positions of the goalkeeper; the center column always corresponds to zone 5. The function `get_zone()` returns the zone of a pixel and the function `extrapolate_to_goal()` calculates the impact point.

Figure 10.1.5 Automated Goalkeeper V.3 Project Python Goal Zone Map



The interface also has a simulation canvas that draws three points: the raw detection (red), the Kalman estimate (green) and the extrapolated impact point (blue), joined by a dashed line, which is useful to verify the prediction without a real kick.

Section 10.1.6: Manual Control Routine Description

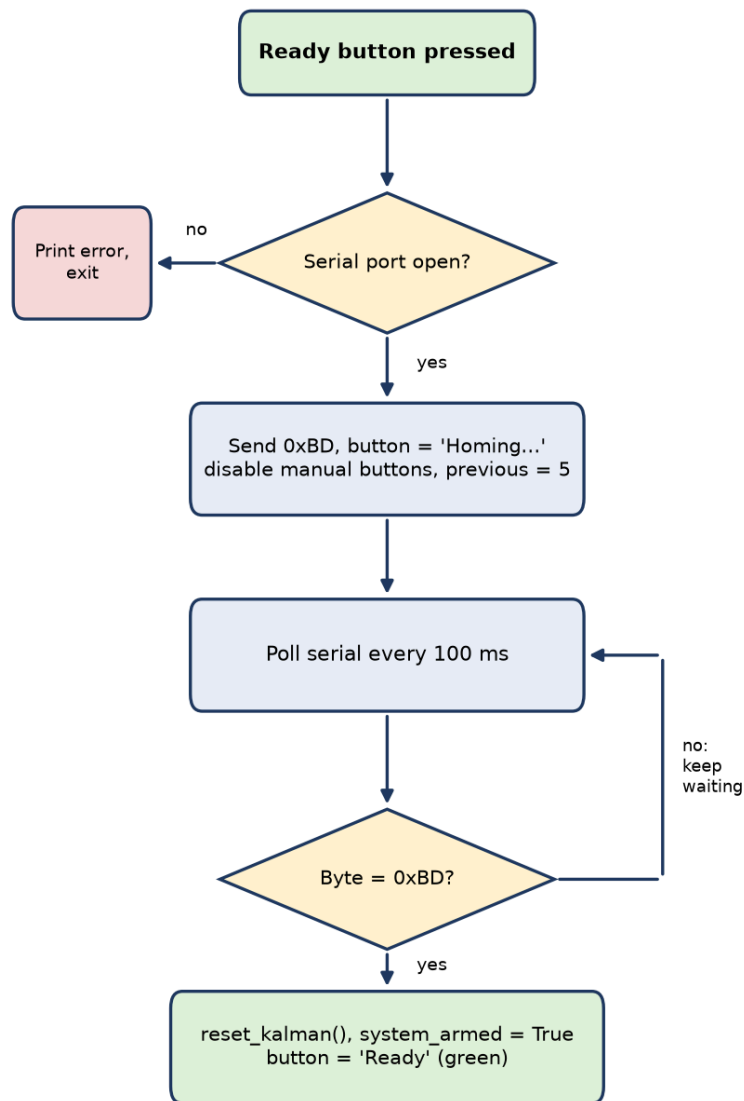
When one of the nine buttons of the interface is pressed, the manual control is used. It selects the position of the button and starts the Position Comparison and Transmission routine to move the goalkeeper to that position. This routine is used for calibration and troubleshooting. The buttons are disabled while the homing sequence is running and are enabled again when the main board reports the result of a shot (BLOCKED, GOAL or NO SENSOR).

Section 10.1.7: Ready and Homing Sequence Routine Description

The Ready button starts the homing sequence (Figure 10.1.6). If the serial port is open, the program sends the homing byte 0xBD (189), changes the button to "Homing..." (orange),

disables the manual buttons and sets the previous position to 5, the center of the goal. Then the program checks the serial port every 100 ms (with `app.after`, so the interface does not freeze) until it receives the byte `0xBD` from the main board. When this happens the Kalman filter is reset, the system is armed, and the button returns to green "Ready". The system does not transmit any predicted zone until it is armed, and it is disarmed again every time a shot result is received. This replaces the Hex 10 homing word of V.2, and the V.2 bug that required to press the button twice does not exist anymore because the third data byte was eliminated.

Figure 10.1.6 Automated Goalkeeper V.3 Project Python Homing Sequence Routine Flowchart

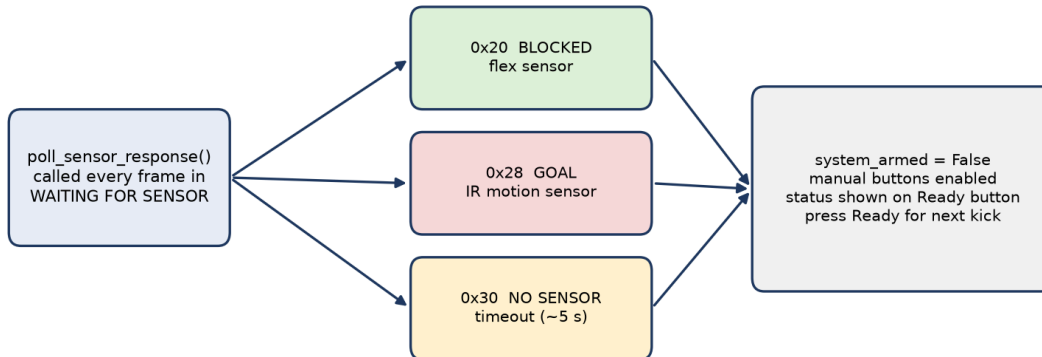


Section 10.1.8: Shot Result Routine Description

While the state machine is in WAITING FOR SENSOR, the function `poll_sensor_response()` is called in every frame and reads the serial port (Figure 10.1.7). Byte 0x20 means that the flex sensor of the goalkeeper was activated (BLOCKED, the button turns green), byte 0x28 that the motion sensor of the goal was activated (GOAL, the button turns red) and byte 0x30 that the main board waited about 5 seconds without any sensor (NO SENSOR, the

button turns yellow). In all cases the system is disarmed and the manual buttons are enabled. The Ready button must be pressed to home the goalkeeper and arm the system for the next kick.

Figure 10.1.7 Automated Goalkeeper V.3 Project Python Shot Result Routine



Section 10.1.9: Calibration and Debug Console Description

The color calibration (Ctrl+Y or the "Camera Calibration" button) opens an OpenCV window with the original image, the image with the mask applied, the mask and the contours, and six trackbars. When Q is pressed the values of the trackbars become the new HSV limits of the ball. The debug console (Ctrl+D) opens a window that mirrors everything printed by the program (transmitted bytes, states, received bytes), which is useful for troubleshooting. Table 10.3 lists the keyboard shortcuts.

Table 10.3 Keyboard shortcuts of the Python program

Shortcut	Function
Ctrl+Y	Start the camera color calibration (Q saves and exits)
Ctrl+R	Reset the Kalman filter and the tracking variables
Ctrl+D	Open / close the debug console
Esc	Close the program (releases the camera and the serial port)

Section 10.2: Assembly Code Description

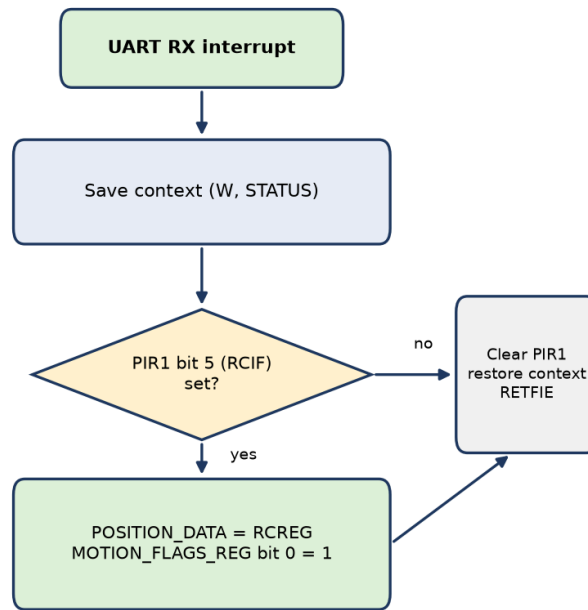
The firmware of the PIC16F1788 (Robokeeper_V.3_Main.asm, Robokeeper_V.3_Setup.inc and Timing_Cycles_1.inc, listed in Appendix D) was modified from

V.2 to receive the one-byte protocol, to generate a trapezoidal speed profile, and to read the sensors of the goal. The internal oscillator runs at 16 MHz with the PLL enabled (32 MHz, an instruction cycle of 125 ns) and the UART is configured for approximately 115.2 kbps (SPBRG = 67, BRG16 = 1, BRGH = 1). Only the UART receive interrupt is enabled. Figures 10.2.7 to 10.2.9 show the direction and function of the pins of the three ports.

Section 10.2.1: Data Received Routine Description

The Received Data Routine happens in the Interrupt Service Routine (ISR) when the RX interrupt flag (RCIF, bit 5 of PIR1) is set (Figure 10.2.1). The ISR saves W and STATUS, reads the received byte from RCREG, stores it in the register POSITION_DATA and sets the bit 0 of the register MOTION_FLAGS_REG to tell the main code that a byte was received. Then it clears the interrupt flags and returns. In V.2 this routine used the handshake byte 0x24, a position byte and a steps and direction byte, controlled by three flags (RC0, RA5 and RC1). In V.3 a single byte is received, so the handshake and the flags were eliminated.

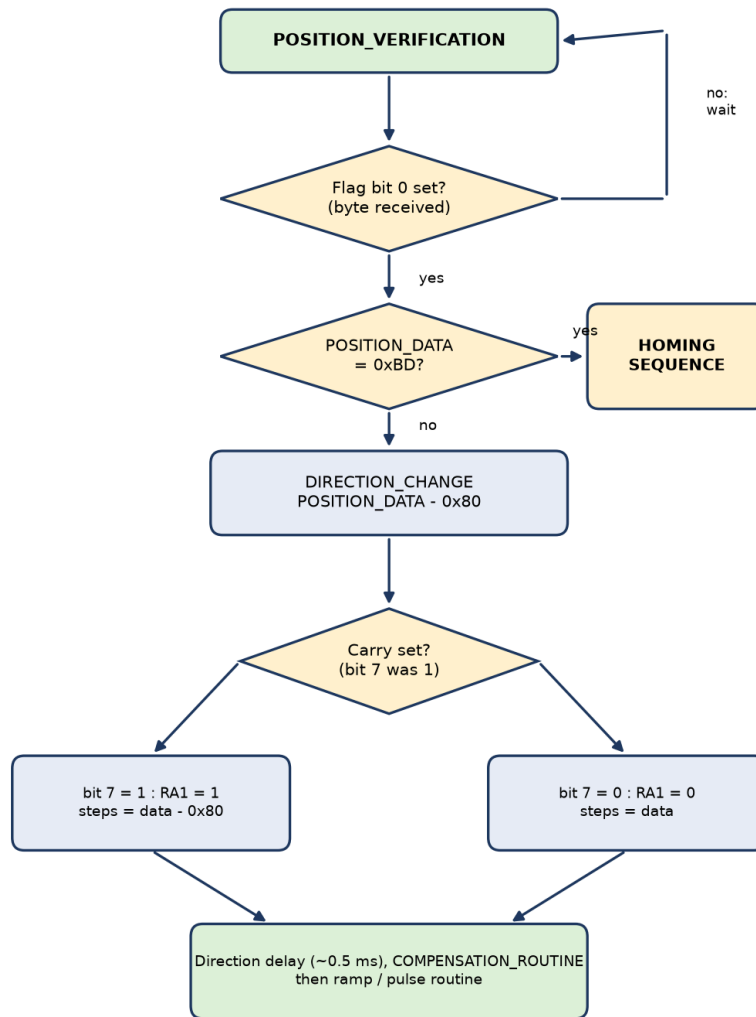
Figure 10.2.1 Automated Goalkeeper V.3 Project Assembly Received Data Routine Flowchart



Section 10.2.2: Main Routine Description

The main routine (POSITION_VERIFICATION) waits until the flag of the ISR is set (Figure 10.2.2). Then it compares the received byte with 0xBD: if they are equal, the homing sequence is started (Section 10.2.5); otherwise, the direction change routine is executed. This routine clears the motion flags and subtracts 0x80 (decimal 128) from the received byte to check if the direction bit is set. If the subtraction does not borrow (the carry bit of STATUS is set), the direction bit was set, the direction output RA1 is set and the number of positions is the result of the subtraction. If it borrows, the direction bit was cleared, RA1 is cleared and the number of positions is the received byte. Then a delay of about 0.5 ms (estimated from the instruction count) is added in order to allow the stepper driver to have enough time to switch the direction, the compensation multiplier is loaded (Table 10.5) and the pulse routine starts.

Figure 10.2.2 Automated Goalkeeper V.3 Project Assembly Main Routine and Direction Change Flowchart



Section 10.2.3: Pulse Routine and Speed Profile Description

The pulse routine (Figure 10.2.3) generates the PUL+ signal on RA0 with a trapezoidal speed profile (Figure 10.2.4) so the motor accelerates and decelerates smoothly. The speed is controlled by two registers (SPEED_REG1 and SPEED_REG2) where only one bit is set at a time; every bit corresponds to a step frequency loaded in TIMING_REG (Table 10.4), from about 2 kHz to 8.5 kHz in steps of 0.5 kHz. At every speed step 24 pulses (0x18) are generated with a 50 % duty cycle, and then the register is shifted to the left (LSLF) to accelerate. When the

maximum speed is reached, the cruise routine generates the pulses at about 9 kHz: the number of pulses is the number of positions multiplied by the steps multiplier ($0x78 = 120$) and by the compensation multiplier of Table 10.5. After the cruise, the registers are shifted to the right (LSRF) to decelerate through the same speed steps until the register is shifted out; this sets the carry bit and ends the movement. The multiplier is an easy way to obtain a large amount of steps without transmitting extra bytes, and the compensation values were determined so the goalkeeper reaches every position.

Figure 10.2.3 Automated Goalkeeper V.3 Project Assembly Pulse Routine Flowchart

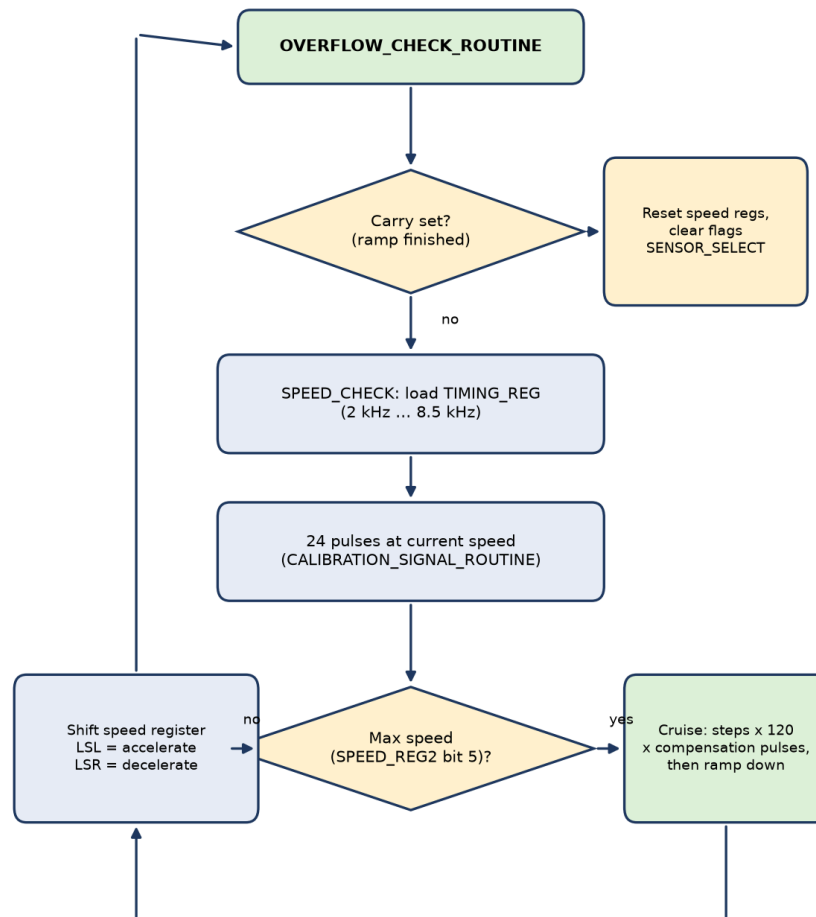


Figure 10.2.4 Automated Goalkeeper V.3 Project Assembly Speed Profile (from the firmware values)

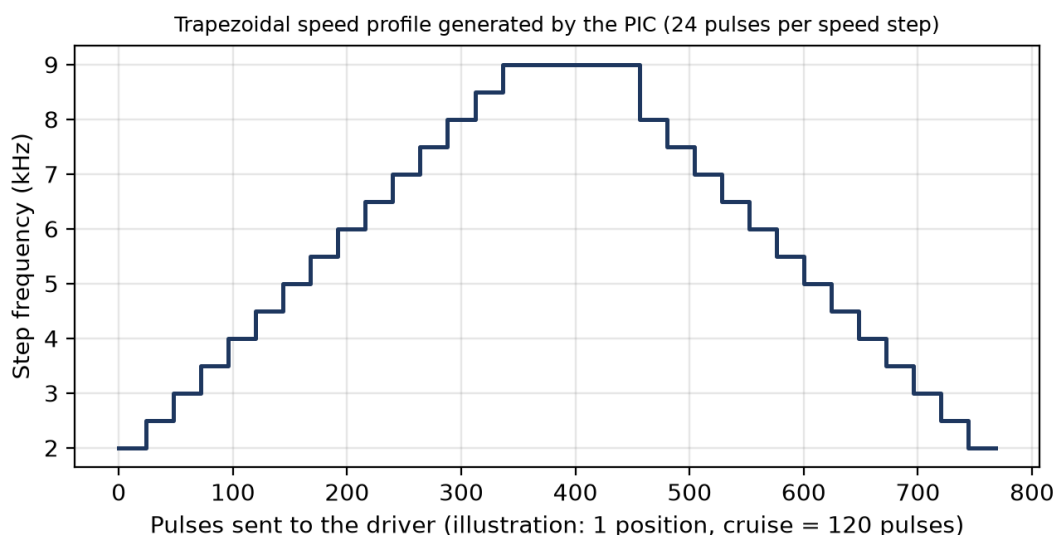


Table 10.4 Speed steps of the ramp (frequencies are nominal, from the value names in the firmware)

Speed step	Register bit	TIMING_REG value	Nominal frequency
1	SPEED_REG1 bit 0	0xA4	2.0 kHz
2	SPEED_REG1 bit 1	0x83	2.5 kHz
3	SPEED_REG1 bit 2	0x6C	3.0 kHz
4	SPEED_REG1 bit 3	0x5D	3.5 kHz
5	SPEED_REG1 bit 4	0x51	4.0 kHz
6	SPEED_REG1 bit 5	0x48	4.5 kHz
7	SPEED_REG1 bit 6	0x40	5.0 kHz
8	SPEED_REG1 bit 7	0x3A	5.5 kHz
9	SPEED_REG2 bit 0	0x35	6.0 kHz
10	SPEED_REG2 bit 1	0x31	6.5 kHz
11	SPEED_REG2 bit 2	0x2D	7.0 kHz
12	SPEED_REG2 bit 3	0x2B	7.5 kHz
13	SPEED_REG2 bit 4	0x27	8.0 kHz
14	SPEED_REG2 bit 5	0x25	8.5 kHz
Cruise	-	0x22 (SIGNAL_DELAY)	9.0 kHz

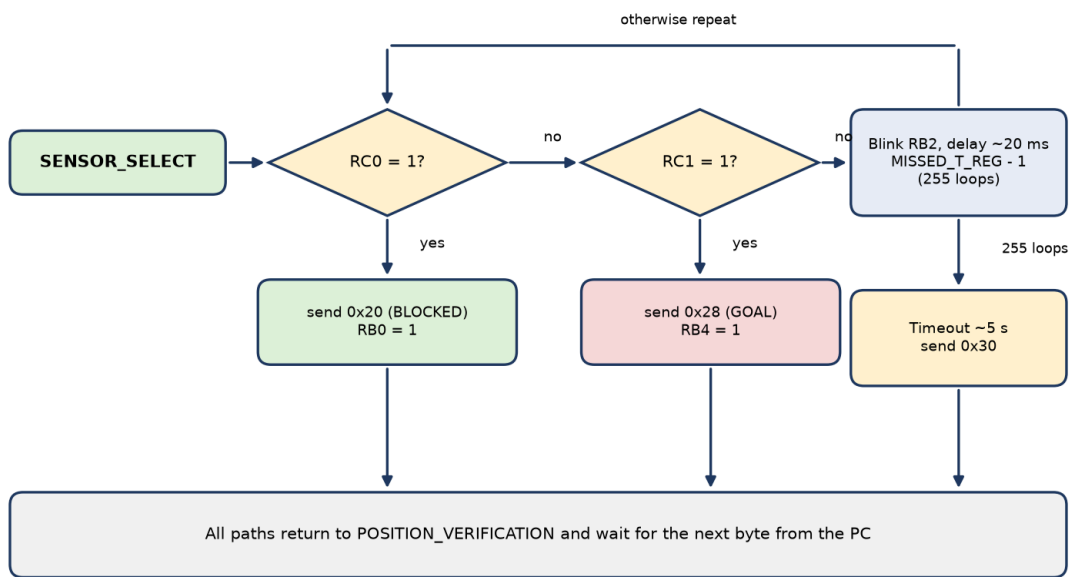
Table 10.5 Compensation multiplier and cruise pulses per number of positions

Positions to move	1	2	3	4	5	6	7	8
Compensation multiplier	1	6	8	9	10	11	11	11
Cruise pulses (x 120)	120	1440	2880	4320	6000	7920	9240	10560

Section 10.2.4: Sensor Result Routine Description

When the movement ends, the routine `SENSOR_SELECT` waits for the sensors of the goal (Figure 10.2.5). If `RC0` (flex sensor) is high, the routine turns on `RB0`, transmits `0x20` (BLOCKED) and returns to the main routine. If `RC1` (motion sensor) is high, it turns on `RB4` and transmits `0x28` (GOAL). While no sensor is active, `RB2` blinks and a delay of about 20 ms is executed in every loop; after 255 loops (about 5 seconds, estimated from the instruction count) the routine transmits `0x30` (NO SENSOR). In all cases, the flags are cleared and the microcontroller waits for the next byte of the PC.

Figure 10.2.5 Automated Goalkeeper V.3 Project Assembly Sensor Result Routine Flowchart



Section 10.2.5: PIC Homing Sequence Routine Description

If during the main routine the byte `0xBD` is received instead of a number of positions, the homing sequence starts (Figure 10.2.6). The sequence pulses `RA0` with a 50 % duty cycle (about 1 ms high and 1 ms low, estimated at roughly 400 Hz including the direction check) and keeps pulsing until the home sensor sends a 5 V signal that sets its input pin (`RA2`) high. During this routine `RA3` and `RA4` are constantly checked in order to detect if the goalkeeper is reaching an

end of the goal and has to change direction: if RA3 (left sensor) is set, the direction output RA1 is cleared, and if RA4 (right sensor) is set, RA1 is set. When the home sensor is detected, the flags are cleared and the byte 0xBD is transmitted to the PC as confirmation. In V.2 the homing word was Hex 10 and it required a third byte; in V.3 it is a single byte (0xBD) and works with the first press.

Figure 10.2.6 Automated Goalkeeper V.3 Project Assembly Homing Sequence Flowchart

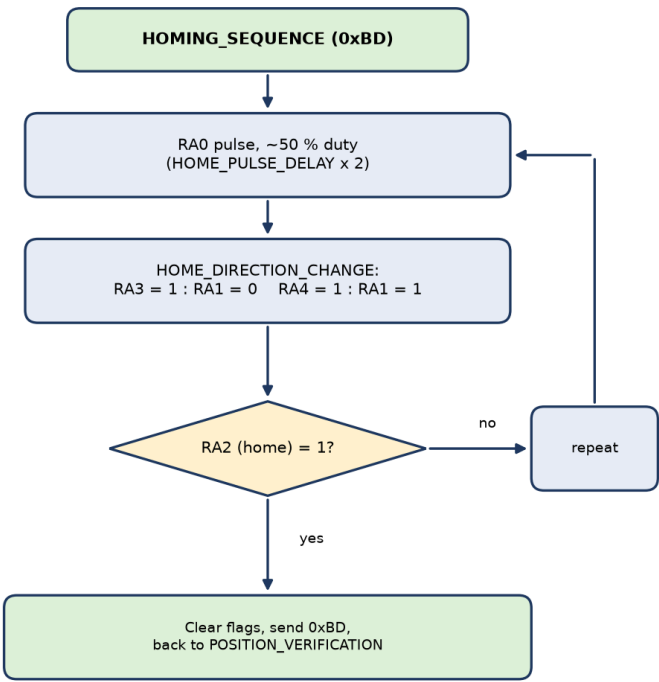


Figure 10.2.7 Automated Goalkeeper V.3 Project PORTA Bitmap

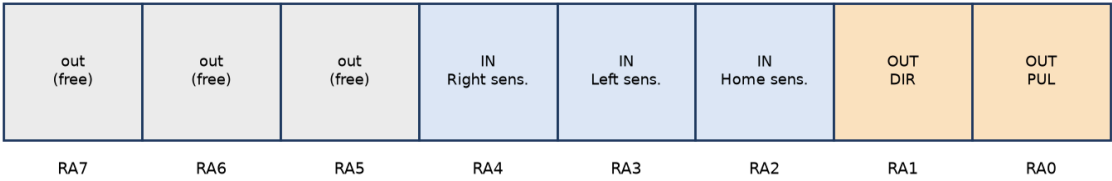


Figure 10.2.8 Automated Goalkeeper V.3 Project PORTB Bitmap

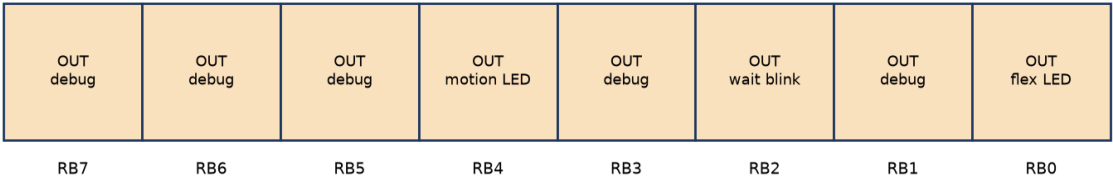
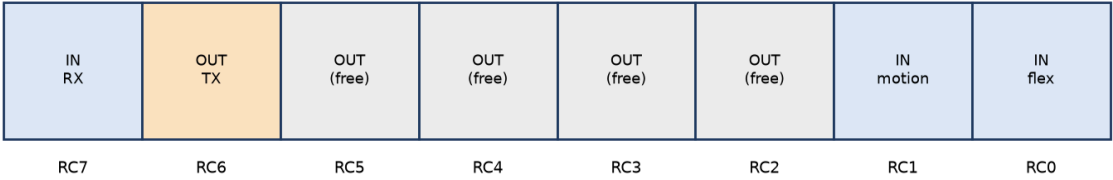


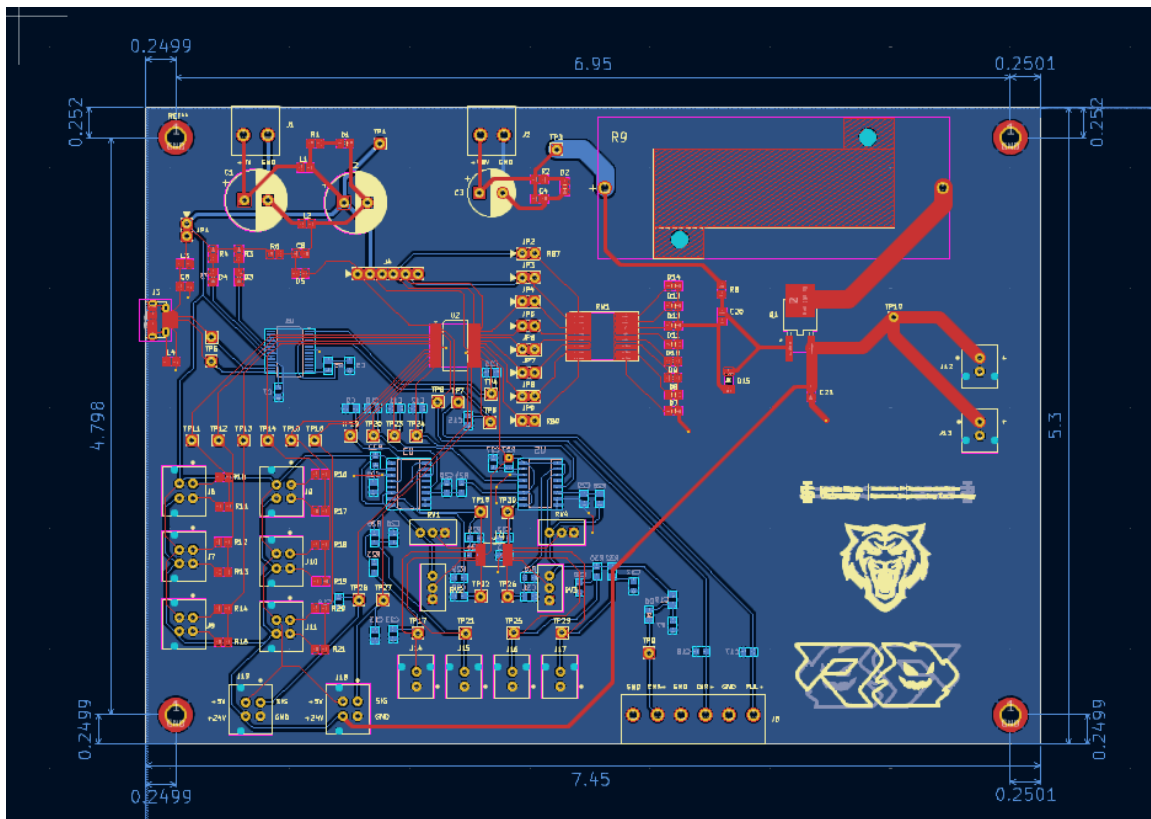
Figure 10.2.9 Automated Goalkeeper V.3 Project PORTC Bitmap



IN: input. OUT: output. Free: configured as output and not used by the firmware.

Mechanical Drawings

Figure 11.1 Automated Goalkeeper V.3 Project Main PCB Mechanical Drawings (KiCad layout with dimensions)



- Main Board Dimensions in mils: 7450 x 5300
- Main Board Dimensions in inches: 7.45 x 5.3
- Main Board Dimensions in millimeters: 189.2 x 134.6
- Mounting holes: 4, located 0.25 in from the edges (6.95 x 4.798 in between centers)

The mechanical parts of the goalkeeper were designed and 3D printed with carbon-fiber-filled filament. The model files (STEP and STL) are delivered with the project and are listed in Table 11.1. Figure 11.2 shows the complete project: the goal structure, the goalkeeper on its carriage, the lead screw with the stepper motor and the base.

Figure 11.2 Automated Goalkeeper V.3 Project Complete Structure



Table 11.1 3D printed parts of the Automated Goalkeeper V.3 (folder Robokeeper V.3 3D Models)

Part	Files
Computer Holder Piece	STEP / STL
Custom Pillow Block	STEP / STL
Pillow Block Cover	STEP / STL
Goalkeeper Holder Piece	STEP / STL
Keeper-Motor Connection with cable spacing	STEP / STL
Motor-Keeper Connectors Part A	STEP / STL
Motor-Keeper Connectors Extension	STEP / STL
Reflective Attachment Part	STEP / STL
Shaft Model	STEP / STL
Tapped Shaft Model	STEP
Spindle Holder 5.0	STEP / STL

Cost Analysis

The bill of materials of the main board (Rev E.2, exported from KiCad) is listed in Table 13.1. The prices of the components, and the cost of the mechanical parts, motor, driver, sensors, camera, power supplies and goal structure, have not been added yet and are marked as pending.

[Pending: BOM with suppliers and prices to be provided]

Table 13.1 Automated Goalkeeper V.3 Main Board Bill of Materials (quantities from KiCad, prices pending)

Item	Designators	Description	Qty	Unit (\$)	Total (\$)
1	J12, J13	Molex_1x2_Header	2	TBD	TBD
2	C10, C11, C9, C12, C18, C17, C16, C15, C14, C13	1 μ F	10	TBD	TBD
3	J17	Flex Sensor 4	1	TBD	TBD
4	J19, J18	IR Motion Sensor	2	TBD	TBD
5	J16	Flex Sensor 3	1	TBD	TBD
6	J15	Flex Sensor 2	1	TBD	TBD
7	J14	Flex Sensor 1	1	TBD	TBD
8	J11, J10, J9, J8, J7, J6	Molex_Header_2x2	6	TBD	TBD
9	R10, R14, R12, R18, R16, R20	20k Ω	6	TBD	TBD
10	D8	Debug 1	1	TBD	TBD
11	D14	Debug 7	1	TBD	TBD
13	R17, R19, R13, R15, R11, R21	5.1k Ω	6	TBD	TBD
14	D12	Debug 5	1	TBD	TBD
15	JP3	Debug 6	1	TBD	TBD
16	RV4, RV1, RV2, RV3	10 k Ω	4	TBD	TBD
17	J1	+5V Supply	1	TBD	TBD
18	JP5	Debug 4	1	TBD	TBD
19	C21, C20	10 μ F	2	TBD	TBD
20	J4	ICSP PORT	1	TBD	TBD
21	D10	Debug 3	1	TBD	TBD
22	C4	0.1 μ F	1	TBD	TBD
23	JP7	Debug 2	1	TBD	TBD
24	R1	3.3 K Ω	1	TBD	TBD
25	C1, C2	470 μ F	2	TBD	TBD
26	C8, C27, C25, C22, C19, C29	22 μ F	6	TBD	TBD
27	D2	+48V LED	1	TBD	TBD
28	U4	MCP6004	1	TBD	TBD
29	C6	0.01 μ F	1	TBD	TBD
30	REF**	RR_3mm_Mounting_Hole	1	TBD	TBD
31	JP6	Debug 3	1	TBD	TBD
32	D13	Debug 6	1	TBD	TBD
33	JP4	Debug 5	1	TBD	TBD
34	JP2	Debug 7	1	TBD	TBD
35	RN1	R_Pack_330	1	TBD	TBD
36	R6, R5	10 K Ω	2	TBD	TBD
37	D1	+5V LED	1	TBD	TBD
38	Q1	NJD35N04	1	TBD	TBD
39	J2	+48V Supply	1	TBD	TBD
40	JP1	Power Select	1	TBD	TBD

Item	Designators	Description	Qty	Unit (\$)	Total (\$)
41	U2	PIC16F1788	1	TBD	TBD
42	JP8	Debug 1	1	TBD	TBD
43	JP9	Debug 0	1	TBD	TBD
44	R8	2.2k Ω	1	TBD	TBD
45	R9	25 Ω , 50W	1	TBD	TBD
46	D11	Debug 4	1	TBD	TBD
47	L2, L1	10 μ H	2	TBD	TBD
48	D3	TXLED	1	TBD	TBD
49	J3	USB_B_Micro	1	TBD	TBD
50	D7	Debug 0	1	TBD	TBD
51	R2	33 K Ω	1	TBD	TBD
52	D4	RXLED	1	TBD	TBD
53	L4, L3	2.2 μ H	2	TBD	TBD
54	D15	23V	1	TBD	TBD
55	R3, R4	270 Ω	2	TBD	TBD
56	C3	100 μ F	1	TBD	TBD
57	D5, D6	1N4001	2	TBD	TBD
58	D9	Debug 2	1	TBD	TBD
59	J5	Stepper Motor Output	1	TBD	TBD
60	C24, C32, C7, C33, C31, C5, C26, C23, C28, C30	0.1 μ F	10	TBD	TBD
61	R28, R24, R22, R27	46 k Ω	4	TBD	TBD
62	U3, U5	74LV221	2	TBD	TBD
63	R7, R33, R31, R25, R29	10 k Ω	5	TBD	TBD
64	R32, R30, R26, R23	24 k Ω	4	TBD	TBD
65	U1	FT232RNL	1	TBD	TBD

Table 13.2 Other components (prices pending)

Other components	Qty	Unit (\$)	Total (\$)
Web camera 1080p	1	TBD	TBD
Stepper motor and driver	1	TBD	TBD
Lead screw and hardware	1	TBD	TBD
Optical fiber proximity sensors	3	TBD	TBD
Flex sensors	TBD	TBD	TBD
IR motion sensors	TBD	TBD	TBD
Cooling fans	2	TBD	TBD
Power supplies (+5 V and +48 V)	2	TBD	TBD
3D printing filament (carbon fiber)	TBD	TBD	TBD
Goal structure, base and goalkeeper panel	1	TBD	TBD

Conclusion/Recommendations

Section 14.1: Conclusions

As outlined in the Automated Goalkeeper Project V.1 and V.2 reports, the previous prototypes emulated the well-known RoboKeeper by integrating object tracking, motor control and real-time processing. During this semester, the pending work of V.2 was addressed: the Pixy V2.1 camera was replaced by a 1080p web camera whose resolution allows a correct tracking of the ball; the Visual Basic program was replaced by a Python program with OpenCV color detection, a Kalman filter that predicts the zone where the ball will arrive, and a homing and shot-result sequence; the main board was redesigned with a +24 V section, flex sensor conditioning, IR motion sensor headers and fan outputs; and the goalkeeper, the lead screw mechanism and the complete goal structure were built. The goalkeeper reaches all nine positions and the materials withstood the ball being kicked with full force, which proves that the materials are appropriate.

However, the project is not yet completely finished. The lower positions (1, 2, 8 and 9) still present problems holding the goalkeeper, the system has not been tested enough to give results of block rate, and the sensors of the new board are not completely integrated with the software and the hardware yet.

Section 14.2: Status of the Objectives

Table 14.1 Status of the objectives of V.3

Objective	Status
Replace the Pixy V2.1 with a 1080p web camera	Completed
Change from Visual Basic to Python with OpenCV	Completed
Trajectory prediction (Kalman filter)	Implemented; validation tests pending
New lead screw and goalkeeper motion mechanism	Completed; lower positions (1, 2, 8, 9) need improvement
Goalkeeper drawn and cut, complete goal structure	Completed; withstood full-force kicks
Improved custom PCB (Rev E.2) with 24 V section and fans	Completed
Fiber optic sensors for homing	Completed

Objective	Status
Flex sensor (goalkeeper) and motion sensor (goal)	Hardware on the board; not completely integrated with software and hardware yet (complete tests pending)
Box to protect the parts and make the system portable	TO CONFIRM
Test results (block rate, response time)	Pending

Section 14.3: Known Issues and Recommended Solutions

The following list is intended for the student that continues the project.

Table 14.2 Known issues and recommended solutions

Issue	Recommended solution
Lower positions (1, 2, 8, 9): the goalkeeper is not held correctly.	Reinforce the carriage and the goalkeeper holder, and support or guide the goalkeeper in those positions (for example counterweight, spring or a mechanical support). Verify the holding torque of the motor.
The Kalman state machine only runs when the ball is detected (in Main Code, the block that starts at "STATE MACHINE" is indented inside "if detected_center is not None"). If the ball is lost, the shot result may not be read.	Move that block one indentation level to the left so it runs in every frame, and use LOST_BALL_THRESHOLD to handle a lost ball.
The firmware only reads RC0 (flex) and RC1 (motion), while Rev E.2 routes flex 1 to 4 to RC0-RC3 and the IR sensors to RC4-RC5 (RC2-RC5 are set as outputs in TRISC).	Set RC0-RC5 as inputs in TRISC and extend SENSOR_SELECT: RC0-RC3 send 0x20 and RC4-RC5 send 0x28.
The comment "maximum of 2 steps" is not implemented, and the compensation table only covers moves of 1 to 8 positions.	Clamp the transmitted move to the range of the goalkeeper, or extend COMPENSATION_ROUTINE.
The PC assumes the goalkeeper is at the last transmitted position; a hit or a lost step changes the real position.	Press Ready to re-home. As improvement, add feedback of the position (sensor or encoder).
The calibration window blocks the interface until Q is pressed.	Run the calibration in a separate thread or window.
Some comments and docstrings do not match the code (for example 0x20 is described as the home confirmation, but the home byte is 0xBD).	Update the comments and remove the unused code (position_calculation, STATE_PREDICTED).

Section 14.4: Future Work

- Run test campaigns to measure the block rate, the time between the kick and the movement of the goalkeeper, and the maximum ball speed, and use them to tune the Kalman parameters.
- Update the firmware to use all the sensors of the board and add feedback of the position of the goalkeeper.
- Complete the box and the frame for portability, and add peripherals (sound effects, display, shot speed) to improve the interaction with the users.

Appendix A: Schematics

Figure 15.1 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 1/7: Top level (rotated 90 degrees)

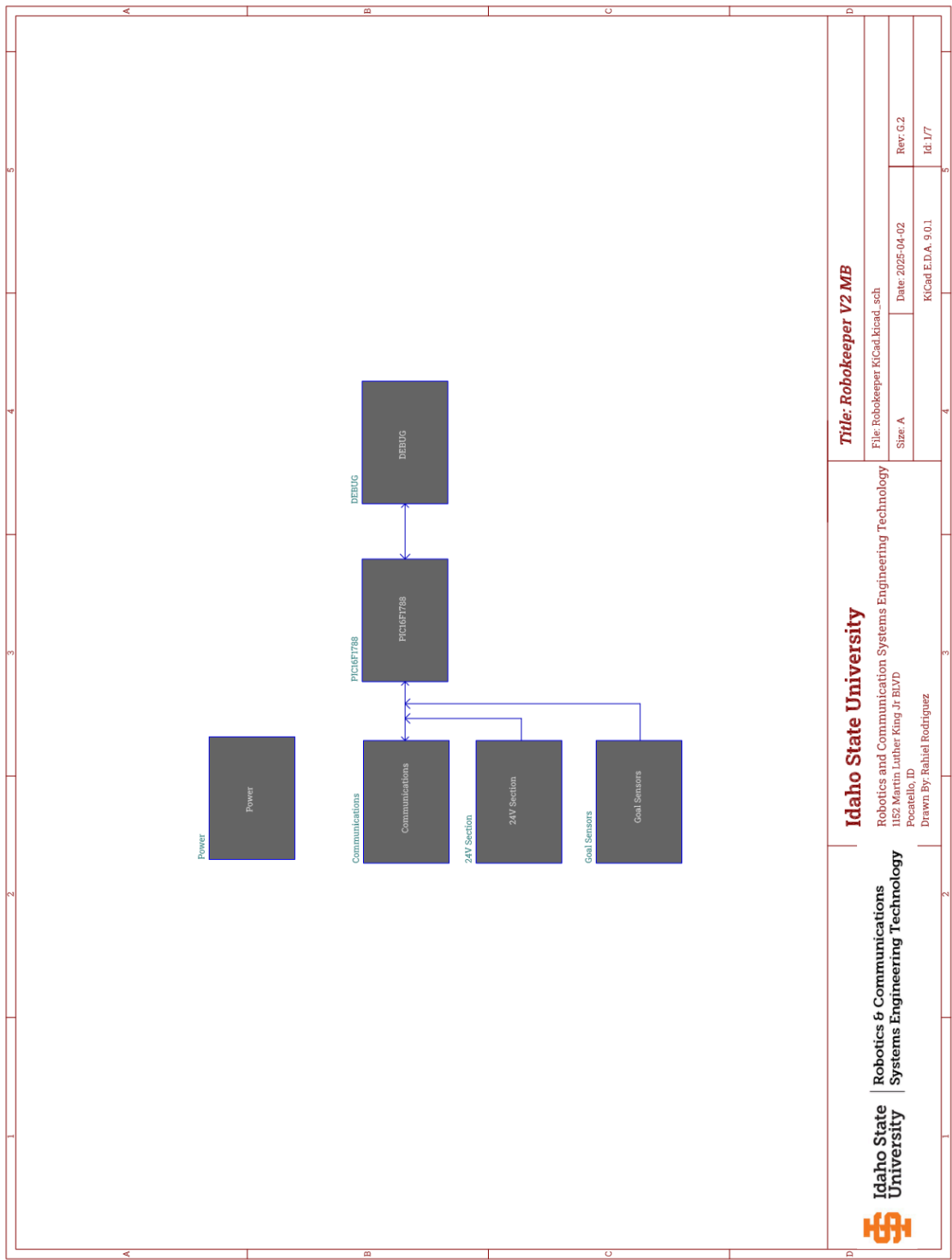


Figure 15.2 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 2/7: Power (rotated 90 degrees)

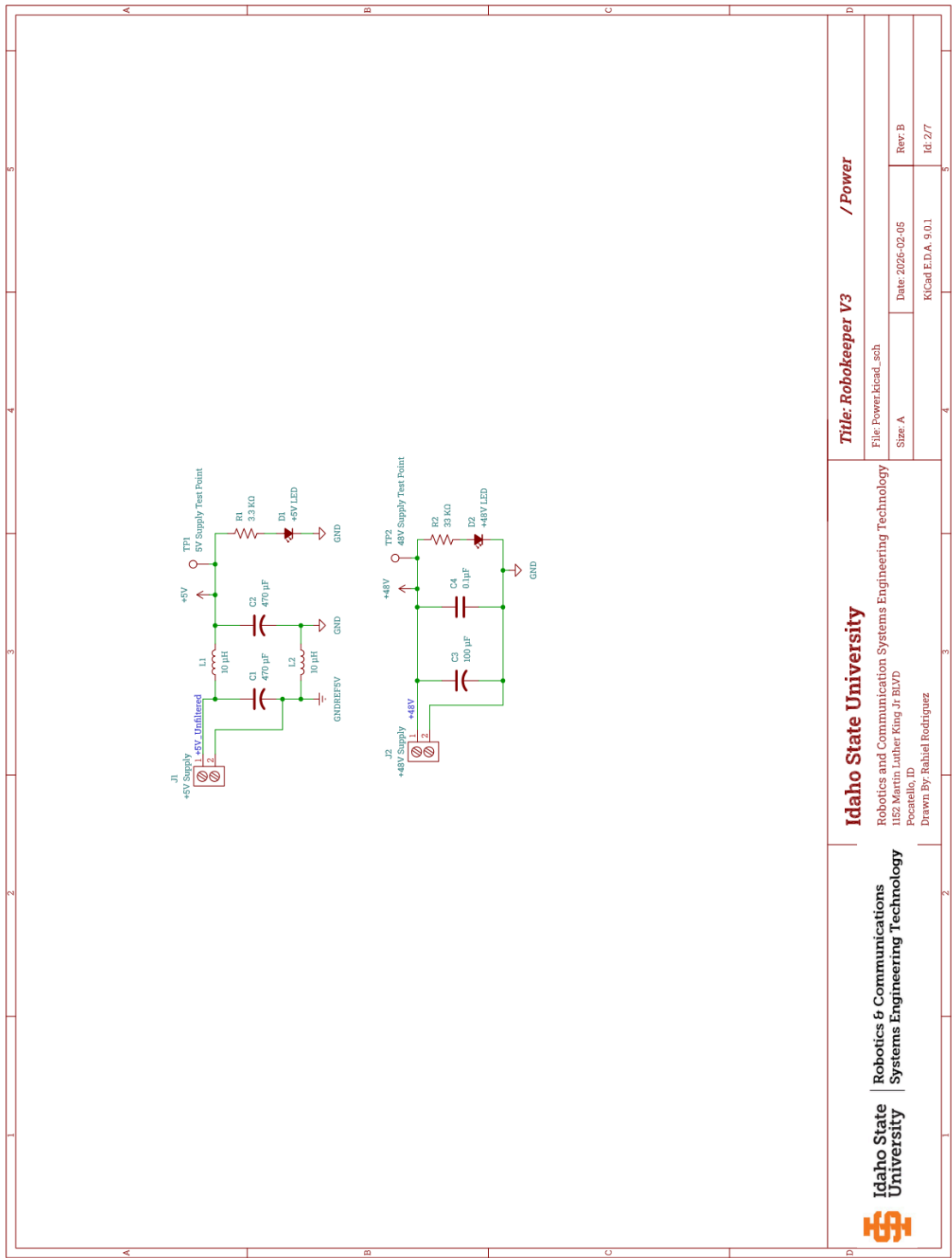


Figure 15.5 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 5/7: Debug (rotated 90 degrees)

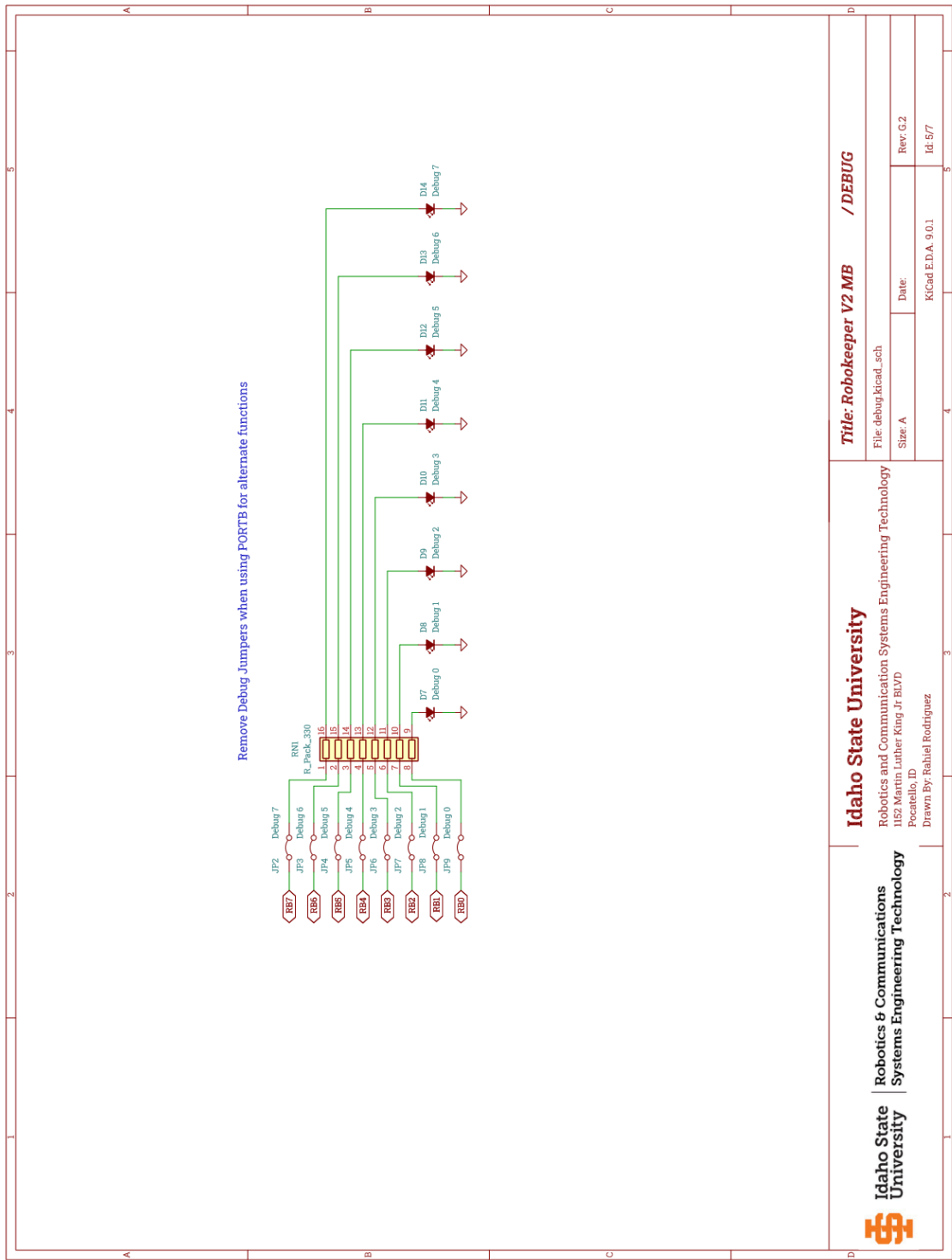


Figure 15.6 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 6/7: 24V Section (rotated 90 degrees)

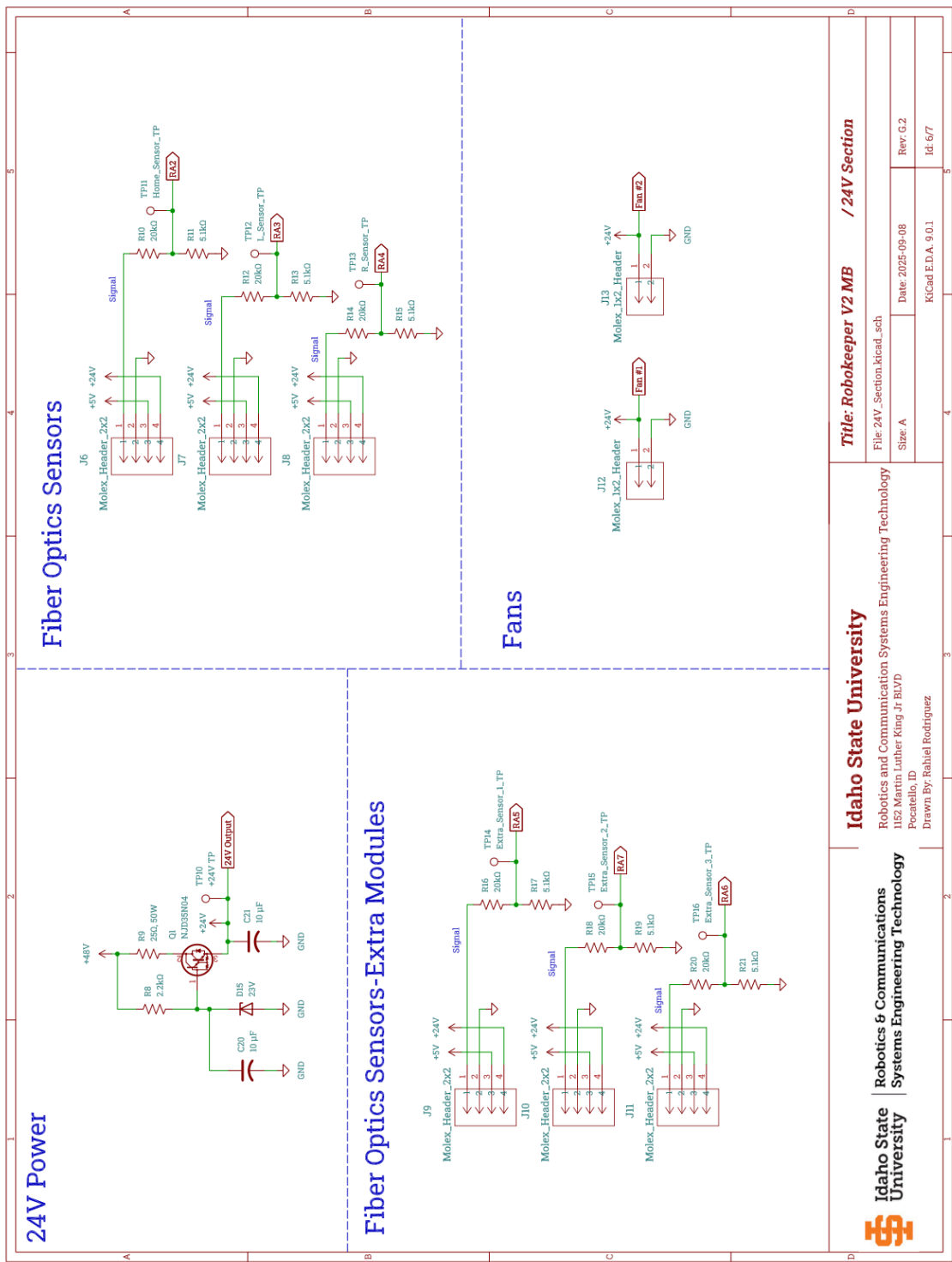
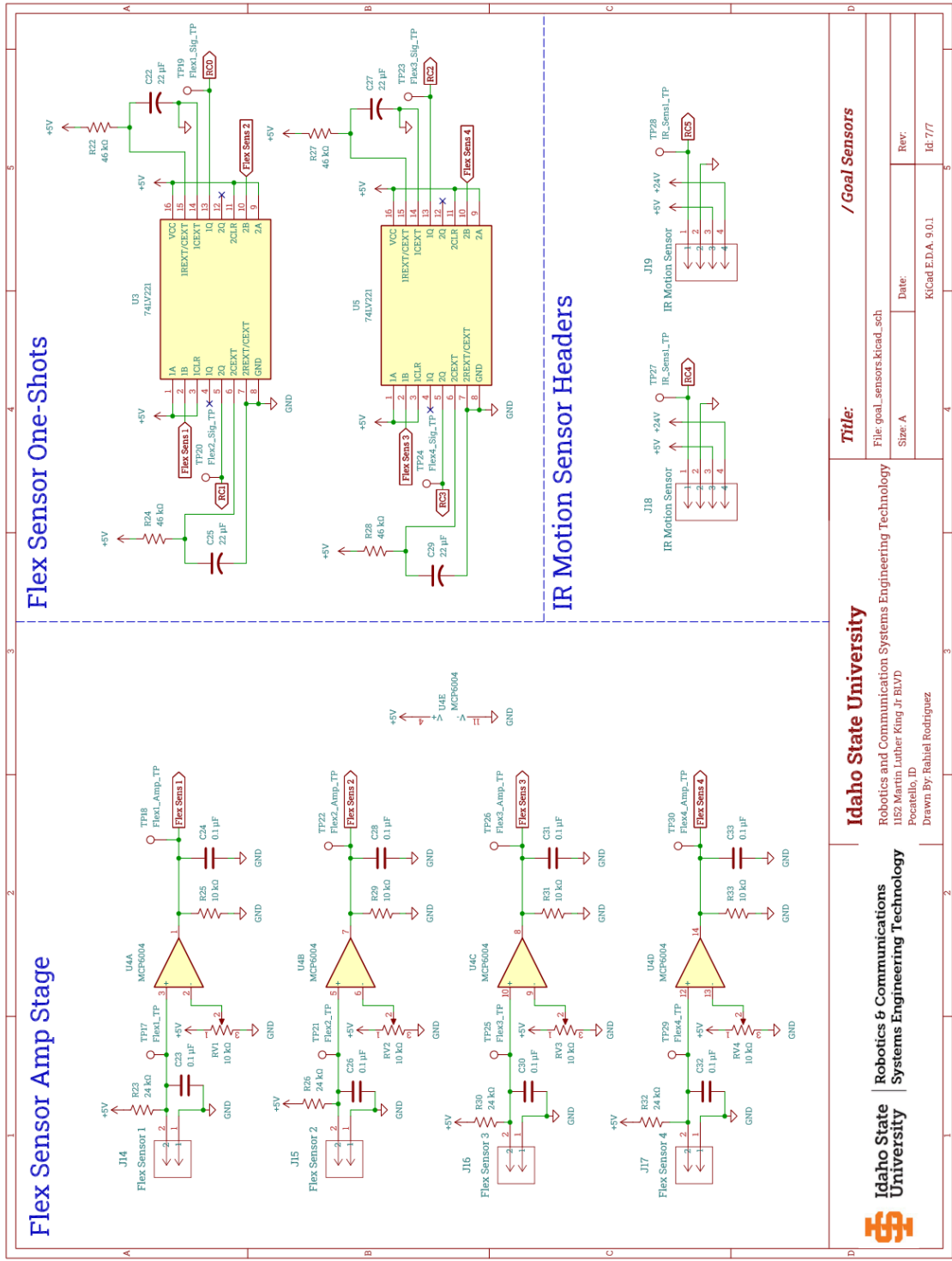
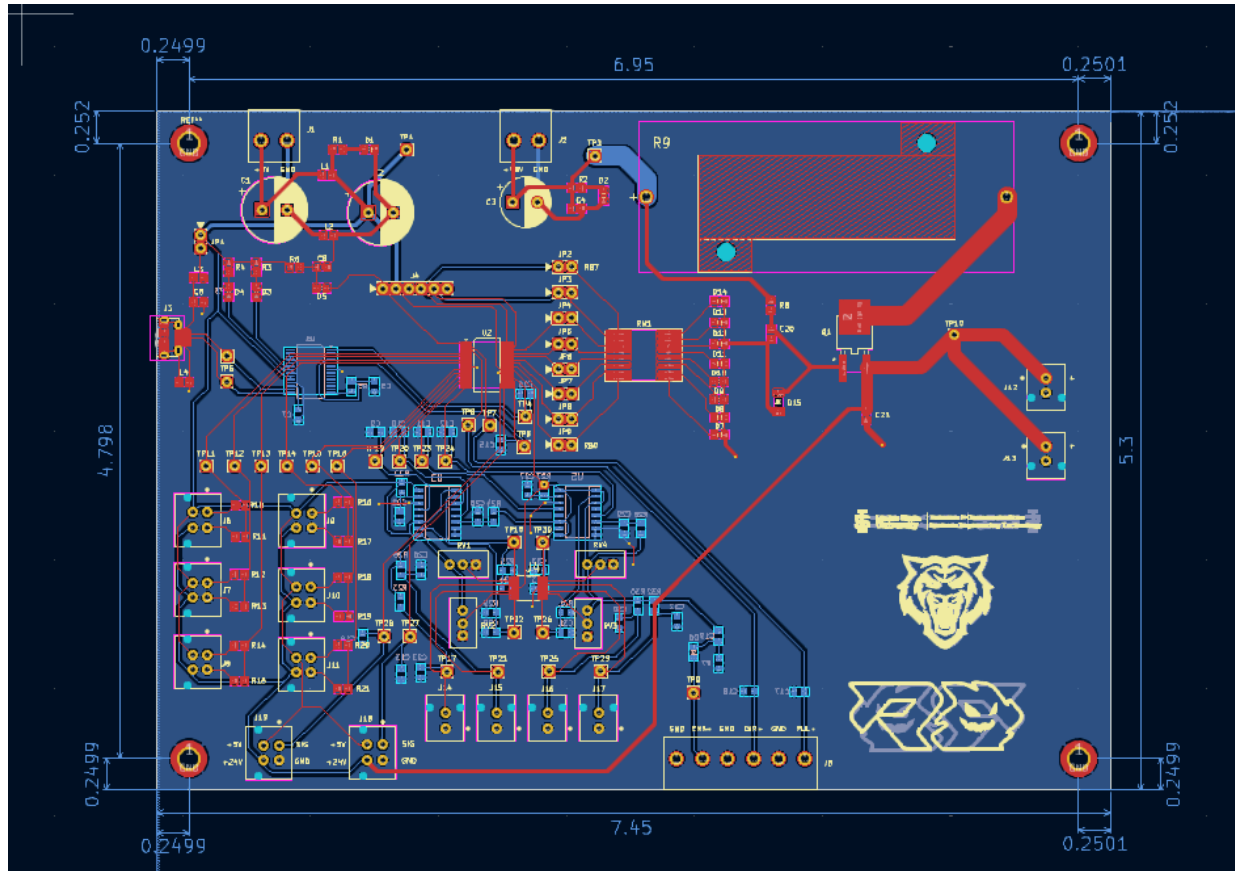


Figure 15.7 Robokeeper V.3 Main Board (Rev E.2) Schematic Sheet 7/7: Goal Sensors (rotated 90 degrees)



Appendix B: PCB Design

Figure 16.1 Main Board PCB Layout (KiCad)



Appendix C: Python Code

Main Code (Robokeeper V.3 Python\Main Code.txt)

```

import tkinter
from tkinter import Canvas
from time import sleep
import customtkinter
import serial
import serial.tools.list_ports
import cv2
import cvzone
import numpy as np
from cvzone.ColorModule import ColorFinder
from PIL import Image
import io
import sys

# --- APP SETUP ---
customtkinter.set_appearance_mode("System")
customtkinter.set_default_color_theme("blue")

app = customtkinter.CTk()
app.geometry("1280x720+0+0")
app.title("Ball Detection & Motor Control - cvzone")

# --- SERIAL PORT ---
pic_Serial_Port = serial.Serial()
ports_List = []

# --- CAMERA ---
camera_x = None

# --- COLOR DETECTION ---
myColorFinder_x = ColorFinder(False)
myColorFinder_calibration = None
hsvVals_x = {'hmin': 0, 'smin': 110, 'vmin': 58, 'hmax': 35, 'smax': 255, 'vmax': 255}

# --- MOTOR POSITION TRACKING ---
position_data = bytearray(1)
objective_position_global = 0
previous_position_global = 0
steps_multiplying_factor = 4

# --- APP STATE FLAGS ---
settings_done = False
manual_port_change = False
camera_running = True
calibration_mode = False
calibration_window_open = False
calibrating_camera = None
x_value = 0
y_value = 0

# --- CONSOLE STATE ---
console_window = None
console_visible = False
original_stdout = sys.stdout

# =====
# KALMAN FILTER SETUP - 6-state constant acceleration model
# State vector: [x, y, vx, vy, ax, ay]
# =====

kalman = cv2.KalmanFilter(6, 2)

# Kinematic transition: x=x+vx+0.5ax | vx=vx+ax | ax=ax (same for y axis)
kalman.transitionMatrix = np.array([
    [1, 0, 1, 0, 0.5, 0 ],
    [0, 1, 0, 1, 0, 0.5],
    [0, 0, 1, 0, 1, 0 ],
    [0, 0, 0, 1, 0, 1 ],
    [0, 0, 0, 0, 1, 0 ],
    [0, 0, 0, 0, 0, 1 ]
], dtype=np.float32)

```

```

# Camera only measures position, not velocity or acceleration
kalman.measurementMatrix = np.array([
    [1, 0, 0, 0, 0, 0],
    [0, 1, 0, 0, 0, 0]
], dtype=np.float32)

# High process noise = filter reacts fast to sudden velocity changes (kick)
kalman.processNoiseCov = np.eye(6, dtype=np.float32) * 2.0

# Moderate measurement noise = reasonable trust in camera detection
kalman.measurementNoiseCov = np.eye(2, dtype=np.float32) * 0.5

# High initial uncertainty = filter converges quickly on first frames after kick
kalman.errorCovPost = np.eye(6, dtype=np.float32) * 100

# --- KALMAN STATE MACHINE ---
STATE_WAITING = "WAITING FOR KICK"
STATE_TRACKING = "BALL TRACKING"
STATE_PREDICTED = "PREDICTED ZONE"

STATE_WAITING_RESULT = "WAITING FOR SENSOR" # zone sent, waiting for PIC response

kalman_state = STATE_WAITING
predicted_zone = None
frames_without_ball = 0
LOST_BALL_THRESHOLD = 8 # consecutive missed frames before declaring ball gone

# --- ZONE TRANSMISSION FLAG ---
# Transmits on the first valid zone prediction after kick detection.
# Reset to False on every reset_kalman() call.
zone_transmitted = False

# --- SYSTEM ARMED FLAG ---
# False at startup and after any sensor result.
# Only set to True after PIC confirms homing (0xBD received).
# Prevents any zone transmission until the full ready sequence is completed.
system_armed = False

# --- KICK DETECTION ---
KICK_VELOCITY_THRESHOLD = 2
KICK_CONFIRM_FRAMES = 1 # consecutive frames above threshold to confirm kick
kick_confirm_count = 0 # counts up when speed exceeds threshold
# --- STATIONARY BALL NOISE SUPPRESSION ---
# If the ball moves less than MIN_MOVEMENT pixels between frames, the detection
# is treated as noise and kalman.correct() is skipped to prevent fake velocity
MIN_MOVEMENT = 1 # pixels - tune up if detection is still jittery
last_detected = None # stores previous frame's detection for movement check

# --- KICK DETECTION COOLDOWN ---
# After a reset, block kick detection for KICK_COOLDOWN_FRAMES frames so the
# filter has time to settle before it can trigger STATE_TRACKING again.
# Without this, residual filter state from the previous kick causes an instant
# false kick detection on the very first frame after reset.
KICK_COOLDOWN_FRAMES = 10 # frames to wait after reset before allowing kick detection
kick_cooldown = 0 # counts down to 0 after each reset

# Process noise levels - swapped dynamically based on state:
# WAITING → low (0.01): filter stays calm while ball is stationary
# TRACKING → high (2.0): filter reacts fast to kick velocity changes
PROCESS_NOISE_CALM = 0.01
PROCESS_NOISE_REACTIVE = 2.0

# --- TRAIL HISTORY ---
detected_trail = [] # raw camera detections (green dot)
predicted_trail = [] # kalman estimates (blue dot)
MAX_TRAIL = 50

# --- GOAL ZONE BOUNDARIES - 3 rows x 5 columns ---
# Column dividers: frame width (640) divided into 5 equal columns = 128px each
# Row dividers: frame height (480) divided into 3 equal rows = 160px each
FRAME_W = 640; FRAME_H = 480
COL = FRAME_W // 5 # 128 px per column
ROW = FRAME_H // 3 # 160 px per row

# Column pixel boundaries (4 vertical dividers for 5 columns)

```

```

X1 = COL          # 128
X2 = COL * 2      # 256
X3 = COL * 3      # 384
X4 = COL * 4      # 512

# Row pixel boundaries (2 horizontal dividers for 3 rows)
# Y increases downward in OpenCV, so row 1 (bottom) has the highest Y values
Y1 = ROW          # 160 - bottom of top row / top of middle row
Y2 = ROW * 2      # 320 - bottom of middle row / top of bottom row

# =====
# SIMULATION CANVAS SETUP
# A tkinter Canvas placed below the camera feed showing three labelled dots:
# RED = raw detected position from color detection
# GREEN = Kalman smoothed/predicted position
# BLUE = extrapolated impact point on the goal boundary
# Canvas coordinates are scaled from camera frame (640x480) → canvas (425x160)
# =====

SIM_W = 530        # canvas width - matches camera feed display width
SIM_H = 425        # canvas height - matches camera feed display height
DOT_R = 7          # dot radius in pixels

# Scale factors from camera frame to simulation canvas
SCALE_X = SIM_W / FRAME_W
SCALE_Y = SIM_H / FRAME_H

# Stores the last known positions for each dot so canvas only redraws when needed
sim_detected_pos = None # RED dot - latest raw detection
sim_kalman_pos   = None # GREEN dot - latest Kalman estimate
sim_impact_pos   = None # BLUE dot - latest extrapolated impact point

# =====
# KALMAN HELPER FUNCTIONS
# =====

def get_zone(x, y):
    """
    Map pixel (x,y) to goal zone 1-9 using a 3-row x 5-column grid.
    Returns None if outside the frame.

    Grid layout (matches the diagram):
    Row top (0 → Y1): cols → zone 3, 4, -, 6, 7
    Row middle (Y1 → Y2): cols → zone 2, 3, 5, 7, 8
    Row bottom (Y2 → H): cols → zone 1, 1, -, 9, 9
    """
    if x <= 0 or x >= FRAME_W or y <= 0 or y >= FRAME_H:
        return None

    # Determine column (0-indexed, left to right)
    col = min(int(x // COL), 4) # clamp to 0-4

    # Determine row (0=top, 1=middle, 2=bottom)
    if y < Y1:
        row = 0 #top
    elif y < Y2:
        row = 1 #middle
    else:
        row = 2 #bottom

    # Zone mapping from diagram (mirrored):
    # col0 col1 col2 col3 col4
    # row 0: 7 6 5 4 3
    # row 1: 8 7 5 3 2
    # row 2: 9 9 5 1 1
    zone_map = {
        (0, 0): 7, (1, 0): 6, (2, 0): 5, (3, 0): 4, (4, 0): 3,
        (0, 1): 8, (1, 1): 7, (2, 1): 5, (3, 1): 3, (4, 1): 2,
        (0, 2): 9, (1, 2): 9, (2, 2): 5, (3, 2): 1, (4, 2): 1,
    }
    return zone_map.get((col, row), None)

def extrapolate_to_goal(px, py, vx, vy, ax, ay):
    """
    Step the ball forward frame-by-frame using kinematics until it hits
    a frame boundary. Returns (impact_point, zone) or (None, None).
    """

```

```

"""
sim_x, sim_y = float(px), float(py)
sim_vx, sim_vy = float(vx), float(vy)
sim_ax, sim_ay = float(ax), float(ay)

for _ in range(200):
    sim_x += sim_vx + 0.5 * sim_ax
    sim_y += sim_vy + 0.5 * sim_ay
    sim_vx += sim_ax
    sim_vy += sim_ay
    if sim_x <= 0 or sim_x >= FRAME_W or sim_y <= 0 or sim_y >= FRAME_H:
        ix = max(1, min(FRAME_W - 1, int(sim_x))) # ← 1-pixel inset
        iy = max(1, min(FRAME_H - 1, int(sim_y))) # ← 1-pixel inset
        return (ix, iy), get_zone(ix, iy)
return None, None

def draw_zone_grid(frame):
    """Draw the 5-col x 3-row grid lines and zone labels onto the OpenCV frame."""
    h, w = frame.shape[:2]
    gray = (100, 100, 100)

    # 4 vertical dividers for 5 columns
    for xv in [X1, X2, X3, X4]:
        cv2.line(frame, (xv, 0), (xv, h), gray, 1)

    # 2 horizontal dividers for 3 rows
    cv2.line(frame, (0, Y1), (w, Y1), gray, 1)
    cv2.line(frame, (0, Y2), (w, Y2), gray, 1)

    # Zone labels – placed at centre of each cell that has a zone assigned
    # Centre x of each column, centre y of each row
    cx = [COL // 2, COL + COL // 2, COL * 2 + COL // 2, COL * 3 + COL // 2, COL * 4 + COL // 2]
    cy = [ROW // 2, ROW + ROW // 2, ROW * 2 + ROW // 2]

    #
    col0  col1  col2  col3  col4
    # row 0:  7    6    5    4    3
    # row 1:  8    7    5    3    2
    # row 2:  9    9    5    1    1
    labels = {
        (0, 0): 7, (1, 0): 6, (2, 0): 5, (3, 0): 4, (4, 0): 3,
        (0, 1): 8, (1, 1): 7, (2, 1): 5, (3, 1): 3, (4, 1): 2,
        (0, 2): 9, (1, 2): 9, (2, 2): 5, (3, 2): 1, (4, 2): 1,
    }
    for (col, row), zone in labels.items():
        cv2.putText(frame, str(zone), (cx[col] - 10, cy[row] + 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 1.0, (60, 60, 60), 2)

def reset_kalman():
    """
    Reset filter state and all tracking variables.
    Called after motor homes or manually via Ctrl+R / Reset button.
    """
    global kalman_state, predicted_zone, frames_without_ball
    global detected_trail, predicted_trail
    global sim_detected_pos, sim_kalman_pos, sim_impact_pos
    global last_detected, zone_transmitted, kick_cooldown
    global kick_confirm_count
    # Note: system_armed is NOT reset here – only ready_sequence controls it
    # Note: system_armed is NOT reset here – only ready_sequence controls it

    # Zero ALL internal filter matrices so the first predict() after reset
    # returns zero velocity – prevents false kick detection on resume
    kalman.statePost = np.zeros((6, 1), dtype=np.float32)
    kalman.statePre = np.zeros((6, 1), dtype=np.float32)
    kalman.errorCovPost = np.eye(6, dtype=np.float32) * 0.1 # low uncertainty = calm start
    kalman.errorCovPre = np.eye(6, dtype=np.float32) * 0.1

    kalman_state = STATE_WAITING
    predicted_zone = None
    frames_without_ball = 0
    detected_trail = []
    predicted_trail = []
    last_detected = None # clear movement history on reset
    zone_transmitted = False # allow transmission on next kick
    kick_cooldown = KICK_COOLDOWN_FRAMES # block kick detection briefly after reset
    kick_confirm_count = 0

```

```

# Restore calm process noise – ball is stationary again after reset
kalman.processNoiseCov = np.eye(6, dtype=np.float32) * PROCESS_NOISE_CALM

# Clear simulation dots
sim_detected_pos = None
sim_kalman_pos = None
sim_impact_pos = None
update_simulation_canvas()
print("Kalman reset – waiting for next kick.")

# =====
# SIMULATION CANVAS FUNCTIONS
# =====

def cam_to_sim(x, y):
    """
    Convert camera frame pixel coordinates (0-640, 0-480)
    to simulation canvas coordinates (0-425, 0-160).
    """
    return int(x * SCALE_X), int(y * SCALE_Y)

def update_simulation_canvas():
    """
    Redraw the simulation canvas with current dot positions.
    Called every frame from update_camera_feeds().

    Dot legend:
    RED = raw camera detection (where color finder says the ball is)
    GREEN = Kalman estimated position (smoothed, continues when ball lost)
    BLUE = extrapolated impact point (where ball will hit the goal)
    """
    sim_canvas.delete("dots") # clear only dots, keep background and grid

    # Draw zone dividers on canvas (scaled from camera frame)
    # (already drawn as permanent lines in build_simulation_canvas)

    # RED dot – raw detected position
    if sim_detected_pos is not None:
        cx, cy = sim_detected_pos
        sim_canvas.create_oval(
            cx - DOT_R, cy - DOT_R, cx + DOT_R, cy + DOT_R,
            fill="#ff3333", outline="#ff8888", width=2, tags="dots"
        )
        sim_canvas.create_text(
            cx, cy - DOT_R - 6,
            text="DET", fill="#ff5555", font=("Consolas", 7, "bold"), tags="dots"
        )

    # GREEN dot – Kalman predicted/smoothed position
    if sim_kalman_pos is not None:
        cx, cy = sim_kalman_pos
        sim_canvas.create_oval(
            cx - DOT_R, cy - DOT_R, cx + DOT_R, cy + DOT_R,
            fill="#33ff66", outline="#88ffaa", width=2, tags="dots"
        )
        sim_canvas.create_text(
            cx, cy - DOT_R - 6,
            text="KLM", fill="#44ff77", font=("Consolas", 7, "bold"), tags="dots"
        )

    # BLUE dot – extrapolated impact point on goal boundary
    if sim_impact_pos is not None:
        cx, cy = sim_impact_pos
        # Draw a larger pulsing ring to make impact stand out
        sim_canvas.create_oval(
            cx - DOT_R - 4, cy - DOT_R - 4, cx + DOT_R + 4, cy + DOT_R + 4,
            fill="", outline="#4488ff", width=1, tags="dots"
        )
        sim_canvas.create_oval(
            cx - DOT_R, cy - DOT_R, cx + DOT_R, cy + DOT_R,
            fill="#3366ff", outline="#88aaff", width=2, tags="dots"
        )
        sim_canvas.create_text(
            cx, cy - DOT_R - 6,
            text="IMP", fill="#6699ff", font=("Consolas", 7, "bold"), tags="dots"

```

```

    )

    # Draw a dashed line from Kalman position to impact point
    if sim_kalman_pos is not None:
        kx, ky = sim_kalman_pos
        sim_canvas.create_line(
            kx, ky, cx, cy,
            fill="#446688", width=1, dash=(4, 3), tags="dots"
        )

def build_simulation_canvas():
    """
    Draw the permanent background elements of the simulation canvas:
    dark background, zone grid lines, zone number labels, and legend.
    Called once at startup.
    """
    # Dark background
    sim_canvas.create_rectangle(0, 0, SIM_W, SIM_H, fill="#0d0d0d", outline="")

    # Zone grid lines - 4 vertical + 2 horizontal dividers (scaled to canvas)
    grid_color = "#2a2a2a"
    for xv in [X1, X2, X3, X4]:
        sx = int(xv * SCALE_X)
        sim_canvas.create_line(sx, 0, sx, SIM_H, fill=grid_color, width=1)
    for yv in [Y1, Y2]:
        sy = int(yv * SCALE_Y)
        sim_canvas.create_line(0, sy, SIM_W, sy, fill=grid_color, width=1)

    # Zone number labels - positioned at cell centres matching the 5x3 grid
    label_color = "#333333"
    # Column centres in camera pixels
    cx = [COL//2, COL + COL//2, COL*2 + COL//2, COL*3 + COL//2, COL*4 + COL//2]
    # Row centres in camera pixels
    cy = [ROW//2, ROW + ROW//2, ROW*2 + ROW//2]
    labels = {
        (0, 0): 7, (1, 0): 6, (2, 0): 5, (3, 0): 4, (4, 0): 3,
        (0, 1): 8, (1, 1): 7, (2, 1): 5, (3, 1): 3, (4, 1): 2,
        (0, 2): 9, (1, 2): 9, (2, 2): 5, (3, 2): 1, (4, 2): 1,
    }
    for (col, row), zone in labels.items():
        sx, sy = cam_to_sim(cx[col], cy[row])
        sim_canvas.create_text(sx, sy, text=str(zone),
                               fill=label_color, font=("Consolas", 10, "bold"))

    # Legend - bottom-left corner
    legend_y = SIM_H - 14
    sim_canvas.create_oval(6, legend_y-5, 16, legend_y+5,
                           fill="#ff3333", outline="")
    sim_canvas.create_text(20, legend_y, anchor="w", text="Detected",
                           fill="#ff5555", font=("Consolas", 7))

    sim_canvas.create_oval(76, legend_y-5, 86, legend_y+5,
                           fill="#33ff66", outline="")
    sim_canvas.create_text(90, legend_y, anchor="w", text="Kalman",
                           fill="#44ff77", font=("Consolas", 7))

    sim_canvas.create_oval(146, legend_y-5, 156, legend_y+5,
                           fill="#3366ff", outline="")
    sim_canvas.create_text(160, legend_y, anchor="w", text="Impact",
                           fill="#6699ff", font=("Consolas", 7))

# =====
# CAMERA FUNCTIONS
# =====

def get_cameras():
    """Scan indices 0-9 and return list of working camera index strings."""
    available_cameras = []
    for i in range(10):
        cap = cv2.VideoCapture(i, cv2.CAP_DSHOW)
        if cap.isOpened():
            available_cameras.append(str(i))
            cap.release()
    return available_cameras

def init_cameras():

```

```

"""Auto-detect cameras on startup and populate the dropdown."""
global camera_x
try:
    cameras = get_cameras()
    if cameras:
        x_camera_PortComboBox.configure(values=cameras)
        x_camera_PortComboBox.set(cameras[0])
        app.update()
        camera_x_change(x_camera_PortComboBox.get())
    else:
        x_camera_PortComboBox.set("No Cameras")
except Exception as e:
    print(f"Error initializing cameras: {e}")

def camera_x_change(camera_index):
    """Release current camera and open new one at selected index."""
    global camera_x
    try:
        if camera_x is not None and camera_x.isOpened():
            camera_x.release()
        camera_x = cv2.VideoCapture(int(camera_index), cv2.CAP_DSHOW)
        camera_x.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        camera_x.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        camera_x.set(cv2.CAP_PROP_FPS, 30)
        print(f"Camera X changed to index {camera_index}")
    except Exception as e:
        print(f"Error changing Camera X: {e}")

# =====
# CALIBRATION FUNCTIONS
# =====

def start_calibration():
    """Entry point for calibration – opens OpenCV trackbar window."""
    global calibrating_camera, calibration_mode, calibration_window_open, myColorFinder_calibration
    if camera_x is None:
        print("No Camera X available for calibration")
        return
    calibrating_camera = 'x'
    calibration_mode = True
    calibration_window_open = True
    myColorFinder_calibration = ColorFinder(True)
    print("Camera X calibration mode started – adjust sliders, press Q to save.")
    calibration_loop()

def calibration_loop():
    """
    Blocking loop showing 4-panel calibration view.
    Press Q to read trackbar values and save to hsvVals_x.
    Note: freezes GUI until Q is pressed – click the OpenCV window first.
    """
    global calibration_mode, calibration_window_open, hsvVals_x
    global myColorFinder_calibration, calibrating_camera, myColorFinder_x

    camera = camera_x
    current_hsvVals = hsvVals_x.copy()
    window_title = "Camera X Color Calibration"

    while calibration_window_open and camera is not None:
        ret, frame = camera.read()
        if not ret:
            break
        frame_color, mask = myColorFinder_calibration.update(frame, current_hsvVals)
        frame_contour, contours = cvzone.findContours(frame, mask, minArea=1000)
        frame_stack = cvzone.stackImages([frame, frame_color, mask, frame_contour], 2, 0.5)
        cv2.imshow(window_title, frame_stack)

        key = cv2.waitKey(1) & 0xFF
        if key == ord('q') or key == ord('Q'):
            hsvVals_x = {
                'hmin': cv2.getTrackbarPos("Hue Min", "TrackBars"),
                'smin': cv2.getTrackbarPos("Sat Min", "TrackBars"),
                'vmin': cv2.getTrackbarPos("Val Min", "TrackBars"),
                'hmax': cv2.getTrackbarPos("Hue Max", "TrackBars"),
                'smax': cv2.getTrackbarPos("Sat Max", "TrackBars"),
                'vmax': cv2.getTrackbarPos("Val Max", "TrackBars"),
            }

```

```

    }
    print(f"Camera X calibration saved: {hsvVals_x}")
    break

cv2.destroyAllWindows()
calibration_window_open = False
calibration_mode = False
calibrating_camera = None
myColorFinder_x = ColorFinder(False)
print("Calibration ended. Returning to normal operation.")

# =====
# MAIN CAMERA + KALMAN LOOP
# Runs every 33ms via app.after() - processes detection, Kalman, motor, GUI
# =====

def update_camera_feeds():
    """
    Core processing loop:
    1. Read camera frame
    2. Color detection → raw ball position
    3. Kalman predict (every frame) + correct (when detected)
    4. State machine → kick detection → extrapolation → motor command
    5. Update OpenCV overlays + tkinter camera label
    6. Update simulation canvas dots
    """
    global camera_running, x_value, y_value
    global kalman_state, predicted_zone, frames_without_ball
    global sim_detected_pos, sim_kalman_pos, sim_impact_pos
    global last_detected, zone_transmitted, kick_cooldown, system_armed
    global kick_confirm_count, kick_cooldown

    if not camera_running:
        return
    if camera_x is None:
        app.after(16, update_camera_feeds)
        return
    if calibration_mode:
        app.after(16, update_camera_feeds)
        return

    ret1, frame_x = camera_x.read()

    if ret1:
        detected_center = None
        impact_point = None
        impact_zone = None

        # — COLOR DETECTION —
        frame_x_color, mask_x = myColorFinder_x.update(frame_x, hsvVals_x)
        frame_x_contour, contours_x = cvzone.findContours(frame_x, mask_x, minArea=1000)

        if contours_x:
            x_value = contours_x[0]['center'][0]
            y_value = contours_x[0]['center'][1]
            detected_center = (x_value, y_value)
            frames_without_ball = 0

            # Update RED dot in simulation canvas
            sim_detected_pos = cam_to_sim(x_value, y_value)
        else:
            frames_without_ball += 1
            sim_detected_pos = None # hide RED dot when ball not detected

        # — KALMAN PREDICT —
        predicted = kalman.predict()
        pred_x = int(predicted[0])
        pred_y = int(predicted[1])
        pred_vx = float(predicted[2])
        pred_vy = float(predicted[3])
        pred_ax = float(predicted[4])
        pred_ay = float(predicted[5])
        speed = np.sqrt(pred_vx**2 + pred_vy**2)

        # Update GREEN dot - always visible once filter has started
        if kalman_state != STATE_WAITING:

```

```

sim_kalman_pos = cam_to_sim(pred_x, pred_y)

# — KALMAN CORRECT —————
# Only correct when ball has moved more than MIN_MOVEMENT pixels since
# last frame — prevents noise jitter from generating fake velocity
if detected_center is not None:
    measurement = np.array([[np.float32(detected_center[0])],
                             [np.float32(detected_center[1])]])

    should_correct = True
    if last_detected is not None:
        dx = detected_center[0] - last_detected[0]
        dy = detected_center[1] - last_detected[1]
        pixel_movement = np.sqrt(dx**2 + dy**2)
        if pixel_movement < MIN_MOVEMENT:
            should_correct = False # treat as noise, skip correction

    if should_correct:
        kalman.correct(measurement)

    last_detected = detected_center
    detected_trail.append(detected_center)
    if len(detected_trail) > MAX_TRAIL:
        detected_trail.pop(0)

predicted_trail.append((pred_x, pred_y))
if len(predicted_trail) > MAX_TRAIL:
    predicted_trail.pop(0)

# — STATE MACHINE —————
if kalman_state == STATE_WAITING:
    kalman.processNoiseCov = np.eye(6, dtype=np.float32) * PROCESS_NOISE_CALM

    if kick_cooldown > 0:
        kick_cooldown -= 1
        kick_confirm_count = 0
        # Hard zero during cooldown — filter just reset, nothing is real
        kalman.statePost[2] = 0.0
        kalman.statePost[3] = 0.0
        kalman.statePost[4] = 0.0
        kalman.statePost[5] = 0.0

    elif speed <= KICK_VELOCITY_THRESHOLD:
        # Ball is stationary — gently bleed off any accumulated noise
        kick_confirm_count = 0
        kalman.statePost[2] *= 0.3
        kalman.statePost[3] *= 0.3
        kalman.statePost[4] *= 0.3
        kalman.statePost[5] *= 0.3

    elif detected_center is not None and speed > KICK_VELOCITY_THRESHOLD:
        # Speed rising — count consecutive frames before confirming kick
        kick_confirm_count += 1
        if kick_confirm_count >= KICK_CONFIRM_FRAMES:
            kick_confirm_count = 0
            kalman_state = STATE_TRACKING
            kalman.processNoiseCov = np.eye(6, dtype=np.float32) * PROCESS_NOISE_REACTIVE
            print(f"Kick confirmed! Speed: {speed:.1f} px/frame over {KICK_CONFIRM_FRAMES}
frames")

        else:
            kick_confirm_count = 0

    elif kalman_state == STATE_TRACKING:
        if speed > 1:
            impact_point, impact_zone = extrapolate_to_goal(
                pred_x, pred_y, pred_vx, pred_vy, pred_ax, pred_ay
            )
            if impact_point is not None:
                sim_impact_pos = cam_to_sim(*impact_point)
            else:
                sim_impact_pos = None
            if impact_zone is not None and not zone_transmitted:
                if not system_armed:
                    print(f"Zone {impact_zone} predicted but system not armed — press Ready
first")

```

```

        else:
            predicted_zone = impact_zone
            zone_transmitted = True
            position_transmission(impact_zone)
            kalman_state = STATE_WAITING_RESULT
            print(f"Zone {impact_zone} transmitted - waiting for sensor response")
    else:
        # Ball slowed below threshold - return to WAITING without transmitting
        print(f"Ball slowed below threshold ({speed:.1f} px/f) - returning to WAITING")
        sim_impact_pos = None
        kalman_state = STATE_WAITING
        kick_cooldown = KICK_COOLDOWN_FRAMES

    elif kalman_state == STATE_PREDICTED:
        impact_zone = predicted_zone

    elif kalman_state == STATE_WAITING_RESULT:
        impact_zone = predicted_zone
        poll_sensor_response()

# --- DRAW ZONE GRID -----
draw_zone_grid(frame_x_contour)

# --- DRAW TRAILS -----
for i, point in enumerate(detected_trail):
    alpha = int(255 * (i / MAX_TRAIL))
    cv2.circle(frame_x_contour, point, 4, (0, alpha, 0), -1)
for i, point in enumerate(predicted_trail):
    alpha = int(255 * (i / MAX_TRAIL))
    cv2.circle(frame_x_contour, point, 4, (alpha, 0, 0), -1)

# --- DRAW VELOCITY ARROW -----
if kalman_state == STATE_TRACKING:
    arrow_end = (int(pred_x + pred_vx * 8), int(pred_y + pred_vy * 8))
    cv2.arrowedLine(frame_x_contour, (pred_x, pred_y), arrow_end,
                    (0, 165, 255), 2, tipLength=0.3)

# --- DRAW IMPACT PREDICTION -----
if impact_point is not None and impact_zone is not None:
    cv2.line(frame_x_contour, (pred_x, pred_y), impact_point, (0,255,255), 1)
    cv2.circle(frame_x_contour, impact_point, 15, (0,0,255), 3)
    cv2.putText(frame_x_contour, f"Zone {impact_zone}",
                (impact_point[0]+18, impact_point[1]),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)

# --- HIGHLIGHT PREDICTED ZONE -----
# Each zone maps to its pixel bounding box (x1, y1, x2, y2)
# Zones 1 and 9 each span two columns; col2 bottom/top rows have no zone
if impact_zone is not None:
    zone_boxes = {
        1: [(X3, Y2, FRAME_W, FRAME_H)], # bottom-right (cols 3-4)
        2: [(X4, Y1, FRAME_W, Y2)], # mid col 4
        3: [(X4, 0, FRAME_W, Y1)], # top col 4
        4: [(X3, Y1, FRAME_W, Y2)], # mid cols 3-4 (two rects)
        5: [(X3, 0, X4, Y1)], # top col 3
        6: [(X2, 0, X3, FRAME_H)], # full center column
        7: [(X1, 0, X2, Y1)], # top col 1
        8: [(X1, 0, X2, Y1)], # top col 1
        9: [(0, Y1, X1, Y2)], # mid col 0
    }
    if impact_zone in zone_boxes:
        overlay = frame_x_contour.copy()
        for (bx1, by1, bx2, by2) in zone_boxes[impact_zone]:
            cv2.rectangle(overlay, (bx1, by1), (bx2, by2), (0, 0, 255), -1)
            cv2.addWeighted(overlay, 0.25, frame_x_contour, 0.75, 0, frame_x_contour)

# --- HUD OVERLAY -----
cv2.putText(frame_x_contour, f"State : {kalman_state}",
            (10,30), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255,255,255), 2)
cv2.putText(frame_x_contour, f"Speed : {speed:.1f} px/f",
            (10,52), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255,255,255), 2)
if impact_zone:
    cv2.putText(frame_x_contour, f"Target: Zone {impact_zone}",
                (10,74), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0,0,255), 2)

```

```

# --- GUI FRAME UPDATE -----
frame_x_display = cv2.cvtColor(frame_x_contour, cv2.COLOR_BGR2RGB)
img_x           = Image.fromarray(frame_x_display)
imgtk_x         = customtkinter.CTkImage(light_image=img_x, dark_image=img_x, size=(425,325))
camera_x_label.configure(image=imgtk_x)
camera_x_label.image = imgtk_x

# --- UPDATE SIMULATION CANVAS -----
update_simulation_canvas()

app.after(16, update_camera_feeds)

# =====
# SERIAL / MOTOR FUNCTIONS
# =====

def getports():
    """Scan available COM ports and populate the dropdown."""
    try:
        ports_List.clear()
        for port in serial.tools.list_ports.comports():
            ports_List.append(port.name)
        picPortComboBox.configure(values=ports_List)
        picPortComboBox.set(ports_List[0])
    except:
        picPortComboBox.set("No Ports Available")

def port_change(save_port_name):
    """Called when user selects a new port from the dropdown."""
    global manual_port_change
    manual_port_change = True
    pic_serial_connect(save_port_name)

def pic_serial_connect(portname):
    """Open serial connection to PIC at 115200 baud."""
    global settings_done, manual_port_change
    if settings_done:
        return
    elif manual_port_change:
        if pic_Serial_Port.is_open:
            pic_Serial_Port.close()
        pic_Serial_Port.port = portname
        pic_Serial_Port.baudrate = 115200
        pic_Serial_Port.open()
        manual_port_change = False
        print(portname, ",", str(pic_Serial_Port.baudrate), "bps")

def default_start():
    """Run once at startup - connects to first available serial port."""
    global previous_position_global, settings_done, manual_port_change
    try:
        if not settings_done:
            getports()
            port_name = picPortComboBox.get()
            manual_port_change = True
            port_change(port_name)
            previous_position_global = 5
            settings_done = True
    except:
        pass

def steps_comparison(previous_position, objective_position):
    """
    Send step delta to PIC.
    Forward: raw difference | Reverse: difference + 128 (sign flag for PIC)
    """
    tx_position = bytearray(1)
    if objective_position > previous_position:
        tx_position[0] = objective_position - previous_position
        pic_Serial_Port.write(tx_position)
        print("Transmitted Position Hex =", f"{hex(tx_position[0])}")
        print("Transmitted Position Dec =", f"{int(tx_position[0])}")
        print("Transmitted Position Binary =", f"{bin(tx_position[0])}")
        sleep(15 / 1000)
    elif objective_position < previous_position:

```

```

        tx_position[0] = (previous_position - objective_position) + 128
        pic_Serial_Port.write(tx_position)
        print("Transmitted Position Hex =", f"{hex(tx_position[0])}")
        print("Transmitted Position Dec =", f"{int(tx_position[0])}")
        print("Transmitted Position Binary =", f"{bin(tx_position[0])}")
        sleep(15 / 1000)
    else:
        print("No Steps Sent, same position as before")

# — KEEPER MOVEMENT LIMIT —————
# Applies only to Kalman predictions – not to manual button presses.
# The keeper always homes to position 5 (center).
# Physical limitations allow a maximum of 2 steps in either direction.

def position_transmission(value):
    """
    Send a move command to the PIC using steps_comparison().
    The PIC expects a 1-byte value where:
    - Forward move: byte = (target - current), MSB = 0
    - Backward move: byte = (current - target) + 128, MSB = 1
    No clamping here – clamp is applied upstream for Kalman predictions only.
    """
    global previous_position_global
    objective = int(value)
    print(f"Transmitting zone {objective} (previous: {previous_position_global})")
    try:
        steps_comparison(previous_position_global, objective)
        previous_position_global = objective
    except Exception as e:
        print(f"Error transmitting zone {value}: {e}")

def poll_sensor_response():
    """
    Called every frame while in STATE_WAITING_RESULT.
    Reads incoming serial bytes and reacts to the three PIC sensor signals:
    0x20 – flex sensor: ball was blocked by keeper ✓
    0x28 – motion sensor: ball got into the net ✗
    0x30 – timeout: no sensor activated
    Does NOT reset Kalman or re-enable buttons – that only happens on Ready press.
    """
    global system_armed
    try:
        if pic_Serial_Port.is_open and pic_Serial_Port.in_waiting > 0:
            rx_byte = pic_Serial_Port.read(1)[0]
            print(f"Sensor response received: {hex(rx_byte)}")

            if rx_byte == 0x20:
                print("✓ BLOCKED – Flex sensor triggered. Keeper stopped the ball.")
                ready_button.configure(text="BLOCKED ✓", fg_color="#005500")
                system_armed = False # disarm – requires Ready press for next kick
                set_manual_buttons(True)

            elif rx_byte == 0x28:
                print("✗ GOAL – Motion sensor triggered. Ball entered the net.")
                ready_button.configure(text="GOAL ✗", fg_color="#550000")
                system_armed = False # disarm – requires Ready press for next kick
                set_manual_buttons(True)

            elif rx_byte == 0x30:
                print("△ NO SENSOR – Timeout. No sensor was activated.")
                ready_button.configure(text="NO SENSOR △", fg_color="#555500")
                system_armed = False # disarm – requires Ready press for next kick
                set_manual_buttons(True)

            else:
                print(f"Unknown byte received: {hex(rx_byte)}")

    except Exception as e:
        print(f"Error reading sensor response: {e}")

def set_manual_buttons(enabled):
    """
    Enable or disable all 9 manual position buttons.
    Called by ready_sequence – buttons are disabled until keeper reaches home.
    """
    state = "normal" if enabled else "disabled"

```

```

for btn in [button1, button2, button3, button4, button5,
            button6, button7, button8, button9]:
    btn.configure(state=state)

def ready_sequence():
    """
    Ready button sequence – three steps:
    1. Send homing command (0xBD = 189) to PIC and mark button as busy
    2. Poll serial port waiting for PIC to reply with 0x20 (home confirmed)
    3. Once 0x20 is received, reset Kalman and mark button as ready
    Polling is done via app.after() so the GUI stays responsive while waiting.
    """
    global previous_position_global

    if not pic_Serial_Port.is_open:
        print("Serial port not open – cannot start homing")
        return

    # Send homing command and update button to show it is working
    tx_position = bytearray(1)
    tx_position[0] = 0xBD # 189 decimal – PIC homing trigger code
    pic_Serial_Port.write(tx_position)
    print("Homing Sequence Started")
    print(f"Transmitted Code: {hex(tx_position[0])}")

    # Disable Ready button and all manual buttons while homing is in progress
    ready_button.configure(text="Homing...", fg_color="orange")
    set_manual_buttons(False)
    previous_position_global = 5

    # Start polling for the 0xBD confirmation byte from PIC
    app.after(100, wait_for_home_confirmation)

def wait_for_home_confirmation():
    """
    Called repeatedly every 100ms via app.after() until PIC sends 0xBD.
    Non-blocking – GUI remains fully responsive while waiting.
    0xBD – PIC homing complete confirmation byte.
    """
    global system_armed
    try:
        if pic_Serial_Port.in_waiting > 0:
            rx_byte = pic_Serial_Port.read(1)[0] # read one byte
            print(f"Received byte: {hex(rx_byte)}")

            if rx_byte == 0xBD:
                # PIC confirmed home position reached – arm the system
                print("Home position confirmed by PIC (0xBD received)")
                reset_kalman()
                system_armed = True # only here is the system allowed to transmit
                print("System armed – ready to predict and transmit.")

                # Restore Ready button – manual buttons stay disabled until
                # a sensor result is received (0x20, 0x28, or 0x30)
                ready_button.configure(
                    text="Ready", fg_color="green", state="normal"
                )
                return # stop polling

            else:
                # Received a different byte – log it and keep waiting
                print(f"Unexpected byte while homing: {hex(rx_byte)} – still waiting...")

        # 0xBD not yet received – check again in 100ms
        app.after(100, wait_for_home_confirmation)

    except Exception as e:
        print(f"Error reading serial during homing: {e}")
        ready_button.configure(text="Ready", fg_color="green", state="normal")

def position_calculation(x_camera_value, y_camera_value):
    """
    Fallback raw-coordinate zone mapping using 5-col x 3-row grid.
    Used in WAITING state before a kick is detected.
    Delegates to get_zone() so the logic is always consistent.
    """

```

```

zone = get_zone(x_camera_value, y_camera_value)
if zone is not None:
    position_transmission(zone)
    print(f"Position {zone}")
else:
    print("No zone matched (ball outside valid area)")

# =====
# DEBUG CONSOLE
# =====

class ConsoleRedirector(io.StringIO):
    """Redirects stdout/stderr to GUI textbox while mirroring to terminal."""
    def __init__(self, widget):
        super().__init__()
        self.widget = widget
    def write(self, text):
        if text.strip():
            self.widget.insert("end", text + "\n")
            self.widget.see("end")
        original_stdout.write(text)
    def flush(self):
        pass

def toggle_console():
    """Open or close the floating debug console window (Ctrl+D)."""
    global console_window, console_visible
    cv2.waitKey(1)
    if not console_visible:
        console_window = customtkinter.CTkToplevel(app)
        console_window.title("Debug Console")
        console_window.geometry("700x500")
        console_window.protocol("WM_DELETE_WINDOW", toggle_console)
        customtkinter.CTkLabel(console_window, text="Debug Console",
                               font=("", 16, "bold")).pack(pady=10)
        console_text = customtkinter.CTkTextbox(console_window, width=680, height=400,
                                                font=("Consolas", 11))
        console_text.pack(padx=10, pady=5)
        customtkinter.CTkButton(console_window, text="Clear",
                                command=lambda: console_text.delete("0.0", "end"),
                                width=150).pack(pady=10)
        sys.stdout = sys.stderr = ConsoleRedirector(console_text)
        console_visible = True
        console_button.configure(text="Close Console", fg_color="red")
        print("=== Debug Console ===")
        print(f"Camera X HSV : {hsvVals_x}")
        print(f"Kalman State : {kalman_state}")
        print(f"Motor Position : {previous_position_global}")
        print("=" * 20)
    else:
        sys.stdout = sys.stderr = original_stdout
        console_window.destroy()
        console_window = None
        console_visible = False
        console_button.configure(text="Open Console", fg_color="gray")

# =====
# APP CLOSE
# =====

def close_app(event=None):
    """Clean shutdown - release camera, close serial port, destroy window."""
    global camera_running, console_visible
    if console_visible:
        sys.stdout = original_stdout
    camera_running = False
    if camera_x is not None:
        camera_x.release()
    if pic_Serial_Port.is_open:
        pic_Serial_Port.close()
    cv2.destroyAllWindows()
    app.destroy()

# — KEY BINDINGS —
app.bind("<Escape>", close_app)
app.bind("<Control-d>", lambda event: toggle_console())

```

```

app.bind("<Control-D>", lambda event: toggle_console())
app.bind("<Control-y>", lambda event: start_calibration())
app.bind("<Control-Y>", lambda event: start_calibration())
app.bind("<Control-r>", lambda event: reset_kalman())
app.bind("<Control-R>", lambda event: reset_kalman())

# =====
# GUI LAYOUT
# =====

# Camera feed and simulation canvas share the same left edge so they
# are perfectly aligned. Both are 425px wide.
# App = 1280px wide → 1280 - 425 - 10 = 845 gives a 10px right margin.
CAM_X = 845 # left edge of both widgets
CAM_TOP = 7 # top of camera feed
SIM_TOP = CAM_TOP + 325 + 18 # camera height (325) + gap for label (18)

# Camera feed display – fixed pixel position so width is never clipped
camera_x_label = customtkinter.CTkLabel(app, text="")
camera_x_label.place(x=CAM_X, y=CAM_TOP)

# — SIMULATION CANVAS —
# Placed directly below the camera feed, same width (425px).
# Uses absolute x/y so canvas is never clipped by relx rounding.
sim_label = customtkinter.CTkLabel(app, text="Kalman Simulation",
                                   font=("Consolas", 11), text_color="#888888")
sim_label.place(x=CAM_X, y=SIM_TOP - 16)

sim_canvas = Canvas(app, width=SIM_W, height=SIM_H,
                    bg="#0d0d0d", highlightthickness=1,
                    highlightbackground="#333333")
sim_canvas.place(x=CAM_X + 210, y=SIM_TOP + 100)

# Draw static background elements once
build_simulation_canvas()

# Manual control buttons (3x3 grid)
manualTitle = customtkinter.CTkLabel(app, text="Manual Control Area", font=(" ", 20))
manualTitle.place(relx=0.35, rely=0.225, anchor=customtkinter.CENTER)

# Manual buttons start disabled – enabled only after keeper reaches home (0x20)
button1 = customtkinter.CTkButton(app, text="Position 1", width=100, height=85, command=lambda:
position_transmission(1))
button1.place(relx=0.25, rely=0.325, anchor=customtkinter.CENTER)
button2 = customtkinter.CTkButton(app, text="Position 2", width=100, height=85, command=lambda:
position_transmission(2))
button2.place(relx=0.35, rely=0.325, anchor=customtkinter.CENTER)
button3 = customtkinter.CTkButton(app, text="Position 3", width=100, height=85, command=lambda:
position_transmission(3))
button3.place(relx=0.45, rely=0.325, anchor=customtkinter.CENTER)
button4 = customtkinter.CTkButton(app, text="Position 4", width=100, height=85, command=lambda:
position_transmission(4))
button4.place(relx=0.25, rely=0.475, anchor=customtkinter.CENTER)
button5 = customtkinter.CTkButton(app, text="Position 5", width=100, height=85, command=lambda:
position_transmission(5))
button5.place(relx=0.35, rely=0.475, anchor=customtkinter.CENTER)
button6 = customtkinter.CTkButton(app, text="Position 6", width=100, height=85, command=lambda:
position_transmission(6))
button6.place(relx=0.45, rely=0.475, anchor=customtkinter.CENTER)
button7 = customtkinter.CTkButton(app, text="Position 7", width=100, height=85, command=lambda:
position_transmission(7))
button7.place(relx=0.25, rely=0.625, anchor=customtkinter.CENTER)
button8 = customtkinter.CTkButton(app, text="Position 8", width=100, height=85, command=lambda:
position_transmission(8))
button8.place(relx=0.35, rely=0.625, anchor=customtkinter.CENTER)
button9 = customtkinter.CTkButton(app, text="Position 9", width=100, height=85, command=lambda:
position_transmission(9))
button9.place(relx=0.45, rely=0.625, anchor=customtkinter.CENTER)

# Left sidebar buttons
calButton = customtkinter.CTkButton(app, text="Camera Calibration", width=120, height=85,
command=start_calibration)
calButton.place(relx=0.025, rely=0.65, anchor=customtkinter.SW)

# Ready button – green when idle, orange while homing, disabled until PIC confirms
ready_button = customtkinter.CTkButton(app, text="Ready", width=120, height=85,

```

```

                                command=ready_sequence, fg_color="green")
ready_button.place(relx=0.025, rely=0.80, anchor=customtkinter.SW)

commButton = customtkinter.CTkButton(app, text="Update Ports", width=120, height=85, command=getports)
commButton.place(relx=0.025, rely=0.95, anchor=customtkinter.SW)

reset_button = customtkinter.CTkButton(app, text="Reset Kalman", width=120, height=60,
                                command=reset_kalman, fg_color="dark green")
reset_button.place(relx=0.025, rely=0.35, anchor=customtkinter.SW)

# Port dropdowns
picPortComboBox = customtkinter.CTkComboBox(app, width=150, command=port_change)
picPortComboBox.place(relx=0.025, rely=0.05, anchor=customtkinter.NW)
pic_port_label = customtkinter.CTkLabel(app, text="PIC Port", font=("", 14))
pic_port_label.place(relx=0.055, rely=0.01, anchor=customtkinter.NW)

x_camera_PortComboBox = customtkinter.CTkComboBox(app, width=150, command=camera_x_change)
x_camera_PortComboBox.place(relx=0.15, rely=0.05, anchor=customtkinter.NW)
x_camera_port_label = customtkinter.CTkLabel(app, text="X Camera Port", font=("", 14))
x_camera_port_label.place(relx=0.17, rely=0.01, anchor=customtkinter.NW)

# Mode switch - Manual ↔ Camera (Kalman) control

# Console toggle
console_button = customtkinter.CTkButton(app, text="Open Console", width=120, height=60,
                                command=toggle_console, fg_color="gray")
console_button.place(relx=0.025, rely=0.50, anchor=customtkinter.SW)

# — STARTUP SEQUENCE —————
default_start()
app.after(100, init_cameras)
app.after(200, update_camera_feeds)

app.mainloop()

```

Appendix D: Assembly Code

Robokeeper_V.3_Main.asm

```

;*****
;
; Filename: Robokeeper_V.3_Main.asm
; Date: September 23 2025
; File Version: 3
; Author: Rahiel Rodriguez
; Company: Idaho State University
; Description: Firmware to control a robotic goalkeeper using a PIC16F1788.
;
;*****
;*****
;
; Revision History:
;1:
;
;
;
;
;
;
;
;
;*****

; PIC16F1788 Configuration Bit Settings

; Assembly source line config statements

#include "p16f1788.inc"
#include "Robokeeper_V.3_Setup.inc"
#include "Timing_Cycles_1.inc"
#include "Testing_Timing.inc"

; CONFIG1
; __config 0x3FDF
; CONFIG1
; __config 0x39C4
__CONFIG __CONFIG1, _FOSC_INTOSC & _WDTE_OFF & _PWRTE_OFF & _MCLRE_ON & _CP_OFF & _CPD_OFF & _BOREN_OFF
& _CLKOUTEN_OFF & _IESO_ON & _FCMEN_ON
; CONFIG2
; __config 0x3FFF
__CONFIG __CONFIG2, _WRT_OFF & _VCAPEN_OFF & _PLLEN_ON & _STVREN_ON & _BORV_LO & _LPBOR_OFF & _LVP_ON

ORG H'000'
GOTO SETUP

ORG H'004'
GOTO INTERRUPT

SETUP
CALL INITIALIZE
GOTO MAIN
; GOTO ENABLE_ROUTINE
;-----ISR-----
INTERRUPT
MOVWF W_SAVE
MOVFW STATUS
MOVWF STATUS_SAVE
CLRF STATUS
BANKSEL PIR1
BTFSC PIR1,5
GOTO RECEIVE_ISR
; BANKSEL INTCON
; BTFSC INTCON,0
; GOTO SENSOR_SELECT
; GOTO FLEX_ISR
; BTFSC IOCCF,1
; GOTO MOTION_SENS_ISR
GOTO END_ISR

```

```

END_ISR
    BANKSEL    PIR1
    CLRF       PIR1
;   BANKSEL    PIR1
;   CLRF       PIR1
    BANKSEL    STATUS
    MOVFW     STATUS_SAVE
    MOVWF     STATUS
    MOVFW     W_SAVE
    RETFIE

RECEIVE_ISR
    BANKSEL    RCREG
    MOVFW     RCREG
    BANKSEL    RECEIVE_SAVE
    MOVWF     RECEIVE_SAVE
    MOVFW     RECEIVE_SAVE
    MOVWF     POSITION_DATA
    BANKSEL    MOTION_FLAGS_REG
    BSF       MOTION_FLAGS_REG,0
    GOTO      END_ISR

;-----MAIN-----
DIRECTION_DELAY      ;Delay loop to make a correct direction change
    MOVLW     0X0F
    BANKSEL    COUNTER2
    MOVWF     COUNTER2

DIR_LOOP2
    MOVLW     0X54
    MOVWF     COUNTER1
DIR_LOOP1
    DECFSZ    COUNTER1
    GOTO      DIR_LOOP1
    DECFSZ    COUNTER2
    GOTO      DIR_LOOP2
    RETURN

OVERFLOW_CHECK_ROUTINE
;   BANKSEL    IOCCF
;   MOVFW     IOCCF
;   MOVWF     SENSORS_F_REG
    BANKSEL    PORTA
    BTFSS     STATUS,0
    GOTO      SPEED_CHECK_ROUTINE
;   BSF       PORTB,0
;   CLRF      MOTION_FLAGS_REG
    MOVLW     H'01'
    MOVWF     SPEED_REG1
    MOVLW     H'01'
    MOVWF     SPEED_REG2
    CLRF      MOTION_FLAGS_REG
;   GOTO      POSITION_VERIFICATION
;   GOTO      SIGNAL_ROUTINE
;   BCF       PORTB,0
    GOTO      SENSOR_SELECT

SPEED_CHECK_ROUTINE
    BANKSEL    PORTA
;   BSF       PORTB,0
    CALL      SPEED_CHECK
    GOTO      CALIBRATION_SIGNAL_ROUTINE

CALIBRATION_SIGNAL_ROUTINE
    BANKSEL    PORTA
    BSF       PORTA,0
    CALL      CALIBRATION_DELAY
    BCF       PORTA,0
    CALL      CALIBRATION_DELAY
    DECFSZ    POSITION_COUNTER
    GOTO      CALIBRATION_SIGNAL_ROUTINE
    MOVLW     H'18'
    MOVWF     POSITION_COUNTER
;   GOTO      POSITION_VERIFICATION
    BTFSC     SPEED_REG2,5
    GOTO      SIGNAL_ROUTINE

```

```

    BTFSC    MOTION_FLAGS_REG,1
    GOTO     FORWARD_SHIFT1
    BTFSC    MOTION_FLAGS_REG,2
    GOTO     REVERSE_SHIFT1
    BTFSC    MOTION_FLAGS_REG,3
    GOTO     FORWARD_SHIFT2
    BTFSC    MOTION_FLAGS_REG,4
    GOTO     REVERSE_SHIFT2
    GOTO     OVERFLOW_CHECK_ROUTINE

FORWARD_SHIFT1
    LSLF     SPEED_REG1,1
    MOVFW    SPEED_REG1
;   MOVWF    PORTB
    GOTO     OVERFLOW_CHECK_ROUTINE
REVERSE_SHIFT1
    LSRF     SPEED_REG1,1
    MOVFW    SPEED_REG1
;   MOVWF    PORTB
    GOTO     OVERFLOW_CHECK_ROUTINE
FORWARD_SHIFT2
;   BSF      PORTB,0
    LSLF     SPEED_REG2,1
    MOVFW    SPEED_REG2
;   MOVWF    PORTB
    GOTO     OVERFLOW_CHECK_ROUTINE
REVERSE_SHIFT2
;   BCF      PORTB,0
    LSRF     SPEED_REG2,1
    MOVFW    SPEED_REG2
;   MOVWF    PORTB
    GOTO     OVERFLOW_CHECK_ROUTINE

SIGNAL_ROUTINE
;   BSF      PORTB,0
    BANKSEL  PORTA
    BSF      PORTA,0
    CALL     SIGNAL_DELAY
    BCF      PORTA,0
    CALL     SIGNAL_DELAY
    DECFSZ   CURRENT_STEPS
    GOTO     SIGNAL_ROUTINE
    MOVFW    STEPS_OBJ
    MOVWF    CURRENT_STEPS
    DECFSZ   STEP_MULT_COUNT
    GOTO     SIGNAL_ROUTINE
    MOVFW    STEPS_OBJ
    MOVWF    CURRENT_STEPS
    MOVLW    STEPS_MULTIPLIER
    MOVWF    STEP_MULT_COUNT
    DECFSZ   COMP_MULT_COUNT
    GOTO     SIGNAL_ROUTINE
    MOVFW    STEPS_OBJ
    MOVWF    CURRENT_STEPS
    MOVLW    STEPS_MULTIPLIER
    MOVWF    STEP_MULT_COUNT
    BSF      MOTION_FLAGS_REG,4
    BCF      MOTION_FLAGS_REG,3
    LSRF     SPEED_REG2,1
    GOTO     OVERFLOW_CHECK_ROUTINE
;   CLRF     MOTION_FLAGS_REG
;   GOTO     POSITION_VERIFICATION

SENSOR_SELECT
    BANKSEL  PORTC
    BTFSC    PORTC,0
    GOTO     FLEX_SENS_ROUTINE
    BTFSC    PORTC,1
    GOTO     MOTION_SENS_ROUTINE
    BSF      PORTB,2
    MOVLW    H'E8'
    MOVWF    TIMING_REG
    CALL     SENSOR_CHECK_DELAY
    DECFSZ   MISSED_T_REG
    GOTO     SENSOR_SELECT
    CLRF     PORTB

```

```

    MOVLW    H'FF'
    MOVWF    MISSED_T_REG
    MOVLW    H'30'
    BANKSEL  TXREG
    MOVWF    TXREG
    GOTO     POSITION_VERIFICATION

FLEX_SENS_ROUTINE
    BANKSEL  PORTB
    BCF      PORTB,2
    BSF      PORTB,0
    MOVLW    H'FF'
    MOVWF    MISSED_T_REG
    BCF      PORTB,2
    MOVLW    H'20'
    BANKSEL  TXREG
    MOVWF    TXREG
    GOTO     POSITION_VERIFICATION

MOTION_SENS_ROUTINE
    BANKSEL  PORTB
    BCF      PORTB,2
    BSF      PORTB,4
    MOVLW    H'FF'
    MOVWF    MISSED_T_REG
    BCF      PORTB,2
    MOVLW    H'28'
    BANKSEL  TXREG
    MOVWF    TXREG
    GOTO     POSITION_VERIFICATION

DIRECTION_CHANGE
    BANKSEL  PORTC
    CLRF     MOTION_FLAGS_REG
    BSF      MOTION_FLAGS_REG,1
    MOVLW    H'80'                ;Direction Bit Check
    SUBWF    POSITION_DATA,0
    MOVWF    SUB_RESULT_SAVE      ;Compares Direction Bit to see if is set or cleared
    BTFS    STATUS,0
    GOTO     RIGHT_DIRECTION      ;Negative Result
    GOTO     LEFT_DIRECTION       ;Positive Result

RIGHT_DIRECTION
    BANKSEL  PORTA
    BCF      PORTA, 1
    MOVFW    POSITION_DATA        ;Direction Bit was cleared, loads original data
    MOVWF    STEPS_OBJ
    CALL     DIRECTION_DELAY      ;Adds delay needed for correct direction change
;   MOVFW    STEPS_OBJ
;   MOVWF    PORTB
    MOVFW    STEPS_OBJ
    MOVWF    CURRENT_STEPS
    CALL     COMPENSATION_ROUTINE
    BCF      STATUS,0
;   BANKSEL  PORTA
;   BSF      PORTB,0
    GOTO     OVERFLOW_CHECK_ROUTINE ;Calls ramp up frequency routine
;   GOTO     SIGNAL_ROUTINE
;   GOTO     SPEED_CHECK_ROUTINE

LEFT_DIRECTION
    BANKSEL  PORTA
    BSF      PORTA, 1
    MOVFW    SUB_RESULT_SAVE      ;Direction Bit was set, loads subtraction result to save steps info
    MOVWF    STEPS_OBJ
    CALL     DIRECTION_DELAY      ;Adds delay needed for correct direction change
;   MOVFW    STEPS_OBJ
;   MOVWF    PORTB
    MOVFW    STEPS_OBJ
    MOVWF    CURRENT_STEPS
    CALL     COMPENSATION_ROUTINE
;   BANKSEL  PORTA
;   BSF      PORTB,0
    BCF      STATUS,0
    GOTO     OVERFLOW_CHECK_ROUTINE ;Calls ramp up frequency routine
;   GOTO     SIGNAL_ROUTINE

```

```

;   GOTO    SPEED_CHECK_ROUTINE

HOMING_SEQUENCE
    BANKSEL    PORTA
    BSF        PORTA,0    ;Sets RA0 for a certain amount of time
    CALL       HOME_PULSE_DELAY
    BCF        PORTA,0    ;clears RA0 for a certain amount of time
    CALL       HOME_PULSE_DELAY    ;So it can be around %50DC
    CALL       HOME_DIRECTION_CHANGE
    CALL       DIRECTION_DELAY
    BTFSS      PORTA, 2
    GOTO       HOMING_SEQUENCE
    BANKSEL    PORTC
    CLRF       MOTION_FLAGS_REG
    MOVLW      H'BD'
    BANKSEL    TXREG
    MOVWF      TXREG
    GOTO       POSITION_VERIFICATION

HOME_DIRECTION_CHANGE    ;Checks if any Direction Sensor is activated
    BANKSEL    PORTA
    BCF        PORTA,0
    BTFSC      PORTA,3    ;If RA3 is set, go to the RIGHT
    BCF        PORTA,1
    BTFSC      PORTA,4    ;If RA4 is set, go to the LEFT
    BSF        PORTA,1
    RETURN

ENABLE_DELAY    ;Enable Timing Loop
    BANKSEL    COUNTER3
    MOVWF      COUNTER3
ENABLE_LOOP3
    MOVLW      0X80
    MOVWF      COUNTER2
ENABLE_LOOP2
    MOVLW      0X80
    MOVWF      COUNTER1
ENABLE_LOOP1
    DECFSZ     COUNTER1
    GOTO       ENABLE_LOOP1
    DECFSZ     COUNTER2
    GOTO       ENABLE_LOOP2
    DECFSZ     COUNTER3
    GOTO       ENABLE_LOOP3
    RETURN

ENABLE_ROUTINE
    BANKSEL    PORTC
;   BSF        PORTC, 3    ;Set Enable Pin to de-activate the Steper Driver
    MOVLW      0X28
    CALL       ENABLE_DELAY    ;Calls the minimum delay needed to allow Driver to Stop
;   BCF        PORTC, 3    ;Clears Enable Pin to activate the Steper Driver
    MOVLW      0X28
    CALL       ENABLE_DELAY    ;Calls the minimum delay needed to allow Driver to Work

;   MOVFW      SPEED_REG
;   MOVWF      PORTB

    MOVLW      STEPS_MULTIPLIER
    MOVWF      STEP_MULT_COUNT
;   BANKSEL    PORTA
;   BSF        PORTB,0
    GOTO       MAIN

MAIN
    BANKSEL    PORTC
    MOVLW      STEPS_MULTIPLIER
    MOVWF      STEP_MULT_COUNT
    GOTO       POSITION_VERIFICATION

POSITION_VERIFICATION
;-----NO TASK SEQUENCE-----
;   MOVLW      H'DD'
;   BANKSEL    TXREG
;   MOVWF      TXREG
;   MOVLW      0X28

```

```

;      CALL      ENABLE_DELAY
BANKSEL    PORTA
BTFSS      MOTION_FLAGS_REG,0 ;Checks if all CPU Information was received
GOTO       POSITION_VERIFICATION ;If not, do not do anything until all information was received
MOVFW      POSITION_DATA
SUBLW      H'BD'
BTFSC      STATUS,2
GOTO       HOMING_SEQUENCE
GOTO       DIRECTION_CHANGE
BANKSEL     MOTION_FLAGS_REG
CLRF       MOTION_FLAGS_REG
GOTO       POSITION_VERIFICATION
END

```

Robokeeper_V.3_Setup.inc

```

;*****
;
; Filename: RoboGoalkeeper_Setup_v.2.INC
; Date: March 11 2025
; File Version: 1
; Author: RAHIEL RODRIGUEZ
; Company: Idaho State University
; Description: Firmware for Setting up the Robotic Goalkeeper Project
;
;*****
;*****
;
; Revision History:
; 1: INITIAL FILE TO SETUP A PIC16F883 SPECIFICS ON SETUP SHOULD BE
; LISTED HERE AS MODIFIED
; STARTED March 11 2025
;
;
;*****

;-----DEFINE PERSONAL GPR LOCATIONS-----
W_SAVE EQU 0X20
STATUS_SAVE EQU 0X21
DIR_FLAG EQU 0X22
ER_SAVE EQU 0X23
DIR_SAVE EQU 0X24
COUNTER1 EQU 0X25
COUNTER2 EQU 0X26
STEPS_OBJ EQU 0X27
RECEIVE_SAVE EQU 0X28
DATA_SAVE EQU 0X29
POSITION_DATA EQU 0X2A
SUB_RESULT_SAVE EQU 0X2B
COUNTER3 EQU 0X2C
STEP_MULT_COUNT EQU 0X2D
STEPS_LOOP_COUNT EQU 0X2E
FREQ_CYCLES_DONE EQU 0X2F
FREQ_CYCLES_OBJ EQU 0X30
FREQ_STEPS EQU 0X31
CURRENT_STEPS EQU 0X32
TIMING_REG EQU 0X33
OVERFLOW_REG EQU 0X34
SPEED_REG1 EQU 0X35
POSITION_COUNTER EQU 0X36
MOTION_FLAGS_REG EQU 0X37
COMP_MULT_COUNT EQU 0X38
TIMING_FLAGS_REG EQU 0X39
SPEED_REG2 EQU 0X3A
SPEED_REG3 EQU 0X3B
SENSORS_F_REG EQU 0X3C
MISSED_T_REG EQU 0X3D
PROTECTION_REG EQU 0X3E

VALUE_2K EQU H'A4'
VALUE_2.5K EQU H'83'
VALUE_3K EQU H'6C'
VALUE_3.5K EQU H'5D'
VALUE_4K EQU H'51'
VALUE_4.5K EQU H'48'
VALUE_5K EQU H'40'
VALUE_5.5K EQU H'3A'
VALUE_6K EQU H'35'
VALUE_6.5K EQU H'31'
VALUE_7K EQU H'2D'
VALUE_7.5K EQU H'2B'
VALUE_8K EQU H'27'
VALUE_8.5K EQU H'25'
VALUE_9K EQU H'22'
VALUE_9.5K EQU H'20'
VALUE_10K EQU H'1E'
;VALUE_10.5K EQU H'1D'
;VALUE_11K EQU H'1C'

```

```
;VALUE_11.5K      EQU    H'1A'
;VALUE_12K        EQU    H'19'
;VALUE_12.5K      EQU    H'18'
;VALUE_13K        EQU    H'17'
;VALUE_13.5K      EQU    H'16'
;VALUE_14K        EQU    H'15'
;VALUE_14.5K      EQU    H'14'
;VALUE_15K        EQU    H'13'
```

```
STEPS_MULTIPLIER  EQU    H'78'
MISSED_T_VALUE    EQU    H'6D'
```

```
SETUP_CODE        CODE    ;START OF THE FILE
```

```
INITIALIZE
```

```
;-----OSCILLATOR SETUP-----
```

```
MOVLW    H'00'
BANKSEL  T1CON
MOVWF    T1CON
MOVLW    B'11111000' ;set FOSC to 16 MHz
BANKSEL  OSCCON
MOVWF    OSCCON
BANKSEL  OSCTUNE
CLRF     OSCTUNE
MOVLW    B'00000000'
BANKSEL  CLKRCON
MOVWF    CLKRCON
```

```
;-----I/O Setup-----
```

```
MOVLW    B'00001000'
BANKSEL  APFCON1
MOVWF    APFCON1
```

```
MOVLW    H'00'
BANKSEL  PSMC1CON
MOVWF    PSMC1CON
MOVLW    H'00'
BANKSEL  PSMC2CON
MOVWF    PSMC2CON
MOVLW    H'00'
BANKSEL  PSMC3CON
MOVWF    PSMC3CON
MOVLW    H'00'
BANKSEL  PSMC4CON
MOVWF    PSMC4CON
```

```
MOVLW    B'00011100'
BANKSEL  TRISA
MOVWF    TRISA
MOVLW    B'00000000'
BANKSEL  PORTA
MOVWF    PORTA
MOVLW    H'00'
BANKSEL  ANSELA
MOVWF    ANSELA
```

```
MOVLW    H'00'
BANKSEL  TRISB
MOVWF    TRISB
MOVLW    H'00'
BANKSEL  PORTB
MOVWF    PORTB
MOVLW    H'00'
BANKSEL  ANSELB
MOVWF    ANSELB
MOVLW    H'00'
BANKSEL  INLVLB
MOVWF    INLVLB
```

```
MOVLW    B'10000011'
BANKSEL  TRISC
MOVWF    TRISC
MOVLW    B'00000000'
BANKSEL  PORTC
MOVWF    PORTC
MOVLW    H'00'
```

```

BANKSEL ANSEL
MOVWF ANSEL
MOVLW H'00'
BANKSEL WPUC
MOVWF WPUC
MOVLW H'00'
BANKSEL ODCONC
MOVWF ODCONC

;-----TIMER2 Setup-----
; BANKSEL T2CON
; CLRF T2CON
; CLRF TMR2
; MOVLW H'FF'
; BANKSEL PR2
; MOVWF PR2
; BANKSEL PIE1
; CLRF PIE1
; BANKSEL PIR1
; CLRF PIR1
; MOVLW B'00000010'
; BANKSEL T2CON
; MOVWF T2CON
;-----PWM Setup-----

;-----UART Setup-----
;Disable MSSP
MOVLW H'00'
BANKSEL SSPCON1
MOVWF SSPCON1

;----> BAUD RATE Setup
MOVLW B'01001000'
BANKSEL BAUDCON
MOVWF BAUDCON
MOVLW D'67'
BANKSEL SPBRG
MOVWF SPBRG

;----> TX Setup
MOVLW B'00100110'
BANKSEL TXSTA
MOVWF TXSTA

;----> RX Setup
MOVLW B'10010000'
BANKSEL RCSTA
MOVWF RCSTA

;----- GPR LOCATIONS-----
MOVLW H'00'
BANKSEL DIR_FLAG
MOVWF DIR_FLAG
MOVLW H'00'
MOVWF ER_SAVE
MOVLW H'00'
MOVWF STEPS_OBJ
MOVLW H'00'
MOVWF STEP_MULT_COUNT
MOVLW H'00'
MOVWF STEPS_LOOP_COUNT
MOVLW H'00'
MOVWF FREQ_STEPS
MOVLW H'00'
MOVWF FREQ_CYCLES_DONE
MOVLW H'00'
MOVWF CURRENT_STEPS
MOVLW H'01'
MOVWF SPEED_REG1
MOVLW H'01'
MOVWF SPEED_REG2
MOVLW H'00'
MOVWF SPEED_REG3
; MOVLW H'00'
; MOVWF TIMING_REG
MOVLW H'00'

```

```

MOVWF    OVERFLOW_REG
MOVLW    H'FF'
MOVWF    POSITION_COUNTER
MOVLW    H'00'
MOVWF    MOTION_FLAGS_REG
MOVLW    H'00'
MOVWF    TIMING_FLAGS_REG
MOVLW    H'FF'
MOVWF    MISSED_T_REG
;-----TIMERS Setup-----

;-----INTERRUPTS Setup-----
BANKSEL   PIE1
CLRF      PIE1
;   BSF    PIE1, 4
BSF        PIE1, 5
BANKSEL   PIE2
CLRF      PIE2
BANKSEL   PIE3
CLRF      PIE3
BANKSEL   PIE4
CLRF      PIE4
BANKSEL   PIR1
CLRF      PIR1
BANKSEL   PIR2
CLRF      PIR2
BANKSEL   PIR3
CLRF      PIR3
BANKSEL   PIR4
CLRF      PIR4
BANKSEL   IOCBP
CLRF      IOCBP
BANKSEL   IOCBN
CLRF      IOCBN
BANKSEL   IOCAP
CLRF      IOCAP
BANKSEL   IOCAN
CLRF      IOCAN
MOVLW     H'02'
BANKSEL   IOCCP
MOVWF     IOCCP
BANKSEL   IOCCN
CLRF      IOCCN
BANKSEL   INTCON
CLRF      INTCON
;   BSF    INTCON, 3
BSF        INTCON, 6
BSF        INTCON, 7
;   BANKSEL PORTA
;   BSF    PORTB, 0
;   CLRF   PORTA
;   CLRF   PORTB
;   CLRF   PORTC
RETURN

```

Timing_Cycles_1.inc

```

;*****
;
; Filename: Timing_Cycles.INC
; Date: September 17 2025
; File Version: 1
; Author: RAHIEL RODRIGUEZ
; Company: Idaho State University
; Description: Firmware for Seting up the Frequency Cycles for FREQ_TC
;
;*****
;*****
;
; Revision History:
; 1: INITIAL FILE TO SETUP A PIC16F883 SPECIFICS ON SETUP SHOULD BE
; LISTED HERE AS MODIFIED
; STARTED March 11 2025
;
;
;*****
;*****

SIGNAL_DELAY
    MOVLW    0X04    ;Sets the delay time to generate the PW
    MOVWF    COUNTER2
TIMING_LOOP2
    MOVLW    VALUE_9K
    MOVWF    COUNTER1
TIMING_LOOP1
    DECFSZ   COUNTER1
    GOTO     TIMING_LOOP1
;    NOP
    DECFSZ   COUNTER2
    GOTO     TIMING_LOOP2
    MOVLW    0X00
    MOVWF    COUNTER2
    MOVLW    0X00
    MOVWF    COUNTER1
;    MOVLW    0X00
;    MOVWF    TIMING_REG
    RETURN

;SET LOW LOOP
CALIBRATION_DELAY
    MOVLW    0X04    ;Sets the delay time to generate the PS
    MOVWF    COUNTER2
CALIBRATION_TIMING_LOOP2
    MOVWF    TIMING_REG
    MOVWF    COUNTER1
CALIBRATION_TIMING_LOOP1
    DECFSZ   COUNTER1
    GOTO     CALIBRATION_TIMING_LOOP1
    DECFSZ   COUNTER2
    GOTO     CALIBRATION_TIMING_LOOP2
    MOVLW    0X00
    MOVWF    COUNTER2
    MOVLW    0X00
    MOVWF    COUNTER1
    RETURN

SENSOR_CHECK_DELAY    ;Sensor Pulsing Delay Loop
    MOVLW    0X0F
    BANKSEL  COUNTER3
    MOVWF    COUNTER3
LOOP3_SENSOR
    MOVLW    0X0F
    MOVWF    COUNTER2
LOOP2_SENSOR
    MOVWF    TIMING_REG
    MOVWF    COUNTER1
LOOP1_SENSOR
    DECFSZ   COUNTER1
    GOTO     LOOP1_SENSOR
    DECFSZ   COUNTER2
    GOTO     LOOP2_SENSOR

```

```

    DECFSZ    COUNTER3
    GOTO      LOOP3_SENSOR
    RETURN

HOME_PULSE_DELAY      ;Home Pulsing Timing Loop
    MOVLW    0x01
    BANKSEL   COUNTER3
    MOVWF    COUNTER3
LOOP3_HOME
    MOVLW    0x10
    MOVWF    COUNTER2
LOOP2_HOME
    MOVLW    0xA6
    MOVWF    COUNTER1
LOOP1_HOME
    DECFSZ    COUNTER1
    GOTO      LOOP1_HOME
    DECFSZ    COUNTER2
    GOTO      LOOP2_HOME
    DECFSZ    COUNTER3
    GOTO      LOOP3_HOME
    RETURN

SPEED_CHECK
    BTFSC     SPEED_REG1,0
    GOTO      LOAD_2K
    BTFSC     SPEED_REG1,1
    GOTO      LOAD_2.5K
    BTFSC     SPEED_REG1,2
    GOTO      LOAD_3K
    BTFSC     SPEED_REG1,3
    GOTO      LOAD_3.5K
    BTFSC     SPEED_REG1,4
    GOTO      LOAD_4K
    BTFSC     SPEED_REG1,5
    GOTO      LOAD_4.5K
    BTFSC     SPEED_REG1,6
    GOTO      LOAD_5K
    BTFSC     SPEED_REG1,7
    GOTO      LOAD_5.5K
    BTFSC     SPEED_REG2,0
    GOTO      LOAD_6K
    BTFSC     SPEED_REG2,1
    GOTO      LOAD_6.5K
    BTFSC     SPEED_REG2,2
    GOTO      LOAD_7K
    BTFSC     SPEED_REG2,3
    GOTO      LOAD_7.5K
    BTFSC     SPEED_REG2,4
    GOTO      LOAD_8K
;    BTFSC     SPEED_REG2,5
;    GOTO      LOAD_8.5K
;    BTFSC     SPEED_REG2,6
;    GOTO      LOAD_9K
;    GOTO      LOAD_9.5K

LOAD_2K
    MOVLW    VALUE_2K
    MOVWF    TIMING_REG
    RETURN
LOAD_2.5K
    MOVLW    VALUE_2.5K
    MOVWF    TIMING_REG
    RETURN
LOAD_3K
    MOVLW    VALUE_3K
    MOVWF    TIMING_REG
    RETURN
LOAD_3.5K
    MOVLW    VALUE_3.5K
    MOVWF    TIMING_REG
    RETURN
LOAD_4K
    MOVLW    VALUE_4K
    MOVWF    TIMING_REG
    RETURN

```

```

LOAD_4.5K
    MOVLW    VALUE_4.5K
    MOVWF    TIMING_REG
    RETURN
LOAD_5K
    MOVLW    VALUE_5K
    MOVWF    TIMING_REG
    RETURN
LOAD_5.5K
    MOVLW    VALUE_5.5K
    MOVWF    TIMING_REG
    BTFSS    MOTION_FLAGS_REG,1
    RETURN
    BSF      MOTION_FLAGS_REG,3
    BCF      MOTION_FLAGS_REG,1
    CLRF     SPEED_REG1
    RETURN
LOAD_6K
;    BSF      PORTB,0
    MOVLW    VALUE_6K
    MOVWF    TIMING_REG
    BTFSS    MOTION_FLAGS_REG,4
    RETURN
    BSF      MOTION_FLAGS_REG,2
    BCF      MOTION_FLAGS_REG,4
    BSF      SPEED_REG1,7
    RETURN
LOAD_6.5K
    MOVLW    VALUE_6.5K
    MOVWF    TIMING_REG
    RETURN
LOAD_7K
    MOVLW    VALUE_7K
    MOVWF    TIMING_REG
    RETURN
LOAD_7.5K
    MOVLW    VALUE_7.5K
    MOVWF    TIMING_REG
    RETURN
LOAD_8K
    MOVLW    VALUE_8K
    MOVWF    TIMING_REG
    RETURN
LOAD_8.5K
    MOVLW    VALUE_8.5K
    MOVWF    TIMING_REG
    RETURN
LOAD_9K
    MOVLW    VALUE_9K
    MOVWF    TIMING_REG
    RETURN
LOAD_9.5K
    MOVLW    VALUE_9.5K
    MOVWF    TIMING_REG
    RETURN

COMPENSATION_ROUTINE
    BANKSEL  PORTB
    MOVLW    H'01'
    SUBWF    STEPS_OBJ,0
    BTFSC    STATUS,2
    GOTO     LOAD_1POS_MULT
    MOVLW    H'02'
    SUBWF    STEPS_OBJ,0
    BTFSC    STATUS,2
    GOTO     LOAD_2POS_MULT
    MOVLW    H'03'
    SUBWF    STEPS_OBJ,0
    BTFSC    STATUS,2
    GOTO     LOAD_3POS_MULT
    MOVLW    H'04'
    SUBWF    STEPS_OBJ,0
    BTFSC    STATUS,2
    GOTO     LOAD_4POS_MULT
    MOVLW    H'05'
    SUBWF    STEPS_OBJ,0

```

```

    BTFSC     STATUS,2
    GOTO      LOAD_5POS_MULT
    MOVLW     H'06'
    SUBWF     STEPS_OBJ,0
    BTFSC     STATUS,2
    GOTO      LOAD_6POS_MULT
    MOVLW     H'07'
    SUBWF     STEPS_OBJ,0
    BTFSC     STATUS,2
    GOTO      LOAD_7POS_MULT
    MOVLW     H'08'
    SUBWF     STEPS_OBJ,0
    BTFSC     STATUS,2
    GOTO      LOAD_8POS_MULT

LOAD_1POS_MULT
    MOVLW     H'01'
    MOVWF     COMP_MULT_COUNT
    RETURN

LOAD_2POS_MULT
    MOVLW     H'06'
    MOVWF     COMP_MULT_COUNT
    RETURN

LOAD_3POS_MULT
    MOVLW     H'08'
    MOVWF     COMP_MULT_COUNT
    RETURN

LOAD_4POS_MULT
    MOVLW     H'09'
    MOVWF     COMP_MULT_COUNT
    RETURN

LOAD_5POS_MULT
    MOVLW     H'0A'
    MOVWF     COMP_MULT_COUNT
    RETURN

LOAD_6POS_MULT
    MOVLW     H'0B'
    MOVWF     COMP_MULT_COUNT
    RETURN

LOAD_7POS_MULT
    MOVLW     H'0B'
    MOVWF     COMP_MULT_COUNT
    RETURN

LOAD_8POS_MULT
    MOVLW     H'0B'
    MOVWF     COMP_MULT_COUNT
    RETURN

```

References

- 4attention GmbH & Co. KG (n.d.). Meet the RoboKeeper: Sports fun in a new dimension.
RoboKeeper. Retrieved November 4, 2024, from <https://www.robokeeper.com/en/>
- "Robots As Athletes: Can AI Beat Renowned NBA Shooting Coach?" TechWire Asia, 21 Jul. 2023, techwireasia.com/2023/07/ai-robots-the-next-big-contenders-in-sports/. Accessed 31 Mar. 2025.
- Hamilton, Eric. "Playing Sports against Robotic Opponents Makes Our Brains Work Harder." University of Florida News, 10 Apr. 2023, news.ufl.edu/2023/04/robotic-sports-neuroscience/. Accessed 31 Mar. 2025.
- Brown, Sara. "A New Study Measures the Actual Impact of Robots on Jobs. It's Significant." MIT Sloan, 29 Jul. 2020, mitsloan.mit.edu/ideas-made-to-matter/a-new-study-measures-actual-impact-robots-jobs-its-significant. Accessed 31 Mar. 2025.